RETRO DEV KIT APPLE 1 — LE PÉCHÉ ORIGINEL

Manuel complet

L'émulateur, la cassette ACI, la carte GEN2, le débogueur — avec les guides complets de l'Integer BASIC et de l'assembleur 6502



VERSION 1.0 NOVEMBRE 2026 ÉDITION FRANÇAISE

Sommaire

L'ÉMULATEUR

1 À propos de ce manuel 7

1.1 Comment ce manuel est organisé 7
1.2 Conventions 7
1.3 Ce qu'il vous faut 8
1.4 Les grands chemins 9

2 La fenêtre et la barre d'outils 10

2.1 Vue d'ensemble 10
2.2 La barre d'outils, bouton par bouton 11
2.3 La barre d'état 13
2.4 Le clavier de l'Apple-1 14
2.5 L'état de la machine 15
2.6 Le programmateur d'EPROM 15

3 L'écran : zoom, phosphore et effet vintage 17

3.1 L'écran d'origine 17
3.2 La rangée d'affichage 17
3.3 L'écran vintage 18

4 La bibliothèque 21

4.1 Les dossiers 21
4.2 Lancer un programme 22
4.3 La fiche d'un programme 23
4.4 Les cassettes reconnues 24
4.5 Les projets GitHub 25
4.6 Le dossier « Art ASCII » 26

5 La cassette et la carte ACI 27

5.1 Les formats de fichiers 27
5.2 Le lecteur virtuel 27
5.3 Son et oscilloscope 28
5.4 Enregistrer depuis l'Apple-1 émulé 29
5.5 Comment fonctionne l'ACI 29
5.6 Les commandes de l'ACI 30
5.7 Vers un vrai Apple-1 30
5.8 Quel niveau faut-il ? Les modèles d'entrée 31
5.9 Numériser une vraie cassette ou un enregistrement de l'Apple-1 32

6 Les cartes graphiques : GEN2 et P-LAB 33

6.1 Brancher la carte 33
6.2 Les commutateurs 34
6.3 Les modes et la mémoire 34
6.4 Les programmes d'essai 35
6.5 Ce qui est approché 36
6.6 La carte graphique P-LAB 36

7 L'éditeur 40

7.1 Les onglets et les fichiers 40
7.2 Rechercher, remplacer et le menu du clic droit 41
7.3 Un programme BASIC 42
7.4 Une source 6502 43

8 Le débogueur, le désassemblage et le dump **45**
8.1 Le panneau du débogueur ... 45
8.2 Les points d'arrêt ... 45
8.3 Déboguer un programme BASIC ... 46
8.4 Déboguer un programme 6502 ... 47
8.5 Le dump ... 47
8.6 En ligne de commande ... 48

9 Photo → ASCII **49**
9.1 Ouvrir une image ... 49
9.2 Les réglages ... 49
9.3 Le programme produit ... 50
9.4 En ligne de commande ... 51

10 La disposition des panneaux **52**
10.1 Déplacer un panneau ... 52
10.2 La disposition de départ ... 52
10.3 L'assistant de démarrage ... 52
10.4 La taille du texte de chaque fenêtre ... 53
10.5 Ce qui est retenu d'un lancement à l'autre ... 53
10.6 Quand l'écran est plus grand que son volet ... 53
10.7 Détacher un panneau ... 54

11 Les préférences **55**

12 La ligne de commande **58**
12.1 L'émulateur en terminal ... 58
12.2 Les conversions ... 59
12.3 BASIC et assembleur ... 60
12.4 Art ASCII ... 60
12.5 PROM ... 61
12.6 L'interface graphique ... 61

13 Dépannage et questions fréquentes **63**
13.1 La fenêtre ne s'ouvre pas ... 63
13.2 Les touches n'arrivent pas à l'Apple-1 ... 63
13.3 « Integer BASIC introuvable » ... 63
13.4 Un programme BASIC s'arrête avec STOPPED AT ... 63
13.5 Le programme ne se lance pas après le chargement ... 63
13.6 La cassette ne se lit pas sur le vrai Apple-1 ... 63
13.7 L'écran est plein de @ en inverse ... 64
13.8 Les caractères sortent lentement ... 64
13.9 Le son de la cassette est muet ... 64
13.10 Le projet GitHub ne se construit pas ... 64
13.11 Ce que l'émulateur ne fait pas ... 64

14 Installer une ROM ou l'Integer BASIC original **65**
14.1 Ce qui est déjà là ... 65
14.2 Integer BASIC original ... 65

L'INTEGER BASIC

15 Premiers pas en BASIC **67**
15.1 Deux BASIC : le RDK G2 Basic et l'original ... 67
15.2 Démarrer le BASIC ... 68
15.3 Mode direct et mode programme ... 68
15.4 Corriger une ligne ... 69
15.5 Un premier programme ... 69

15.6 Les commandes ... 69
15.7 Trois programmes à taper ... 70

16 Référence des instructions et des fonctions 72

16.1 Les instructions ... 72
16.2 Les fonctions ... 73
16.3 Les opérateurs ... 73
16.4 Les extensions du RDK G2 Basic ... 74
16.5 Les mots-clés sans effet ... 75

17 Variables, chaînes, tableaux et mémoire 76

17.1 Les nombres ... 76
17.2 Les noms de variables ... 76
17.3 Les chaînes ... 76
17.4 Les tableaux ... 77
17.5 La mémoire du BASIC ... 77
17.6 Mélanger BASIC et langage machine ... 79

18 Messages d'erreur et limites 80

18.1 Les messages ... 80
18.2 Les limites ... 81
18.3 Gagner de la mémoire ... 81

19 Les bugs connus et leurs contournements 82

19.1 Sortir d'une boucle FOR par GOTO ... 82
19.2 L'opérateur ^ bloque la machine ... 82
19.3 Le message « PPED AT » ... 83
19.4 IF ... THEN ne conditionne que la première instruction ... 83
19.5 −32768 ... 83
19.6 ASC(ne fonctionne pas ... 84
19.7 Les mots-clés graphiques sans effet ... 84
19.8 Le bug des 251 caractères de l'Apple II ... 84
19.9 STOPPED AT 65535 ... 85
19.10 USR plante la machine ... 85
19.11 HIMEM= ne peut pas baisser ... 85
19.12 Le deux-points refusé en mode direct ... 85
19.13 INPUT ajoute toujours « ? » ... 85
19.14 La boucle FOR passe au moins une fois ... 85
19.15 Une touche arrête le programme ... 86
19.16 Résumé ... 86

20 Porter un programme Applesoft ou Microsoft BASIC 87

20.1 Les équivalences ... 87
20.2 La méthode ... 88
20.3 Un exemple de portage ... 88
20.4 L'expérience des portages ... 89

L'ASSEMBLEUR 6502

21 Le 6502 en bref 91

21.1 Les registres ... 91
21.2 Les modes d'adressage ... 91
21.3 Les instructions par famille ... 92
21.4 Un 6502 NMOS, pas un 65C02 ... 93
21.5 La carte mémoire en une figure ... 93

22 L'assembleur du devkit 94

22.1 Le cycle de travail ... 94
22.2 Le listing et les symboles ... 95
22.3 La ligne de commande ... 96

22.4 La syntaxe ... 97
22.5 Les directives ... 98
22.6 Les macros, .repeat et l'assemblage conditionnel ... 99
22.7 Les messages d'erreur ... 99
22.8 Compatibilité avec ca65 et xa ... 100

23 Le clavier, l'écran et le Woz Monitor 101

23.1 Les quatre registres du PIA ... 101
23.2 Lire une touche ... 102
23.3 Écrire un caractère ... 102
23.4 Les routines du Woz Monitor ... 103
23.5 Lancer et terminer un programme ... 103
23.6 Le PIA, la cassette et la carte couleur ... 104

24 Exemples commentés 105

24.1 `hello.s` : afficher un message ... 106
24.2 `clavier.s` : l'écho des touches ... 107
24.3 `hexa.s` : un vidage façon moniteur ... 107
24.4 `decimal.s` : afficher un nombre en décimal ... 108
24.5 `jeu.s` : devinez le chiffre ... 110
24.6 `macros.s` : laisser travailler l'assembleur ... 112
24.7 `gen2-lores.s` : la couleur en basse résolution ... 113
24.8 `gen2-hires.s` : la haute résolution ... 115

25 Astuces, pièges et limites 118

25.1 Où placer un programme ... 118
25.2 Écrire du code qui tient dans 256 octets de saut ... 118
25.3 Les chaînes et le bit 7 ... 118
25.4 Attendre un peu ... 119
25.5 Déboguer efficacement ... 119
25.6 Vers le véritable Apple-1 ... 120
25.7 Les limites de l'assembleur intégré ... 120

ANNEXES

A Carte mémoire 122

B Les commandes du Woz Monitor 124

C Les temps de l'ACI 125

D Le jeu de caractères du 2513 126

E Les raccourcis clavier 127

F Pièges et comportements à connaître 128

G Licence et crédits 130

PARTIE I

L'émulateur

La fenêtre, l'écran, la bibliothèque, la cassette ACI, les cartes graphiques GEN2 et P-LAB, le débogueur et la ligne de commande

CHAPITRE 1

À propos de ce manuel

Retro Dev Kit Apple 1 — *« Le péché originel » — reproduit l'Apple-1 de 1976 au cycle près : le 6502, le terminal vidéo, le clavier, la carte cassette ACI et, en option, la carte couleur GEN2 d'Uncle Bernie et la carte graphique P-LAB à TMS9918A. Il est livré avec son propre BASIC libre, le* ***RDK G2 Basic****. Autour de la machine, il ajoute un atelier complet : bibliothèque de programmes, éditeur, assembleur, débogueur et export vers un vrai Apple-1.*

Ce manuel décrit tout l'émulateur, version 1.0 (la première version publique ; les numéros 7.x désignaient les versions de test). Il complète le *Guide de démarrage rapide*, qui suffit pour installer le programme et lancer une première cassette. Ici, chaque panneau, chaque bouton et chaque commande sont expliqués, avec des exemples que vous pouvez taper tels quels.

1.1 Comment ce manuel est organisé

Partie I — L'émulateur

La fenêtre et ses panneaux, l'écran vintage, la bibliothèque, la cassette et l'ACI, les cartes graphiques GEN2 et P-LAB, l'éditeur, le débogueur, l'art ASCII, les préférences, la ligne de commande et le dépannage.

Partie II — L'Integer BASIC

Le BASIC de Steve Wozniak et son clone libre RDK G2 Basic : toutes les instructions, les fonctions et les opérateurs, la mémoire, les messages d'erreur et les bugs connus, avec leurs contournements.

Partie III — L'assembleur 6502

Le 6502 depuis le début, l'assembleur intégré et ses directives, les routines du Woz Monitor, des exemples complets et les pièges à éviter.

Annexes

Carte mémoire, commandes du Woz Monitor, format de l'ACI, jeu de caractères du 2513, raccourcis clavier, puis l'index des commandes.

1.2 Conventions

Les touches sont écrites ainsi : Ctrl+R, RETURN. Les boutons de l'interface sont en gras : ▶ **Lancer**. Les adresses sont en hexadécimal, précédées de `$` comme dans les documents d'époque : `$FFEF`. Le Woz Monitor, lui, les écrit sans `$` : `FFEF`.

Les sessions sur l'écran de l'Apple-1 ont un fond sombre. Ce que **vous** tapez est en blanc, ce que la machine répond est en vert :

```
300R

0300: A2
BONJOUR, APPLE-1 !
```
WOZ MONITOR

Quatre sortes d'encadrés jalonnent le texte :

> ✦ **ASTUCE**
> Un raccourci, une meilleure façon de faire.

> ! **ATTENTION**
> Un piège qui fait perdre du temps ou des données.

> ▣ **SUR LA VRAIE MACHINE**
> Ce qui se passe sur un vrai Apple-1, et ce qui change par rapport à l'émulateur.

> **BUG CONNU**
> Un défaut de la ROM d'origine, avec son contournement. Chaque bug porte un statut : VÉRIFIÉ (reproduit dans l'émulateur avec le vrai Integer BASIC) ou NON VÉRIFIÉ (cité d'après une source, sans essai).

Tous les exemples de ce manuel ont été essayés dans l'émulateur. Les programmes 6502 de la partie III sont dans le dossier `docs/exemples/asm/` de la distribution. Ils donnent exactement les mêmes octets avec l'assembleur intégré et avec ca65.

1.3 Ce qu'il vous faut

L'émulateur fonctionne sous Linux sans rien installer. Le Woz Monitor et la ROM de l'ACI sont intégrés, ainsi que le **RDK G2 Basic**, un BASIC libre compatible avec l'Integer BASIC (chapitre 14) : le bouton **BASIC** marche tout de suite, sur une installation neuve, sans fichier à chercher. L'**Integer BASIC** d'Apple (1976) n'est **pas** fourni, parce qu'il reste protégé par le droit d'auteur. Si vous voulez l'original, utilisez votre propre copie (par exemple la cassette `BASIC.wav` de la collection « Apple1-Cassettes » d'archive.org) et choisissez-la dans ⚙ **Préférences → Avancé → BASIC**.

> ! **LES ROM ET LES CASSETTES RESTENT CHEZ VOUS**
> L'émulateur ne livre ni l'Integer BASIC d'Apple ni les cassettes du commerce. Sa table de reconnaissance ne contient que des tailles, des empreintes et des paramètres. Gardez vos copies dans votre bibliothèque personnelle.

> **LICENCE**
> Retro Dev Kit Apple 1 est un logiciel libre sous **GNU GPL 3.0** (sans garantie). Certaines parties gardent leur propre licence MIT : le RDK G2 Basic, ainsi que les tests et les glyphes GEN2 d'Uncle Bernie. Voir `LICENSE` et `NOTICE` dans l'archive, et l'annexe « Licence et crédits ».

1.4 Les grands chemins

Quatre manières d'utiliser l'émulateur reviennent sans cesse. Elles sont détaillées plus loin, chacune avec son schéma :

Je veux…	Le chemin	Voir
jouer à un programme d'époque	bibliothèque → ▶	chapitre 4
écrire un programme BASIC	📝 **Éditeur** → ▶ **Lancer** → 📼 **Exporter WAV**	partie II
écrire un programme 6502	📝 **Éditeur** (`.s`) → 📃 **Listing** → 🐛 **Débogueur**	partie III
envoyer un programme sur un vrai Apple-1	📼 **Exporter WAV** (`.aiff`) → smartphone → ACI	chapitre 5

CHAPITRE 2

La fenêtre et la barre d'outils

2.1 Vue d'ensemble

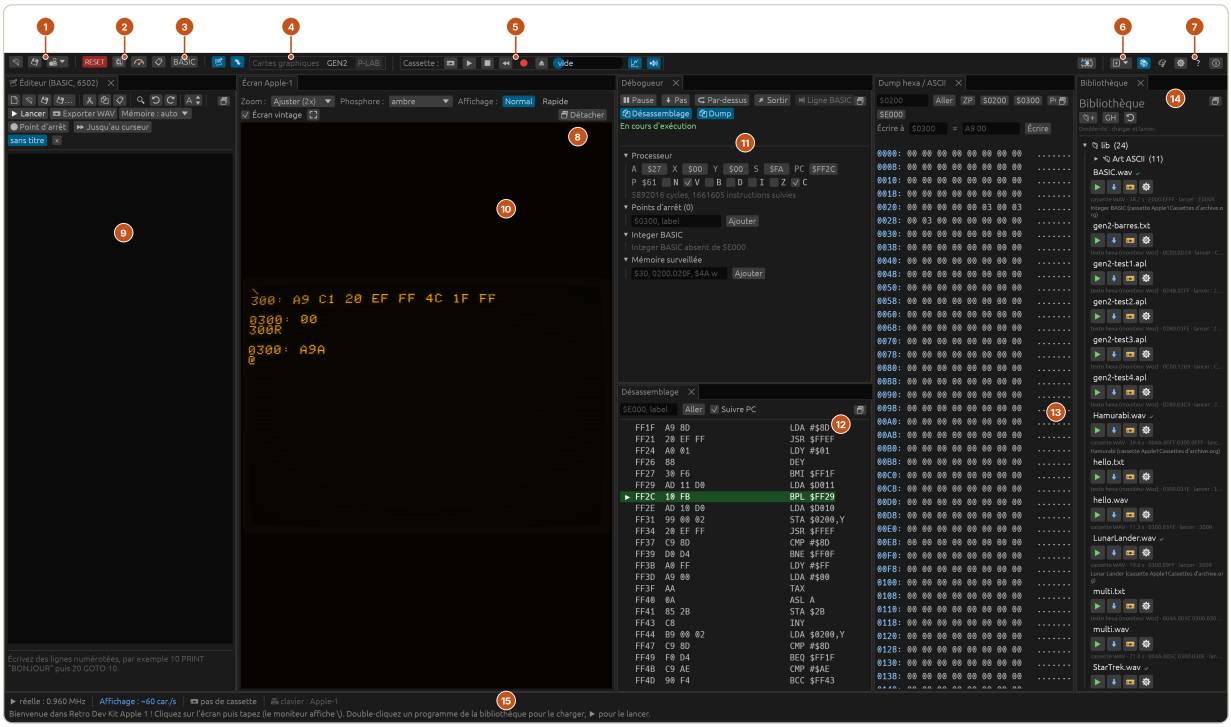


Figure 1 — La disposition par défaut : tous les panneaux ouverts et ancrés.

1. **Charger, Sauver, État** : vidages hexadécimaux `.txt`, binaires `.bin` et `.prg` ; Sauver écrit aussi des cassettes `.wav` / `.aiff` ; État enregistre et restaure la machine.
2. **RESET, Effacer l'écran, Vitesse (jauge), Coller** : les boutons de la machine (Ctrl+R, Ctrl+L, F3, Ctrl+V).
3. **BASIC** : charge le RDK G2 Basic intégré en `$E000` (ou votre Integer BASIC, si vous l'avez choisi).
4. **Fenêtres** : le menu ⊞ **Disposition** ▾, puis un bouton par panneau (Écran monochrome, Écran couleur, Éditeur, Débogueur, Désassemblage, Dump, Bibliothèque, ASCII).
5. **Cartes graphiques** : **GEN2** et **P-LAB** branchent ou débranchent les cartes.
6. **Cassette** : **Insérer** une cassette, puis lecture, stop, rembobiner, enregistrer, éjecter ; oscilloscope et son.
7. **EPROM**, ⚙ **Préférences**, **?** (bulles d'aide détaillées, aussi F1) et **ℹ** (à propos, raccourcis).
8. **Rangée d'affichage de l'écran** : zoom (1x à 8x ou Ajuster), phosphore, affichage **Normal** ou **Rapide**, écran vintage, plein écran, **Détacher**.
9. **Éditeur** : programmes BASIC et sources 6502 en onglets.
10. **Écran Apple-1** : 40 × 24 caractères ; cliquez dessus pour taper, clic droit pour Copier, Coller, Effacer l'écran et Reset.
11. **Débogueur** : pause, pas à pas, registres, points d'arrêt, variables.
12. **Désassemblage** : le code autour du compteur ordinal (PC).

13. **Dump** : la mémoire en hexadécimal et en ASCII, modifiable.
14. **Bibliothèque** : vos programmes, chacun avec ses petites icônes ▶ ⬇ 📼 ⚙.
15. **Barre d'état** : vitesse, débit d'affichage, cassette, destination du clavier ; dessous, le dernier message.

Tous les panneaux (écran, éditeur, débogueur, bibliothèque…) sont des onglets que vous pouvez déplacer, regrouper ou détacher dans leur propre fenêtre : c'est l'objet du chapitre 10. Au premier lancement, la fenêtre s'ouvre agrandie et une fenêtre **Votre espace de travail** propose cinq dispositions (chapitre 10) ; la taille et la position de la fenêtre et l'arrangement de ses panneaux sont retenus, et retrouvés tels quels au lancement suivant.

2.2 La barre d'outils, bouton par bouton

La barre d'outils tient sur une seule rangée : la machine et ses outils à gauche, le lecteur de cassette au milieu, les panneaux (groupe Fenêtres), les cartes et les réglages à droite. Quand la fenêtre est trop étroite, les boutons ne gardent que leur icône (le texte reste dans l'infobulle), ou la barre passe sur plusieurs lignes si vous décochez **Barre compacte** dans les préférences.



Figure 2 — La barre d'outils jusqu'au groupe Cartes graphiques : fichiers, machine, fenêtres et cartes. Suit la zone Cassette (chapitre 5), puis les réglages.

Bouton	Ce qu'il fait
Charger	Ouvre un programme et le met en mémoire : un `.txt` (vidage du Woz Monitor) ou un `.hex` ; pour un `.bin`, l'adresse est demandée. Les cassettes (`.wav`, `.aiff`) ne passent pas par ici : elles s'insèrent avec 📼 **Insérer**, dans la zone Cassette.
Sauver	Demande une ou plusieurs plages (`0300.03FF`, `004A.00FF 0300.0FFF`…), puis un fichier. L'extension choisit le format : `.wav`, `.aiff`, `.txt` ou `.bin`.
RESET (bouton rouge)	Le bouton RESET de l'Apple-1 (Ctrl+R). Le 6502 et le PIA repartent, la mémoire et l'écran restent.
CL **Effacer l'écran**	Le bouton CLEAR SCREEN (Ctrl+L).
Jauge (Vitesse réelle / Vitesse rapide)	Bascule entre 1 MHz d'époque et la vitesse maximale (F3). Un cadran de compteur : allumé en bleu quand la vitesse rapide est active.
📋 **Coller**	Tape le contenu du presse-papiers au clavier de l'Apple-1, à la cadence d'un clavier (Ctrl+V). Idéal pour un vidage hexadécimal.
BASIC	Charge le **RDK G2 Basic** intégré en `$E000` (7 933 octets, puis `E000R` le démarre). Si vous avez choisi l'Integer BASIC d'origine dans ⚙ **Préférences → Avancé → BASIC**, c'est lui qui est chargé : une cassette, un vidage `.txt` ou un `.bin` de 4096 octets, cherché dans la bibliothèque (`BASIC.wav`, `integer_basic.bin`, tout nom qui contient « basic »), sinon demandé. Clic droit : choisir un autre fichier.
Fenêtres (groupe : ⊞ **Disposition** ▾ et un bouton par panneau)	Montre ou cache chaque panneau : **Écran monochrome** (le terminal de l'Apple-1), **Écran couleur** (l'écran de la carte graphique branchée, GEN2 ou P-LAB : le même bouton sert pour les deux ; sans carte branchée, il ouvre un choix pour en brancher une, et **P-LAB** affiche alors son écran), 📝 **Éditeur** (chapitre 7), 🐛 **Débogueur**, **Désassemblage**, **Dump** (chapitre 8), 📚 **Bibliothèque** et 🎨 **ASCII** (chapitre 9). Un bouton allumé signifie que le panneau est visible ; un panneau déjà affiché peut aussi se fermer avec la croix de son onglet.

Bouton	Ce qu'il fait
Cartes graphiques : GEN2 et P-LAB	Un groupe nommé. **GEN2** branche ou débranche la carte couleur d'Uncle Bernie (chapitre 6) ; **P-LAB** fait de même pour la carte P-LAB à TMS9918A (section 6.6). Un bouton est allumé tant que sa carte est branchée ; l'écran d'une carte apparaît dans son propre panneau, montré par le bouton **Écran couleur**.
📸 **État ▾**	Un seul bouton, avec un menu : **Enregistrer l'état…** (Ctrl+Maj+S), **Charger un état…** (Ctrl+Maj+O), puis les derniers états, chacun avec son titre et sa date (section 2.5).
EPROM (puce à fenêtre UV, à droite de la barre, avant ⊞ **Disposition**)	Ouvre le programmateur d'EPROM : lire, écrire ou vérifier une puce avec un programmateur `minipro` (section 2.6).
Cassette : 📼 **Insérer** ▶ ■ ⏪ ● ⏏	Le lecteur, réuni dans une zone. 📼 **Insérer** choisit un `.wav` ou un `.aiff` et le met dans le lecteur comme cassette virtuelle ; puis lecture, stop, rembobiner, enregistrer, éjecter. La zone montre le nom, la position et la durée.
📈 et 🔊	L'oscilloscope de la cassette, et le son du signal (chapitre 5).
⊞ **Disposition ▾** (premier bouton du groupe Fenêtres)	Un menu : **Par défaut** (tous les panneaux, comme au premier lancement), **Jeu** (écran Apple-1 et bibliothèque), **BASIC** (éditeur et écran), **Assembleur** (éditeur, écran et débogueur), **Débogage** (éditeur, débogueur, désassemblage et dump) ; puis **Enregistrer la disposition actuelle…**, qui demande un nom, et vos dispositions enregistrées par ordre alphabétique (clic droit : Renommer, Supprimer). Une coche marque le dernier choix ; tout est retenu dans `gui.json`.
⚙	Les préférences (chapitre 11).
?	Le mode aide (F1) : des bulles détaillées sur chaque commande, dans les 7 langues de l'interface.
i	« À propos », avec la liste des raccourcis clavier.

Fenêtres de fichiers. Ouvrir, Enregistrer sous et Exporter utilisent la fenêtre de fichiers propre à l'émulateur. Un bouton bleu 📁+ **Nouveau dossier**, en haut à gauche, crée un dossier dans le dossier affiché, avec une bulle d'aide pour l'expliquer ; tapez le nom, appuyez sur Entrée, et le nouveau dossier est prêt à recevoir votre fichier.

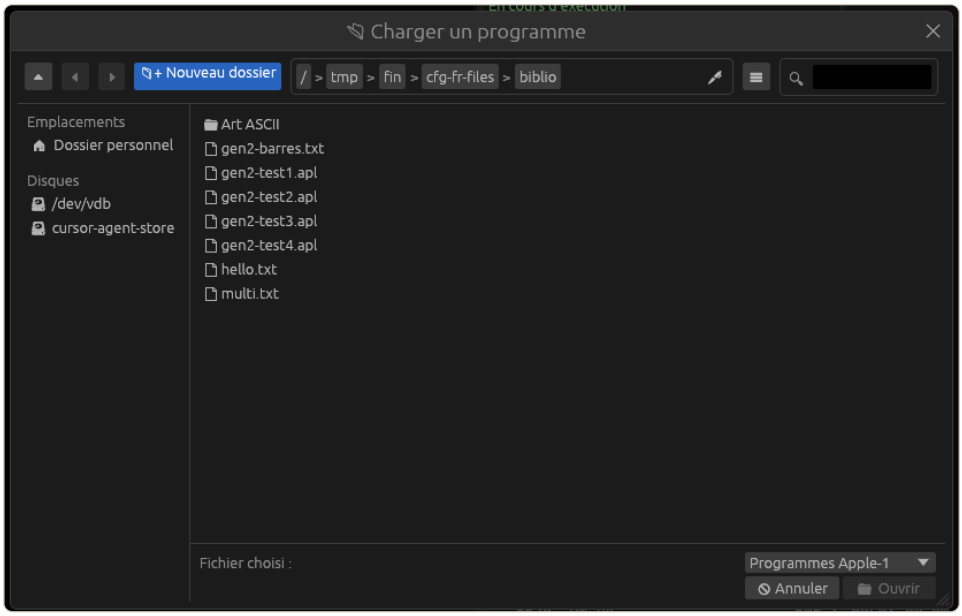


Figure 3 — La fenêtre Charger, avec le bouton 📁+ Nouveau dossier en haut à gauche.

Les réglages d'affichage ne sont pas dans la barre d'outils : chaque écran a sa propre **rangée d'affichage**, en haut de son panneau (section 3.2).



Figure 4 — La rangée d'affichage de l'écran de l'Apple-1 : zoom, phosphore, vitesse d'affichage, écran vintage et plein écran ; Détacher est au bout de la rangée.

> ✦ **LE MODE AIDE**
>
> Quand vous découvrez un panneau, allumez **?** : chaque bouton explique ce qu'il fait, au survol. Éteint, il ne reste que les petites infobulles habituelles (désactivables dans ⚙ → **Interface**). Le réglage est retenu.

2.3 La barre d'état



Figure 5 — La barre d'état pendant la lecture de BASIC.wav : vitesse, limite d'affichage, cassette, clavier, et le dernier message dessous.

- **Vitesse mesurée** : environ 0,960 MHz en vitesse réelle. C'est la fréquence effective de l'Apple-1, qui suspend le 6502 quatre cycles sur soixante-cinq pour rafraîchir sa mémoire dynamique.
- **Affichage ~60 car./s** : le terminal de l'Apple-1 n'affiche qu'un caractère par image, soit environ 60 par seconde. Cliquez pour basculer entre cette limite et l'affichage instantané, comme avec les boutons **Normal** / **Rapide** des rangées d'affichage.
- **Cassette** : nom, position et durée, ou « pas de cassette ».
- **Clavier** : à qui vont les touches (l'Apple-1, ou un champ de texte de l'interface).

2.4 Le clavier de l'Apple-1

Cliquez sur l'écran : les touches vont à l'Apple-1. Le clavier d'origine n'a que des majuscules, et l'émulateur convertit les minuscules.

Touche du PC	Pour l'Apple-1
lettres, chiffres, symboles	le caractère, en majuscule
Entrée	RETURN (code `$8D`)
Retour arrière	`_` : le Woz Monitor et le BASIC effacent le caractère précédent (il reste affiché, suivi de `_`)
Échap	ESC : annule la ligne en cours dans le Woz Monitor
Ctrl+lettre	le caractère de contrôle (`Ctrl+D` = `$84` …)
Ctrl+R / Ctrl+L	les boutons RESET et CLEAR SCREEN

Vous pouvez **sélectionner du texte à la souris** sur l'un ou l'autre écran : appuyez et glissez sur les caractères, la sélection est surlignée. Un **clic droit** sur l'un des écrans (celui de l'Apple-1 ou le moniteur GEN2) ouvre un petit menu : **Copier** met le texte sélectionné dans le presse-papiers (ou tout le texte de l'écran quand rien n'est sélectionné), **Coller (taper le texte)** tape le presse-papiers au clavier de l'Apple-1, comme si vous le saisissiez, puis **Effacer l'écran** et **Reset**.

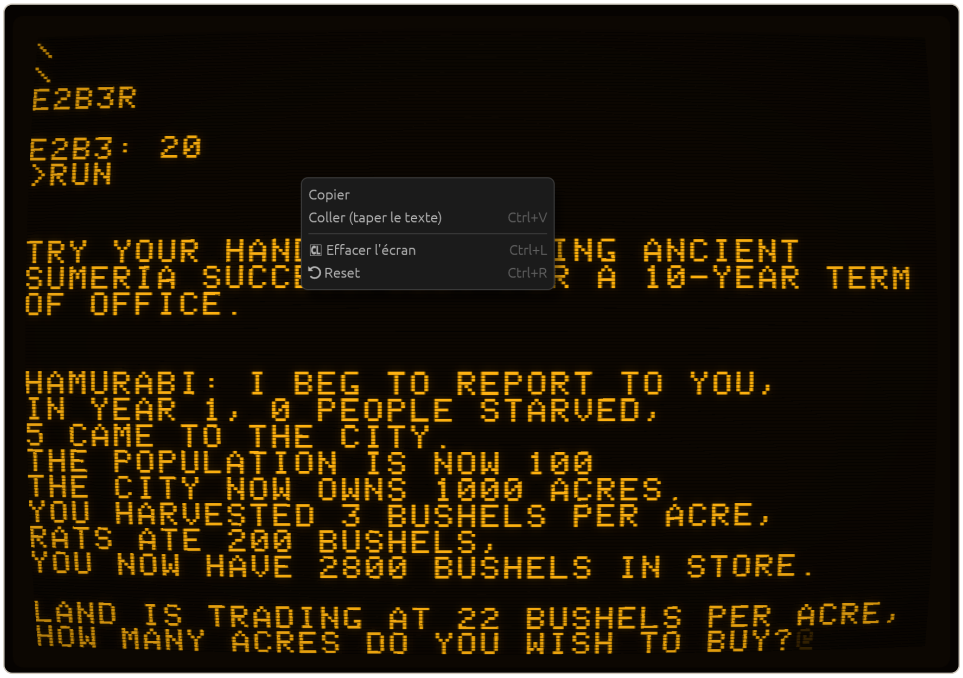


Figure 6 — Le menu du clic droit sur l'écran de l'Apple-1, pendant une partie de Hamurabi.

> ▣ **L'ÉCRAN DE L'APPLE-1 N'EFFACE RIEN**
>
> Le terminal ne sait qu'écrire un caractère et passer à la ligne. Il n'y a ni curseur mobile, ni effacement d'un caractère : la touche d'effacement affiche `_`, et le caractère reste à l'écran. C'est normal.

2.5 L'état de la machine

Le bouton 📸 **État** de la barre d'outils prend un instantané complet de la machine et le remet plus tard, exactement au même point : vous pouvez arrêter un jeu ou un programme en cours, fermer l'émulateur, et le reprendre le lendemain. Le bouton ouvre un menu :

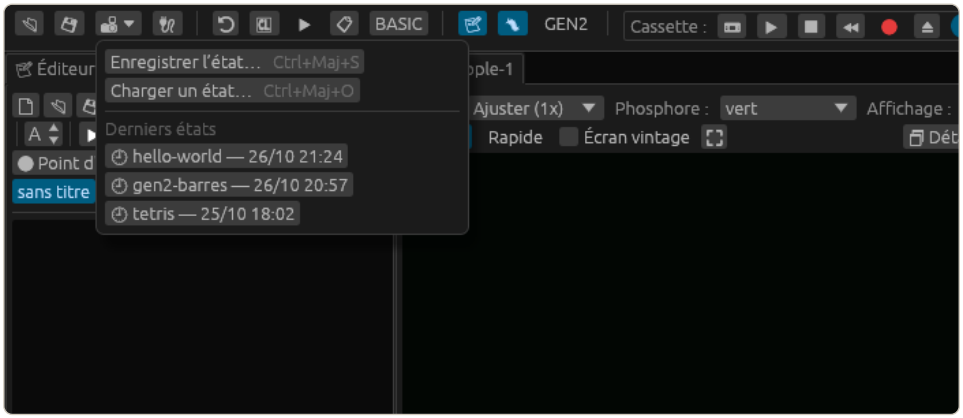


Figure 7 — Le menu État : enregistrer, charger, puis les derniers états avec leur date.

- **Enregistrer l'état…** (Ctrl+Maj+S) demande un nom de fichier et écrit un fichier `.a1state`. Le dossier proposé est `etats`, à côté du dossier de la bibliothèque.
- **Charger un état…** (Ctrl+Maj+O) remplace la machine par l'état choisi. La barre d'état montre le fichier et la valeur de PC.
- **Derniers états** liste les huit derniers, avec leur titre et la date de l'enregistrement ; un clic en charge un aussitôt. Si le fichier a été déplacé ou supprimé, un message le dit.

Un état contient la mémoire, les registres du 6502, le PIA, l'écran, la carte GEN2 si elle est branchée, et la cassette virtuelle avec sa position. Il ne contient ni vos préférences ni la bibliothèque. Un fichier illisible (abîmé, ou qui n'est pas un état) est refusé avec un message, et la machine reste intacte.

> ✦ **UN USAGE TYPIQUE**
>
> Enregistrez un état juste avant un passage difficile d'un jeu, ou à la fin d'une longue séance de BASIC : un état est aussi un moyen pratique de partager un programme en cours d'exécution avec un ami qui utilise la même version.

2.6 Le programmateur d'EPROM

EPROM, dans la barre d'outils, ouvre une fenêtre pour lire, écrire et vérifier une puce EPROM ou EEPROM depuis l'émulateur, par exemple pour graver la ROM du BASIC, un programme pour l'extension GEN2, ou le contenu de votre propre mémoire.

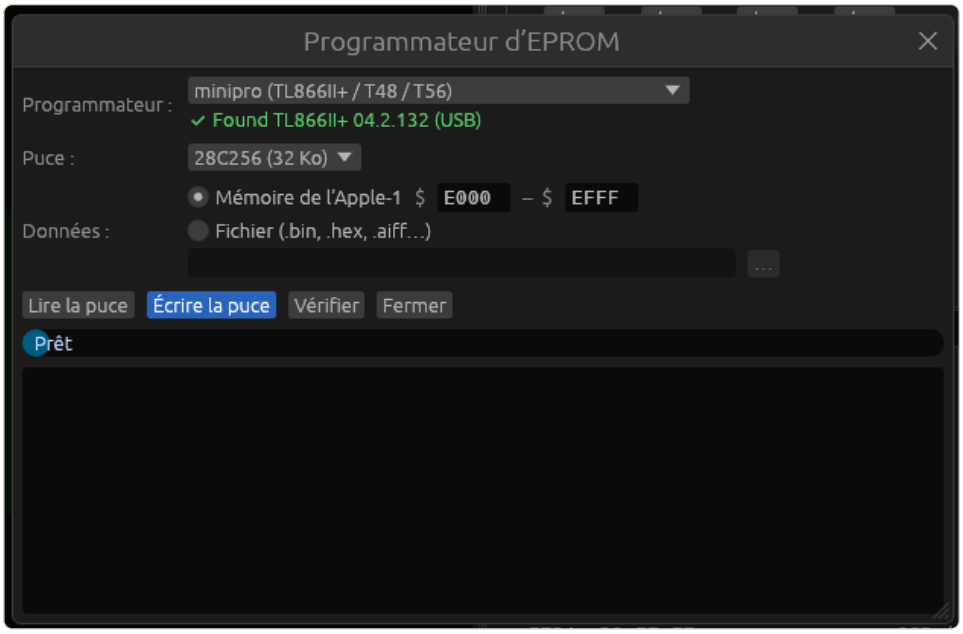


Figure 8 — Le programmateur d'EPROM : programmateur, puce, source des données et les trois opérations.

Seuls les programmateurs pilotés par l'outil libre `minipro` sont pris en charge : les XGecu **TL866II+**, **T48** et **T56**. Sous Debian, Ubuntu et Linux Mint, installez-le avec `sudo apt install minipro`, branchez le programmateur sur un port USB, puis appuyez sur **Détecter**. Si `minipro` manque, la fenêtre le dit et explique quoi faire ; rien d'autre n'est nécessaire du côté de l'émulateur.

Réglage	Rôle
Programmateur	le programmateur détecté, avec son firmware (par exemple « Found TL866II+ 04.2.132 (USB) »)
Puce	les puces courantes du monde Apple-1 : 2716, 2732, 2764, 27C64, 27C128, 27C256, 27C512, 28C64, 28C256, SST27SF256, SST27SF512, et la ROM masquée 23128 (lecture seule)
Données	la **mémoire de l'Apple-1** (une plage d'adresses, `$E000` – `$EFFF` par défaut), ou un **fichier** (`.bin`, `.hex`, `.aiff` …)

Trois boutons agissent sur la puce, et une barre de progression et un journal montrent ce qui se passe :

- **Lire la puce** lit toute la puce. Avec la mémoire ou un fichier existant comme données, le contenu est **comparé** et les différences sont listées : un fichier existant n'est jamais écrasé. Avec un nouveau nom de fichier, le contenu y est enregistré.
- **Écrire la puce** programme la puce avec les données choisies. La puce doit être vierge ou effaçable, et une confirmation du résultat s'affiche à la fin.
- **Vérifier** compare la puce avec les données choisies, octet par octet, et signale chaque différence (`$E010: puce $4C, données $A9`).

Fermer ferme la fenêtre.

La fenêtre ne modifie jamais la mémoire émulée : écrire et vérifier la lisent, et lire ne fait que la comparer ou remplir un fichier.

CHAPITRE 3

L'écran : zoom, phosphore et effet vintage

3.1 L'écran d'origine

L'Apple-1 affiche 40 colonnes × 24 lignes de majuscules. Chaque caractère est une matrice de 5 × 7 points dans une case de 7 × 8, lue dans la ROM du générateur de caractères **Signetics 2513** : l'émulateur utilise le contenu de cette ROM, point par point (annexe D). L'image fait donc 280 × 192 points.

Le curseur est un `@` qui clignote, piloté par un NE555 : 2,18 Hz, visible environ les deux tiers du temps. Sur certaines cartes, la tolérance du condensateur donne plutôt 1,7 Hz ; l'option `--cursor-hz 1.7` le reproduit.

3.2 La rangée d'affichage

Chaque écran a sa propre rangée de réglages, en haut de son panneau : l'écran de l'Apple-1, et le moniteur couleur quand la carte GEN2 est branchée (chapitre 6).

Réglage	Rôle
Zoom	une liste : `1x` à `8x` agrandissent l'image d'un facteur entier, les points restent carrés et nets ; **Ajuster** choisit le plus grand facteur qui tient dans le panneau (le facteur retenu est indiqué entre parenthèses). Au clavier : Ctrl++, Ctrl+−, Ctrl+0 (ajuster).
Phosphore	écran de l'Apple-1 seulement : le vert des moniteurs de l'époque, le blanc d'un téléviseur, ou l'ambre.
Affichage : Normal / Rapide	**Normal** affiche les caractères au rythme du vrai terminal, environ 60 par seconde ; **Rapide** affiche tout d'un coup. Ce réglage est commun : le changer sur un écran le change sur l'autre.
Écran vintage	l'effet de tube des années 1970 (section suivante) ; chaque écran a sa case.
⛶	plein écran (F11).
Détacher	met l'écran dans sa propre fenêtre (chapitre 10).

Chaque écran est **centré** dans son panneau, horizontalement et verticalement, avec sa légende centrée sous l'image. Si le zoom choisi est plus grand que le panneau, l'écran s'affiche au plus grand zoom qui tient au lieu d'être coupé. Quand la fenêtre est plus étroite que la rangée d'affichage, la rangée passe sur une deuxième ligne, et **Détacher** reste à l'extrémité droite de la dernière ligne.

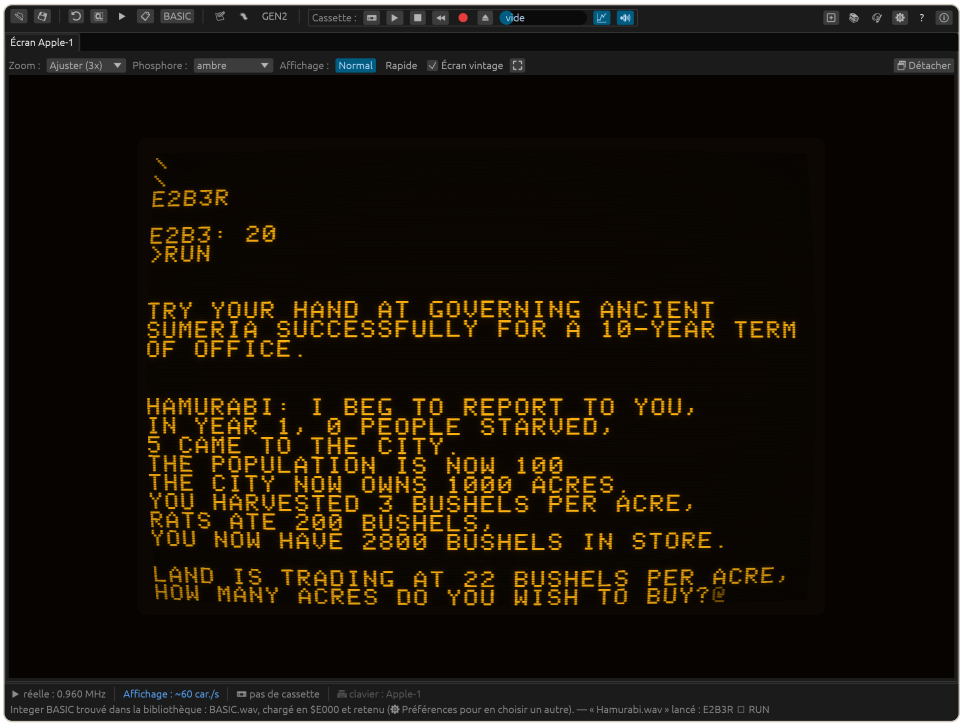


Figure 9 — Le phosphore ambre, avec l'écran vintage et une légère courbure.

3.3 L'écran vintage

La case **Écran vintage** de la rangée d'affichage imite un moniteur à tube des années 1970. L'effet principal est la fine ligne sombre entre deux lignes de balayage : chaque rangée de points devient une bande horizontale, comme sur la photo d'un vrai Apple-1. Cette ligne garde la même épaisseur en pixels de votre écran à tous les zooms : elle reste fine et nette, même en plein écran.

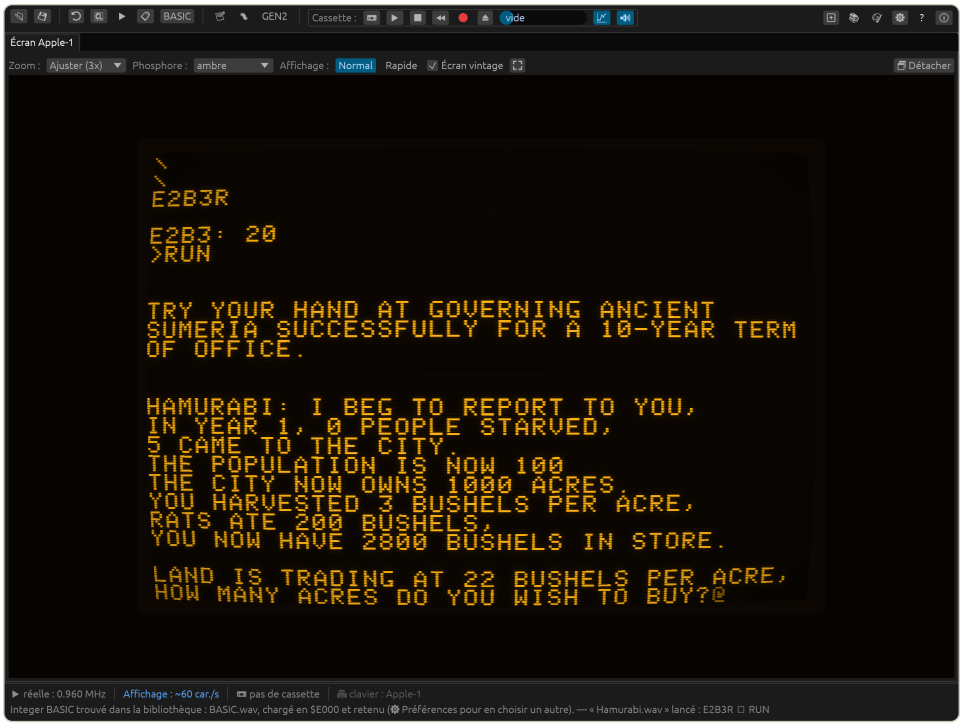


Figure 10 — L'écran vintage, avec un tube légèrement bombé (0,15) et les coins assombris, pendant une partie de Hamurabi.

Dans ⚙ **Préférences → Avancé → Écran vintage : réglages fins**, vous réglez l'**intensité** et choisissez les effets ; ces réglages sont communs aux deux écrans. Chaque curseur affiche sa **valeur numérique** à côté de lui (courbure, intensité…) :

Effet	Par défaut	Rôle
séparation des lignes de balayage	activé	la ligne sombre entre deux rangées de points
points adoucis horizontalement	activé	un léger flou, horizontal seulement, comme le faisceau
rémanence du phosphore	courte	les points s'éteignent en moins d'un dixième de seconde (un peu plus lentement en ambre) ; tout à gauche : aucune
halo lumineux	activé, discret	une lueur autour des points, qui restent nets
tube bombé et coins assombris	désactivé	avec un curseur **Courbure**
scintillement	désactivé	une très légère variation de luminosité
masque RVB du phosphore	activé	écran couleur de la carte GEN2 seulement : de fines bandes verticales rouges, vertes et bleues séparées par de minces traits sombres, comme sur un tube de téléviseur couleur. Il est visible à partir du zoom 3, et sa force suit l'**intensité**

L'effet peut aussi s'appliquer au moniteur couleur de la carte GEN2, par la case de sa propre rangée d'affichage. Il est calculé par le processeur, à la résolution de l'Apple-1, et seulement quand l'image change : il reste fluide même sans accélération graphique.

En ligne de commande : `--screening`, `--sans-screening`, `--gen2-screening` (moniteur GEN2), ou par exemple `--screening-options intensite=0.7,remanence=0.5,courbure=0.4,sans-halo` (valeurs de 0 à 1).

> ✦ **ÉCRAN DÉTACHÉ ET DEUXIÈME MONITEUR**
>
> **Détacher**, au bout de la rangée d'affichage, met l'écran de l'Apple-1 dans sa propre fenêtre. Placez-la sur un deuxième moniteur, en plein écran, avec l'effet vintage : le clavier va à l'Apple-1 dès que cette fenêtre a le focus. Avec **Ajuster**, l'image suit la taille de la fenêtre, par pas entiers (**Échelle entière**, dans la barre de titre de la fenêtre, le plus net) ou en la remplissant.

CHAPITRE 4

La bibliothèque

La bibliothèque est le point de départ de presque tout : elle range vos cassettes, vos vidages et vos sources, reconnaît les cassettes d'époque, et lance un programme d'un clic, BASIC compris.

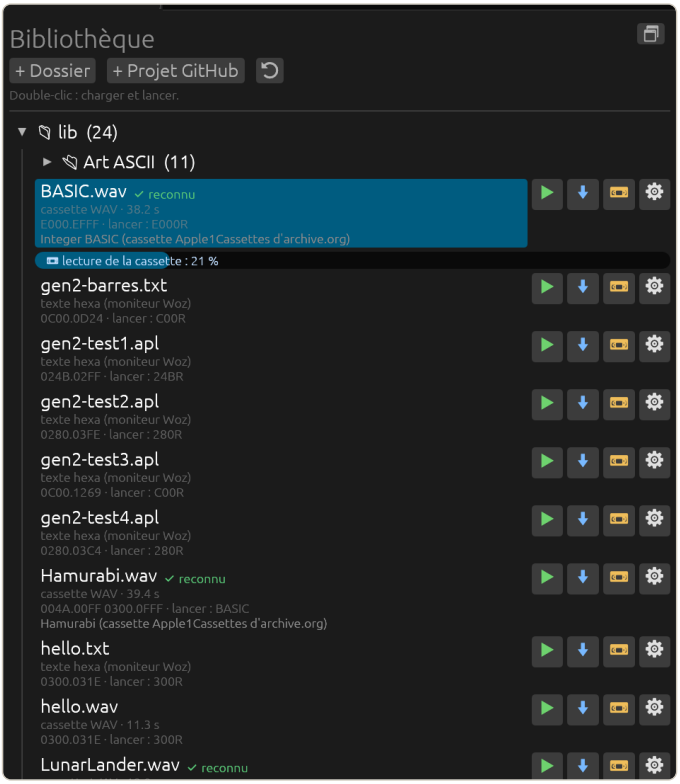


Figure 11 — Le panneau de la bibliothèque : le dossier principal, le dossier « Art ASCII », des cassettes reconnues et un projet GitHub. BASIC.wav est en cours de lecture.

4.1 Les dossiers

Le dossier principal est `~/apple1/programmes`. Il est créé au premier lancement, avec les exemples rangés dans cinq dossiers : **Art ASCII** (onze exemples d'art ASCII 40 × 24, chapitre 9), **BASIC** (programmes `.bas` et `.hex.txt` BASIC), **6502** (sources `.s` / `.asm` avec leurs `.bin` / `.hex`), **GEN2 COLOR** (les tests de la carte GEN2 de Bernie, `gen2-*.apl`, et les jeux couleur pour G2 Basic) et **WAV** (toutes les cassettes). Dans le panneau, ces cinq dossiers sont listés directement au premier niveau, dans l'ordre **BASIC**, **6502**, **GEN2 COLOR**, **WAV**, **Art ASCII** : le dossier racine (le nom du répertoire sur le disque) n'apparaît pas au-dessus d'eux, et les fichiers laissés en vrac dans ce dossier ne sont pas montrés. Rangez-les dans l'un des cinq dossiers. Un dossier que vous ajoutez, ou un projet GitHub, garde son propre titre. Une bibliothèque qui existe déjà n'est jamais réorganisée, sauf que ses

dossiers **Assembleur** et **Uncle Bernie** (les noms des versions 7.8 à 7.9.1) sont renommés **6502** et **GEN2 COLOR**, leur contenu étant conservé. Pour en choisir un autre : ⚙ **Préférences** → **Dossier de la bibliothèque** → **Parcourir…** (**Par défaut** revient à `~/apple1/programmes`).

+ Dossier, en haut du panneau, ajoute d'autres dossiers ; le dossier principal reste toujours là. La bibliothèque est un arbre : dossiers et sous-dossiers se replient, et l'état est mémorisé. Chaque dossier garde ses propres réglages dans un fichier `programs.json`. Clic droit sur un dossier ajouté : l'ouvrir dans le gestionnaire de fichiers, ou le retirer de la liste (rien n'est effacé sur le disque).

Panneau étroit. La bibliothèque peut être aussi étroite que vous voulez (jusqu'à une largeur minimale, en dessous de laquelle le séparateur s'arrête). Chaque entrée s'adapte alors : le nom prend toute la largeur, coupé par « … » s'il est trop long (la bulle d'aide montre le nom complet), les petites icônes ▶ ⬇ 📼 ⚙ passent à la ligne suivante, et la description est limitée à une ou deux lignes, également coupées par « … », le texte complet étant dans la bulle d'aide. Dans l'en-tête, **+ Dossier** et **+ Projet GitHub** deviennent des icônes avec bulle d'aide.

Les fichiers montrés sont : `.wav .aiff .bin .hex .txt .apl .bas .s .asm .prg`. La liste se met à jour toute seule, toutes les deux secondes : copiez un fichier dans le dossier, il apparaît.

Pour chaque programme, la liste indique le nom, le type, les plages d'adresses, la durée de la cassette, la commande de lancement et vos notes.

Les jeux de Claude IA

Les dossiers **BASIC** et **6502** reçoivent aussi cinq petits jeux écrits par Claude IA, chacun en version française et anglaise : *Le Pendu* (`pendu.bas`, `hangman.bas`), *Les Allumettes* (`allumettes.bas`, `matches.bas`) et *La Bataille navale* (`bataille.bas`, `battleship.bas`) en BASIC ; *Tunnel* (`tunnel.s`, `tunnel-en.s`) et *Morpion* (`morpion.s`, `tictactoe.s`) en assembleur 6502. Un double-clic sur un jeu BASIC le lance avec le RDK G2 Basic (l'émulateur démarre le BASIC, charge le listing et tape `RUN`) ; les sources en assembleur sont assemblées par l'assembleur intégré. Elles incluent `apple1.inc`, installé avec elles. Le dossier **GEN2 COLOR** contient aussi *Puissance 4 en couleur* (`puissance4-couleur.bas`), un jeu pour le G2 Basic et la carte GEN2, signé Claude IA : Q et S déplacent le pion au-dessus de la grille, Espace le lâche. Les jeux font partie du projet et sont sous sa licence (annexe « Licence et crédits »).

4.2 Lancer un programme

Chaque programme a quatre petites icônes à sa droite :

Icône	Action
▶	**Charger et lancer** : charge le programme s'il ne l'est pas, puis le lance : RESET, puis sa commande tapée au clavier (`300R`, ou `E2B3R` et `RUN` pour un programme BASIC). Si le programme a besoin de l'Integer BASIC et qu'il n'est pas en `$E000`, l'émulateur le charge d'abord. Un listing BASIC est tapé dans l'Integer BASIC, un source 6502 est assemblé.
⬇	**Charger** : met le programme en mémoire, instantanément, sans le lancer.
📼	**Par la cassette** : comme sur la vraie machine. L'émulateur fait RESET, tape `C100R` puis la commande de lecture de l'ACI (par exemple `0300.031ER`), et la cassette défile en vitesse réelle, avec son son et une barre de progression. Les `.txt` et `.bin` sont d'abord convertis en cassette. Grisée pour un source (`.bas`, `.s`), qui n'a pas d'enregistrement.

Icône	Action
⚙	**Paramètres** : plages, adresse de chargement d'un `.bin`, adresse et commande de lancement, type, besoin du BASIC, notes.

Le chargement est **instantané**, sauf si vous choisissez la cassette avec 📼. Un **double-clic** sur un programme le charge, et le lance s'il a une commande de lancement (comme ▶). La cassette ne sert que si vous la demandez ; le lecteur de la barre d'outils montre alors la bande et sa position.

Quand l'éditeur est ouvert, ▶, ⬇ ou un double-clic sur un listing BASIC ou un source 6502 l'**ouvre aussi dans un onglet de l'éditeur**, marqué 📚 ; s'il est déjà ouvert, son onglet est simplement affiché.

Le **clic droit** sur un programme propose le reste : charger, charger et lancer, insérer comme cassette sans rien taper, paramètres, renommer, supprimer (avec confirmation), ouvrir le dossier.

4.3 La fiche d'un programme

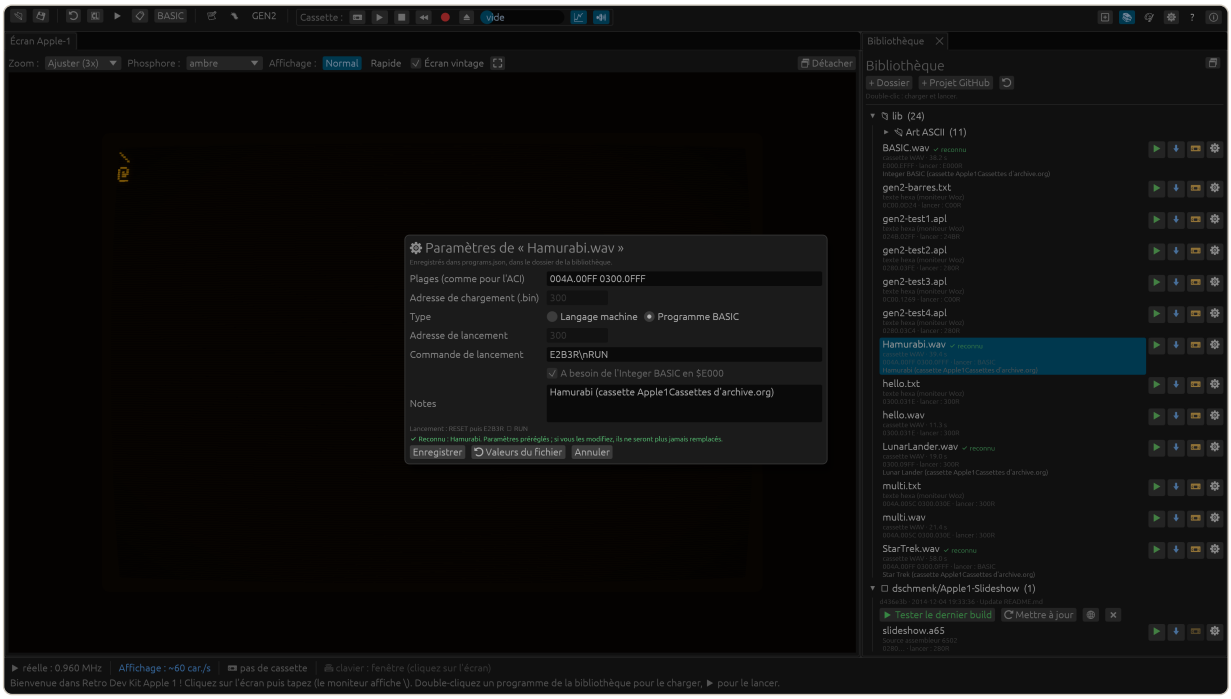


Figure 12 — La fiche ⚙ de Hamurabi : deux plages (page zéro du BASIC, puis le programme), lancement par E2B3R puis RUN.

Champ	Rôle
Plages	Les zones de mémoire, écrites comme pour l'ACI : `0300.031E` , ou `004A.00FF 0800.0FFF` pour un programme BASIC. Chaque enregistrement de la cassette va dans sa plage, dans l'ordre.
Adresse de chargement	Pour un `.bin` , qui ne contient pas d'adresse.
Lancement	L'adresse et la commande tapée par ▶. `\n` représente RETURN entre deux lignes : `E2B3R\nRUN` .

Champ	Rôle
Type	**Langage machine**, **Programme G2 Basic** (le choix par défaut pour un listing BASIC, puisque l'émulateur le fournit) ou **Programme Integer BASIC** (pour ceux qui préfèrent l'original d'Apple : ▶ charge alors l'Integer BASIC d'origine choisi dans les préférences). Le type choisi pour un programme l'emporte sur le réglage général du BASIC des préférences.
A besoin d'un BASIC	▶ chargera d'abord le BASIC du type du programme.
Notes	Texte libre, affiché dans la liste.

Les valeurs sont préremplies d'après le fichier et vérifiées à l'enregistrement ; **Valeurs du fichier** les relit.

> **! TOUJOURS QUATRE CHIFFRES : 004A.00FF, JAMAIS 4A.FF**
>
> La fiche refuse `4A.FF`. Ce n'est pas un caprice : la ROM de l'ACI n'efface pas son accumulateur au point, et lirait `$004A-$4AFF` (voir la section 5.6). L'émulateur écrit donc toujours les adresses sur quatre chiffres.

4.4 Les cassettes reconnues

Quand un fichier est **reconnu**, sa fiche est remplie toute seule et la liste affiche ↻ **reconnu**. L'émulateur décode les enregistrements et compare leur taille et leur empreinte SHA-256 à une table intégrée : les 24 cassettes de la collection « Apple1Cassettes » d'archive.org. Le nom du fichier n'a alors aucune importance. À défaut, il reconnaît le **nom** (majuscules, espaces et tirets ignorés) avec les mêmes tailles d'enregistrements ; la bulle d'aide signale alors que le contenu diffère.

Cassette	Plages	Lancement
`BASIC.wav` (Integer BASIC)	`E000.EFFF`	`E000R`
`Hamurabi.wav`, `StarTrek.wav`, `Wumpus.wav`, `Slots.wav`, `Craps.wav`, `Bowling.wav`, `Buzzword.wav`, `WordCross.wav`, `AppleiEnhancedCheckers.wav`, `LunarlanderasciiGraphics.wav`	`004A.00FF 0300.0FFF`	`E2B3R ⏎ RUN`
`AceyDucey.wav`, `Blackjack.wav`, `HiLo.wav`	`004A.00FF 0800.0FFF`	`E2B3R ⏎ RUN`
`Football.wav`	`004A.00FF 0400.2FFF`	`E2B3R ⏎ RUN` (12 Ko)
`DealOrNoDeal.wav`, `StarTrek2003.wav`	`004A.00FF 0300.3FFF`	`E2B3R ⏎ RUN` (16 Ko)
`APPLE30TH.wav`	`0280.0FFF`	`280R`
`ASMmchess.wav`	`0300.0BFF`	`300R`
`Microchess.wav`	`1000.18FF`	`1000R`
`LunarLander.wav`	`0300.09FF`	`300R`
`Mastermind.wav`	`0300.03FF`	`300R`
`Pasart.wav`	`0300.05FF`	`300R`

Cassette	Plages	Lancement
`Life.wav`	`2000.21FF`	`2000R` (12 Ko)

Vos propres réglages ne sont jamais écrasés : un programme que vous avez réglé vous-même garde sa fiche, seul l'indicateur « reconnu » apparaît. Les programmes qui dépassent les 8 Ko de base reçoivent automatiquement la RAM nécessaire au lancement, comme avec une carte d'extension.

Cassette inconnue. L'émulateur devine ce qu'il peut : un enregistrement de 4096 octets qui commence par `JMP` est l'Integer BASIC ; deux enregistrements dont le premier fait 182 octets (`$4A-$FF`) sont un programme BASIC, placé d'après ses pointeurs LOMEM et HIMEM. La liste indique alors « (devinées) » ; vérifiez avec ⚙. Un programme en langage machine sans adresses n'est pas deviné : indiquez sa plage.



Figure 13 — Numériser une vraie cassette : de la bande au fichier, puis à la bibliothèque.

4.5 Les projets GitHub

+ Projet GitHub accepte une adresse (`https://github.com/jefftranter/6502/tree/master/asm/jmon`) ou simplement `propriétaire/dépôt` . Les fichiers utiles sont copiés dans `~/.cache/apple1-emu/github/<propriétaire>/<dépôt>/<commit>/` , et le projet devient un nœud de l'arbre, avec son commit, sa date et son message. **Mettre à jour** va chercher le dernier commit ; 🌐 ouvre la page du dépôt. Tout le réseau se fait en arrière-plan.

Tester le dernier build essaie, dans l'ordre :

1. les fichiers de la dernière **Release** du dépôt (`.wav` / `.aiff` , puis `.bin` , puis `.hex` / `.txt`) ;
2. le dernier **artefact GitHub Actions** (un jeton est nécessaire) ;
3. la **compilation des sources** : un listing BASIC est tapé dans l'Integer BASIC puis lancé par `RUN` ; un source 6502 est assemblé avec **ca65/ld65** (paquet `cc65`), avec `-D APPLE1` par défaut ; un vidage Woz est chargé.

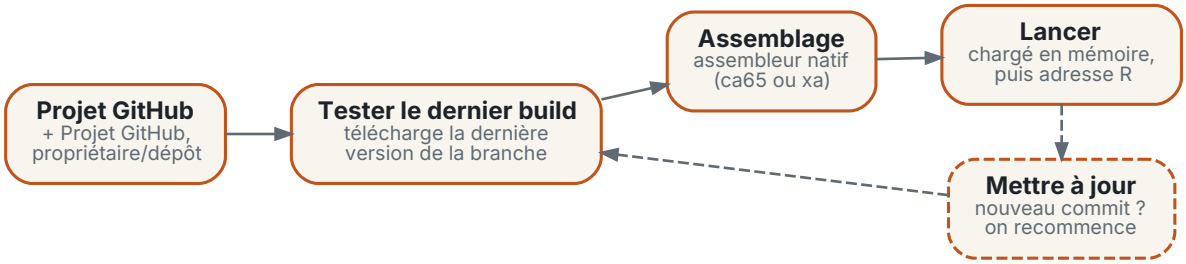


Figure 14 — Un projet GitHub, de l'adresse du dépôt au programme lancé.

Un fichier `apple1.toml` (ou `apple1.cfg`) à la racine du dépôt précise ce qu'il faut construire :

apple1.toml

```toml
file    = "src/jeu.s"     # programme à lancer
load    = $0300           # adresse de chargement (sinon le .org du source)
run     = $0300           # adresse de lancement (sinon celle de chargement)
ld65    = "apple1.cfg"    # configuration ld65 (sinon -t none)
defines = "APPLE1"        # symboles -D pour ca65
```

✦ LE JETON GITHUB EST FACULTATIF

Les dépôts publics n'en demandent pas. Un jeton (variable `GITHUB_TOKEN`, ou ⚙ **Préférences**) relève la limite de requêtes et donne accès à vos dépôts privés et aux artefacts. Il n'est jamais affiché ni écrit dans les messages, et `gui.json` passe alors en lecture réservée à votre compte.

4.6 Le dossier « Art ASCII »

La bibliothèque contient aussi un dossier **Art ASCII** : des dessins au format de l'écran (pomme, carte Apple-1, puce 6502, cassette, fusée…) et des portraits de Steve Wozniak et Steve Jobs convertis depuis des photos libres. Un `.txt` d'art ASCII montre un aperçu quand il est choisi, avec → **BASIC**, → **6502** et 🎨 **Ouvrir dans l'outil** ; ▶ lance tout de suite sa version 6502. Le chapitre 9 décrit l'outil.

CHAPITRE 5

La cassette et la carte ACI

L'Apple-1 n'avait ni disquette ni ROM BASIC : tout passait par une bande audio et la carte ACI (Apple Cassette Interface). L'émulateur exécute la vraie ROM de l'ACI ; ses fichiers WAV et AIFF se lisent donc sur un vrai Apple-1, et ceux d'un vrai Apple-1 se lisent dans l'émulateur.

5.1 Les formats de fichiers

Extension	Contenu	Lecture	Écriture
`.wav`	cassette ACI, PCM 8 à 32 bits ou flottant, mono ou stéréo, toute fréquence	oui	44,1 kHz mono 16 bits par défaut
`.aif`, `.aiff`	cassette ACI (AIFF et AIFF-C)	oui	profil smartphone : 48 kHz, 2 canaux identiques, 16 bits
`.txt`, `.hex`, `.apple1`	vidage du Woz Monitor : lignes `0300: A9 C1 20 …`	oui	oui
`.bin`	octets bruts, sans adresse (`FICHIER.bin@300`)	oui	oui
`.bas`	listing Integer BASIC (texte)	dans l'éditeur	dans l'éditeur
`.s`, `.asm`, `.a65`	source 6502 (ca65, ou xa)	dans l'éditeur	dans l'éditeur

Les cassettes exportées par l'émulateur portent leurs adresses dans un commentaire du fichier (`APPLE1 ACI 0300.031E`), que les lecteurs audio ignorent. Une cassette d'une autre provenance n'a pas d'adresses : la bibliothèque les reconnaît ou les devine (chapitre 4), et la ligne de commande accepte `--addr` ou `--range`.

Le décodeur tolère le bruit, les faibles niveaux, l'inversion de polarité, une vitesse de bande de ±10 % et plusieurs enregistrements dans un même fichier. Les WAV de l'outil **c2t** d'Egan Ford sont lus aussi. Un **ronflement du secteur** (50 ou 60 Hz, même aussi fort que le signal de la cassette) est filtré avant la détection des fronts : un long enregistrement fait avec un câble mal blindé se lit donc correctement.

5.2 Le lecteur virtuel

La zone **Cassette :** de la barre d'outils réunit toutes les commandes du magnétophone : 📼 **Insérer**, ▶ lecture, ■ stop, ⏪ rembobiner, ● enregistrer, ⏏ éjecter, puis le nom et la position de la bande, l'oscilloscope 📈 et le son 🔊. Une cassette s'insère par 📼 **Insérer** (un `.wav` ou un `.aiff`), ou par le

clic droit d'un programme de la bibliothèque (« Insérer comme cassette ») : vous tapez alors vous-même `C100R` et la plage. L'icône 📼 d'un programme de la bibliothèque va plus loin : elle insère la cassette et tape les commandes pour vous.



Figure 15 — La zone Cassette : pendant la lecture de BASIC.wav : 7,6 secondes sur 35,8.

La cassette suit le temps **émulé** : en vitesse rapide (F3), le chargement est bien plus court que les 10 secondes d'en-tête, et il fonctionne quand même. Sans la cassette, les chargements de la bibliothèque sont instantanés (chapitre 4).

Charger l'Integer BASIC « à l'ancienne » se fait ainsi :

```
WOZ MONITOR
C100R
C100: A9*
E000.EFFFR
E000R

E000: 4C
>
```

`C100R` démarre le programme de l'ACI, qui répond par son invite `*`. La ligne `E000.EFFFR` demande de **lire** la bande dans `$E000-$EFFF`. Quand la bande s'arrête, l'ACI rend la main au moniteur (`\`) ; `E000R` démarre le BASIC, dont l'invite est `>`.

5.3 Son et oscilloscope

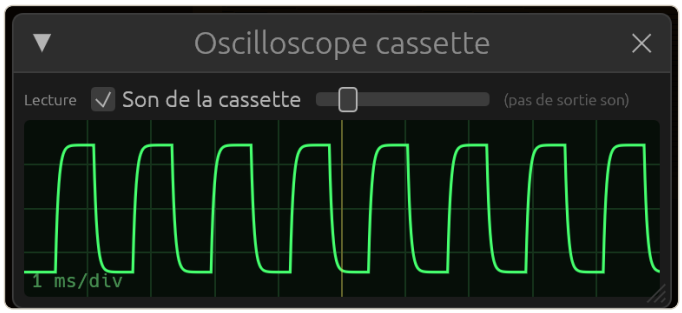


Figure 16 — L'oscilloscope pendant la lecture de BASIC.wav : la case « Son de la cassette », le volume, et le signal à 1 ms par division.

Pendant une lecture ou un enregistrement, 🔊 fait entendre le signal, comme le haut-parleur d'un magnétophone, à volume faible par défaut. 📈 ouvre l'**oscilloscope** : une trace verte sur une grille sombre, 1 ms par division, qui montre 10 ms autour de la tête de lecture, ou les 10 dernières millisecondes écrites par l'Apple-1. On y reconnaît l'en-tête (≈ 800 Hz), les bits 0 (≈ 2 kHz) et les bits 1 (≈ 1 kHz). Le son est muet en vitesse maximale.

5.4 Enregistrer depuis l'Apple-1 émulé

Pour produire une cassette comme en 1976, armez l'enregistreur, puis demandez à l'ACI d'écrire :

1. Cliquez sur ● et choisissez le fichier (`.wav` ou `.aiff`).
2. Tapez `C100R` , puis la plage suivie de `W` , par exemple `0300.03FFW` , puis RETURN .
3. L'ACI écrit un en-tête de 10 secondes, puis les données. Le fichier est écrit 1,5 s après la fin, ou au ■.

Le plus simple reste **Sauver** dans la barre d'outils, ou 📼 **Exporter WAV** dans l'éditeur : le signal est le même, au cycle près, et il est produit instantanément.

5.5 Comment fonctionne l'ACI

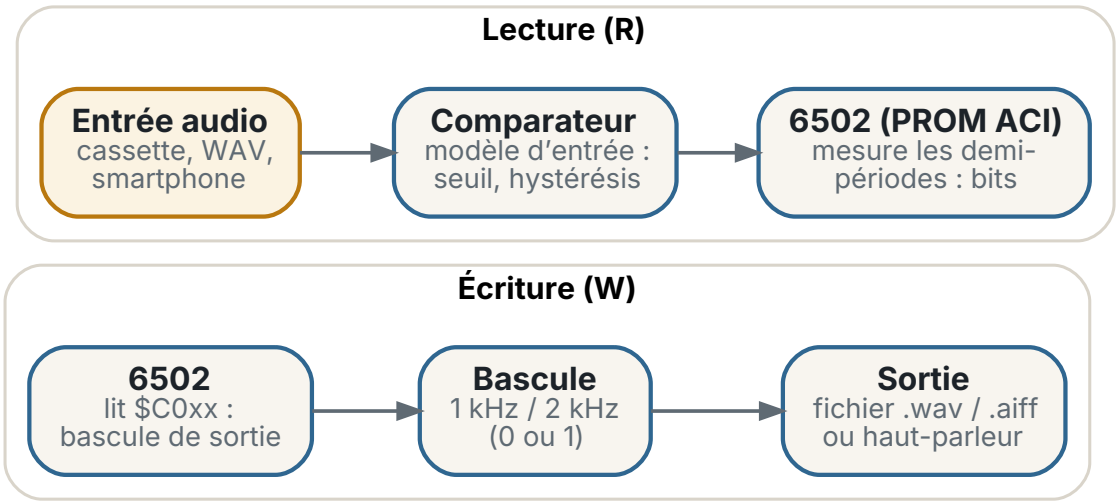


Figure 17 — Les deux sens de l'ACI : à l'écriture, chaque accès à $C0xx fait basculer la sortie ; à la lecture, le comparateur modifie la ligne A0 de la PROM.

La carte est d'une simplicité remarquable : une PROM de 256 octets visible en `$C000` et `$C100` , une bascule et un comparateur.

- **Écriture.** Chaque accès à une adresse `$C000-$C0FF` fait basculer la sortie (74LS74). Le programme de la ROM règle la durée entre deux accès : c'est elle qui code les bits.
- **Lecture.** Le niveau de l'entrée bande, après le comparateur LM311, modifie la ligne d'adresse A0 de la PROM quand A7 = 1. La même instruction lit donc `$C080` ou `$C081` selon le niveau : la ROM mesure les demi-périodes par une boucle de 12 cycles.
- **Codage.** Un « 1 » dure environ deux fois plus longtemps qu'un « 0 » : 1 kHz pour un 1, 2 kHz pour un 0. Les octets partent bit de poids fort en premier, sans parité ni somme de contrôle.

L'annexe C donne les temps exacts, en cycles et en microsecondes.

5.6 Les commandes de l'ACI

Après `C100R`, l'ACI attend une ligne au format du moniteur, terminée par `R` (lire) ou `W` (écrire) :

Ligne	Effet
`0300.03FFR`	lit la bande dans `$0300-$03FF`
`0300.03FFW`	écrit `$0300-$03FF` sur la bande, avec un en-tête de 10 s
`004A.00FFR 0300.0FFFR`	lit **deux** enregistrements successifs : la page zéro du BASIC, puis le programme
`004A.00FFW800.FFFW`	la forme « collée » du manuel BASIC de 1976, acceptée aussi

> **! LES ZÉROS DE 004A.00FF SONT OBLIGATOIRES**
>
> L'analyseur de la ROM n'efface pas son accumulateur hexadécimal au point (« Doesn't clear HEX1!!! », dit le listing). Les chiffres de la fin s'ajoutent derrière ceux du début : `4A.FF` signifie `$004A-$4AFF`, soit près de 19 Ko ! `0300.0FFF` fonctionne, parce que les quatre chiffres chassent le 3 de `0300`. Écrivez toujours les deux adresses sur quatre chiffres. L'émulateur reproduit ce comportement, puisqu'il exécute la vraie ROM.

Un programme BASIC se sauve en deux enregistrements : la page zéro `$004A-$00FF` (où sont les pointeurs LOMEM, HIMEM et la fin du programme), puis le programme lui-même, qui finit toujours à HIMEM. Pour le recharger, le BASIC doit déjà être en mémoire, puis on revient au BASIC « à chaud » par `E2B3R`, qui garde le programme.

WOZ MONITOR

```
C100R
C100: A9*
004A.00FFR 0300.0FFFR
E2B3R

E2B3: 20
>RUN
```

5.7 Vers un vrai Apple-1

C'est l'usage qui justifie l'émulateur : préparer un programme sur le PC, puis le jouer dans l'ACI d'une vraie machine, depuis un smartphone ou la sortie casque d'un ordinateur.

1. **Préparez le fichier.** 📼 **Exporter WAV** dans l'éditeur, ou `apple1-emu export prog.txt -o prog.aiff`. Choisissez `.aiff` pour un smartphone. Notez la ligne affichée, par exemple `C100R` puis `0300.031ER`.
2. **Réglez le lecteur.** Volume au maximum (acceptez l'avertissement « volume élevé »), tous les effets audio coupés (égaliseur, Dolby, « amélioration audio »), mode avion, pas de Bluetooth. Un lecteur simple comme VLC convient.
3. **Sur l'Apple-1**, tapez `C100R` : le moniteur répond `C100: A9*`.

4. Tapez la ligne de lecture, par exemple `0300.031ER`, mais **n'appuyez pas encore sur RETURN**.
5. Lancez la lecture, puis appuyez sur RETURN **pendant l'en-tête** (il dure environ 10 s, après une seconde de silence).
6. À la fin, l'ACI revient au moniteur (\ ). Lancez le programme : `300R`, ou `E2B3R` puis `RUN` pour un programme BASIC.

Pourquoi ce profil AIFF ? 16 bits non compressé, sans codec avec perte : un MP3 ou un AAC arrondirait les fronts. 48 kHz, la fréquence native d'Android, pour éviter tout rééchantillonnage. Deux canaux strictement identiques, parce qu'un câble qui relie gauche et droite sur FROM TAPE ne doit jamais recevoir deux signaux différents. Un signal carré à −1 dBFS : l'entrée de l'ACI d'origine est un passe-haut (0,01 µF) qui ne voit guère que les fronts, et un carré à fronts raides est ce qui lui convient le mieux. Une seconde de silence au début et à la fin, parce qu'un téléphone coupe parfois le début d'un son.

> ▣ **NE VOUS FIEZ PAS À LA LED**
>
> Le manuel de 1976 demande un niveau tel que la LED de l'ACI soit « juste allumée ». Elle ne s'allume pourtant qu'à environ 1,2 V, bien plus que ce dont le comparateur a besoin. Une LED éteinte ne prouve donc pas que le signal est trop faible. Si la lecture ne finit jamais, la ROM attend encore des fronts : faites RESET et montez le volume.

> ! **JAMAIS DE MP3 NI D'AAC**
>
> Un format avec perte décale les fronts et rend la cassette illisible. Utilisez WAV, AIFF, ou FLAC si votre lecteur le décode.

5.8 Quel niveau faut-il ? Les modèles d'entrée

La commande `info --aci-input` simule l'entrée analogique de la carte (condensateur, polarisation, hystérésis du LM311) pour une série de tensions crête, et dit à partir de quel niveau le fichier serait lu :

TERMINAL

```
apple1-emu info prog.aiff --aci-input stock     # carte de 1976, 0,01 µF
apple1-emu info prog.aiff --aci-input mimeo     # 0,1 µF (conseil du guide Mimeo)
apple1-emu info prog.aiff --aci-input gen2      # ACI Gen2 d'Uncle Bernie
apple1-emu info prog.aiff --aci-input stock --volts 0.5 --series 1200
```

Modèle	Ce qu'il représente
`stock`	l'ACI d'origine : 0,01 µF vers le comparateur polarisé à 2,5 V, réaction 3k/47k
`mimeo`	la modification de Mike Willegal : 0,1 µF, plus fiable
`gen2`	l'ACI Gen2 d'Uncle Bernie : 100 nF appairé, 300k au lieu de 47k (moins d'hystérésis)

`--volts` fixe la tension crête à la prise pour la pleine échelle numérique ; `--series` ajoute une résistance série (par exemple les 1,2 kΩ d'un câble de smartphone). Avec le modèle `stock`, les fichiers de l'émulateur sont lus dès **environ 0,25 V crête** ; une sortie casque de smartphone au maximum donne en général 0,5 à 1 V. Ce modèle est simplifié : ni bruit, ni dérive du point de polarisation.

5.9 Numériser une vraie cassette ou un enregistrement de l'Apple-1

Pour récupérer un programme tapé sur un vrai Apple-1, enregistrez la sortie **TO TAPE** de l'ACI par une entrée micro (téléphone, ou carte son du PC avec Audacity en 44,1 ou 48 kHz), en coupant le gain automatique et la réduction de bruit. Sur l'Apple-1 : `C100R`, puis `0300.03FFW` et RETURN. Arrêtez quelques secondes après le retour du `\`, puis :

TERMINAL

```
apple1-emu import enregistrement.aiff --addr 300 -o prog.txt
apple1-emu import startrek.aiff --range "004A.00FFR 0300.0FFFR" -o startrek.txt
```

> ▣ **LA SORTIE TO TAPE EST UN NIVEAU MICRO**
>
> Elle ne fournit qu'environ 50 mV crête à crête. Regardez la forme d'onde dans Audacity : ni écrêtage, ni saut de niveau pendant l'en-tête. L'import de l'émulateur tolère un gain quelconque et un décalage continu.

CHAPITRE 6

Les cartes graphiques : GEN2 et P-LAB

En 2026, Uncle Bernie a conçu pour l'Apple-1 une carte graphique couleur, la « GEN2 », qui reprend les modes d'affichage de l'Apple II. L'émulateur la simule en option : ses commutateurs, sa mémoire, son balayage et son écran couleur. Ce chapitre décrit aussi, à la fin, la carte graphique P-LAB à TMS9918A.

6.1 Brancher la carte

La carte est **débranchée par défaut**. Trois façons de la brancher :

- **GEN2**, dans le groupe « Cartes graphiques » de la barre d'outils : la carte est branchée ou débranchée tout de suite ; le programme en cours continue (faites un RESET s'il utilisait la carte) ;
- ⚙ **Préférences → Carte couleur Uncle Bernie GEN2**, avec le choix 60 ou 50 Hz ;
- en ligne de commande, `apple1-emu-gui --carte-couleur` (`--couleur-50hz`, `--sans-carte-couleur`).

Sur la vraie machine, la carte a sa propre sortie vidéo pour un second moniteur, en couleur ; le terminal de l'Apple-1 reste sur sa sortie d'origine et fonctionne comme avant. La fenêtre montre donc **deux écrans** : celui de l'Apple-1, et le **moniteur couleur** dessous ou à côté. Une légende indique le mode courant. Le moniteur couleur a sa propre rangée d'affichage : zoom, affichage **Normal** / **Rapide** (commun avec l'écran de l'Apple-1), sa propre case **Écran vintage**, plein écran et **Détacher**, pour le mettre sur un deuxième moniteur.



Figure 18 — Le moniteur couleur : les barres LORES de l'exemple gen2-lores.s, et la légende du mode.

> ▣ **LA CARTE EST AUSSI UNE EXTENSION MÉMOIRE**
> Avec la GEN2, la RAM couvre `$0000-$BFFF` (plus `$E000-$EFFF`). Les pages graphiques sont dans cette mémoire : un programme ne doit pas s'y installer.

6.2 Les commutateurs

La carte reprend les commutateurs `$C050-$C057` de l'Apple II, déplacés en `$C250-$C257`. Ils réagissent **uniquement en lecture** : une écriture est sans effet. En assembleur, on les actionne donc avec `BIT` ou `LDA`, jamais avec `STA`.

Adresse	Nom (`gen2.inc`)	Effet
`$C250`	`SS_GR`	mode graphique
`$C251`	`SS_TEXT`	mode texte
`$C252`	`SS_FULL`	plein écran
`$C253`	`SS_MIX`	mixte : 4 lignes de texte en bas
`$C254`	`SS_PAGE1`	page 1
`$C255`	`SS_PAGE2`	page 2
`$C256`	`SS_LORES`	basse résolution
`$C257`	`SS_HIRES`	haute résolution

Le bit 7 d'une lecture vaut **HSTo** : 1 pendant le retour ligne et le retour trame, 0 pendant la partie visible. Un programme peut s'en servir pour se synchroniser sur le balayage (c'est ce que fait TEST4). Les bits 0 à 6 sont flottants. La carte répond aussi sur ses miroirs (`$C258`, `$C3F1`…) et laisse libres `$C0xx` et `$C1xx` pour l'ACI. RESET ne touche pas aux commutateurs ; l'émulateur démarre en texte, page 1.

En Integer BASIC, une lecture suffit aussi : `X=PEEK(-15792)` lit `$C250` (−15792 = `$C250` − 65536).

6.3 Les modes et la mémoire

Mode	Définition	Page 1	Page 2
TEXTE	40 × 24 caractères	`$0400-$07FF`	`$0800-$0BFF`
LORES	40 × 48 blocs, 16 couleurs	`$0400-$07FF`	`$0800-$0BFF`
HIRES	280 × 192 points, 6 couleurs	`$2000-$3FFF`	`$4000-$5FFF`
MIXTE	graphique, avec les 4 dernières lignes en texte		

Adresses des lignes. Comme sur l'Apple II, les lignes ne se suivent pas en mémoire. La rangée de texte *r* (0 à 23) commence en :

```
$0400 + (r & 7) × $80 + (r >> 3) × $28
```

Chaque octet de la page LORES porte **deux** blocs : le quartet bas pour le bloc du haut, le quartet haut pour celui du bas.

En HIRES, chaque ligne de balayage *y* (0 à 191) fait 40 octets de 7 points (bit 0 à gauche, bit 7 pour la palette) et commence en :

```
$2000 + (y & 7) × $400 + ((y >> 3) & 7) × $80 + (y >> 6) × $28
```

Couleurs LORES (dans l'ordre des valeurs 0 à 15) :

0	1	2	3	4	5	6	7
noir	magenta	bleu foncé	violet	vert foncé	gris	bleu moyen	bleu clair
8	**9**	**10**	**11**	**12**	**13**	**14**	**15**
marron	orange	gris	rose	vert clair	jaune	aigue-marine	blanc

Couleurs HIRES (artefacts NTSC) : bit 7 à 0, violet sur les colonnes paires et vert sur les impaires ; bit 7 à 1, bleu et orange ; deux points voisins allumés donnent du blanc. Ainsi `$55` / `$2A` en alternance donne du violet, `$2A` / `$55` du vert, `$D5` / `$AA` du bleu.

Caractères du mode texte : codes `$00-$3F` en inverse, `$40-$7F` clignotants, `$80-$FF` normaux ; `$E0-$FE` sont des minuscules et `$FF` un damier.

6.4 Les programmes d'essai

Le dossier `exemples/` (et la bibliothèque, si elle est créée à neuf) contient une démo et les quatre tests d'Uncle Bernie :

Fichier	Lancement	Contenu
`gen2-barres.txt`	`C00R`	les 16 couleurs LORES en barres, plus une ligne de texte en mode mixte
`gen2-test1.apl`	`24BR`	TEST1 : test de la RAM de la carte, avec des rayures HIRES
`gen2-test2.apl`	`280R`	TEST2 : diagonale LORES, puis mode mixte et « machine à écrire » (une touche pour commencer, ESC pour sortir)
`gen2-test3.apl`	`C00R`	TEST3 : les 256 codes de caractères, puis l'éditeur plein écran
`gen2-test4.apl`	`280R`	TEST4 : mesure de HST0, puis une fenêtre de 4 lignes de texte qui défile sur du LORES

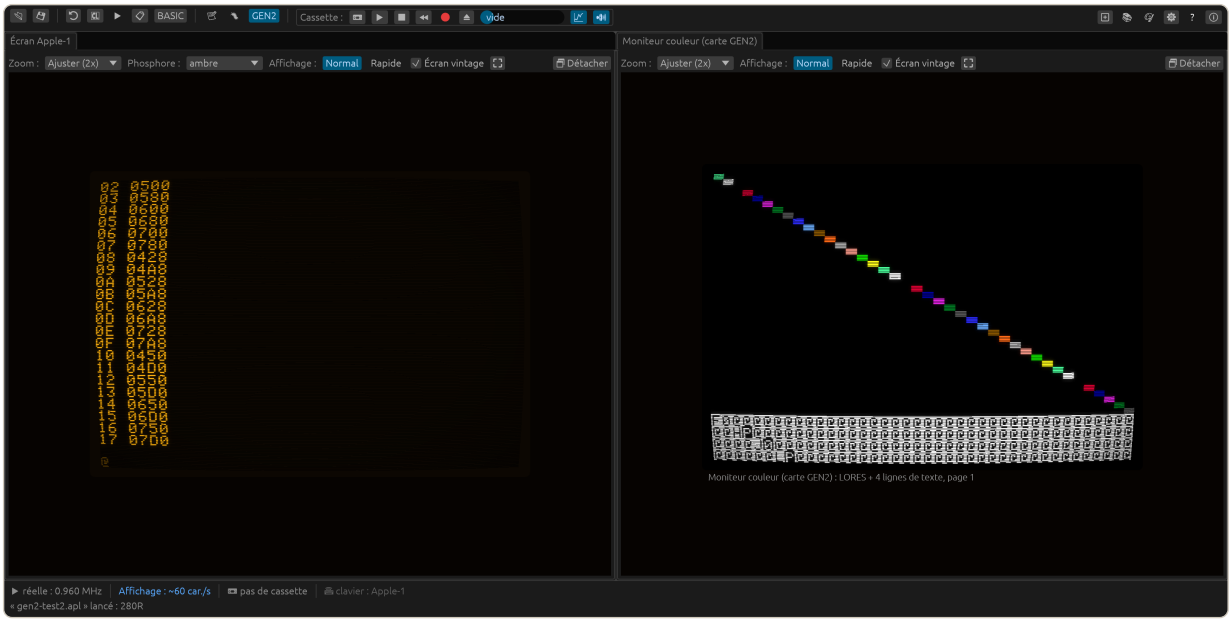


Figure 19 — TEST2 d'Uncle Bernie : la diagonale LORES sur le moniteur couleur.

Les sources `gen2-lores.s` et `gen2-hires.s` de la partie III (chapitre 23) montrent comment écrire soi-même dans les pages graphiques.

6.5 Ce qui est approché

Les teintes exactes dépendent du NTSC, du moniteur et du condensateur ajustable de la carte : l'émulateur utilise des valeurs proches de l'Apple IIgs, dans l'ordre documenté. Le dessin des minuscules est fait main (l'EPROM de la carte n'est pas publiée). Le décalage d'un demi-point en HIRES quand le bit 7 est à 1 est ignoré. Un changement de mode compte à partir de la ligne de balayage suivante. Au démarrage, la mémoire est à zéro, ce qui donne un écran de `@` en inverse tant que le programme ne l'a pas rempli. La carte GEN1 de 2023 (en `$C050`) n'est pas émulée.

6.6 La carte graphique P-LAB

La « Apple-1 Graphic Card » de P-LAB donne à l'Apple-1 une vraie puce vidéo, le **TMS9918A** *de Texas Instruments (celui de la ColecoVision, du MSX et du TI-99/4A) : 256 × 192 points, 16 couleurs, 32 sprites et 16 Ko de mémoire vidéo. L'émulateur la reproduit en option, à côté de la* GEN2.

Brancher la carte

La carte est **débranchée par défaut**. Cliquez sur **P-LAB** dans le groupe « Cartes graphiques » de la barre d'outils (le bouton s'allume), ou choisissez **Brancher la carte P-LAB** dans le bouton **Écran couleur** quand aucune carte n'est branchée, ou lancez `apple1-emu-gui --plab` (`--sans-plab` pour l'ôter). L'**écran P-LAB** apparaît dans un panneau (le bouton **Écran couleur** du groupe Fenêtres le montre ou le cache) ; comme tout panneau, il s'ancre, se met en onglet ou se détache dans sa propre fenêtre.

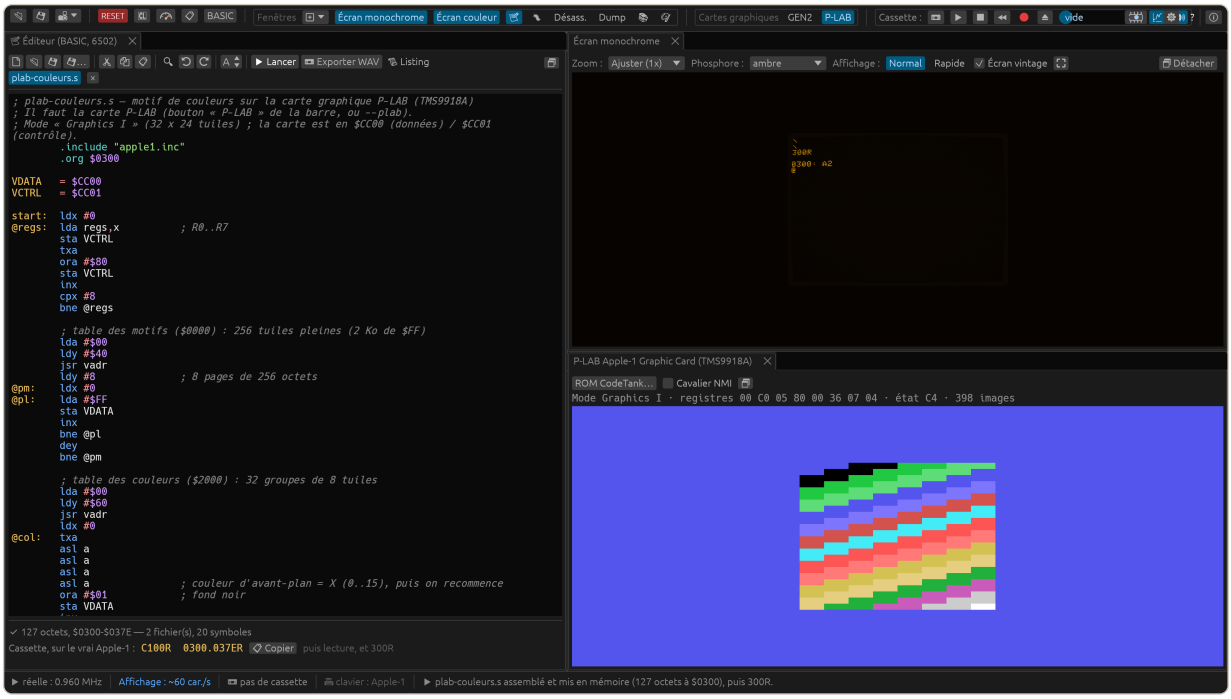


Figure 20 — L'écran P-LAB sous le terminal de l'Apple-1, avec le motif de couleurs de l'exemple plab-couleurs.s. La ligne sous la barre du panneau donne le mode d'affichage, les huit registres et l'état.

Le terminal de l'Apple-1 continue de marcher comme avant. La carte prend les adresses `$CC00-$CFFF` : **`$CC00` est le port de données** (lire ou écrire la mémoire vidéo) et **`$CC01` le port de contrôle** (adresse vidéo et registres en écriture, état en lecture). L'émulateur ne décode que la ligne d'adresse A0 dans la fenêtre `$CC00-$CFFF`, si bien que la paire se répète sur tout le kilo-octet ; le manuel de la carte ne le précise pas, c'est une hypothèse (voir la fin de cette section).

Le panneau a deux commandes et une ligne d'information :

- **ROM CodeTank…** met une image de 16 Ko dans la carte fille CodeTank : elle se lit alors en `$4000-$7FFF`, en lecture seule, par-dessus la RAM. Tapez `4000R` sur le terminal pour la lancer. **Retirer la ROM** l'ôte ;
- **Cavalier NMI** est le cavalier IRQ/NMI de la carte. Décoché (le réglage normal), l'interruption de fin d'image du TMS9918A (60 fois par seconde) arrive au 6502 en IRQ ; coché, en NMI. L'interruption n'a lieu que si le programme l'autorise (bit 5 du registre 1) ;
- la **ligne d'information** montre le mode (Graphique I, Graphique II, Texte, Multicolore), les registres R0 à R7 en hexadécimal, le registre d'état et le nombre d'images dessinées.

La programmer

Le TMS9918A n'est accessible que par deux ports. Pour écrire dans la mémoire vidéo, envoyez l'adresse au port de contrôle (l'octet bas, puis l'octet haut **OU** `$40`), puis écrivez les octets au port de données : l'adresse s'incrémente toute seule. Pour écrire un registre, envoyez la valeur, puis le numéro du registre **OU** `$80`, au port de contrôle. Lire le registre d'état (bit 7 = drapeau d'image, bit 6 = cinquième sprite, bit 5 = collision) l'efface.

L'exemple `plab-couleurs.s` (dans la bibliothèque, dossier **6502** ; aussi dans `docs/exemples/asm/`) met le mode Graphique I, remplit les tables des motifs, des couleurs et des noms, et laisse un motif de couleurs sur l'écran de la carte. Lancez-le depuis l'éditeur (▶ **Lancer**, puis `300R`, ou assemblez et lancez en un clic) :

plab-couleurs.s (extrait) 6502

```
VDATA   = $CC00
VCTRL   = $CC01

start:  ldx #0
@regs:  lda regs,x          ; R0..R7
        sta VCTRL           ; valeur
        txa
        ora #$80
        sta VCTRL           ; numéro du registre + $80
        inx
        cpx #8
        bne @regs
```

Charger les démos publiques

Les démos du projet `nippur72/apple1-videocard-lib` (écrites en KickC) existent sous quatre formes ; l'émulateur les connaît toutes :

Fichier	Ce que c'est	Comment le charger
`*.bin`	programme brut qui démarre en `$0280`	**Charger** (la boîte d'adresse propose `280`), puis `280R` sur le terminal
`*.woz`	la même chose, en texte pour le Woz Monitor	**Charger**, puis `280R`
`*.prg`	une adresse de départ sur 2 octets (petit-boutiste), puis le code	**Charger** : l'adresse est lue dans le fichier
`*_jukebox.bin`	image d'EPROM de 16 Ko pour le CodeTank	**ROM CodeTank...**, puis `4000R`

Ces démos ne sont pas distribuées avec ce kit (leur auteur n'a pas publié de licence) ; téléchargez-les depuis la page du projet et gardez-les dans votre bibliothèque. L'émulateur a été vérifié avec quatre d'entre elles (le menu des démos avec ses écrans et le ballon qui rebondit, la main d'Amiga, le visionneur de photos et Tetris), en versions RAM et jukebox.

Ce qui est émulé, et ce qui ne l'est pas

▣ CE QUE FAIT L'ÉMULATEUR

Les quatre modes d'affichage (Graphique I et II, Texte, Multicolore) ; les 16 Ko de mémoire vidéo ; les 32 sprites, de 8 × 8 ou 16 × 16, agrandis ou non, avec la limite de cinq sprites par ligne et le drapeau de collision ; le drapeau d'image et l'interruption de fin d'image, en IRQ ou en NMI ; le CodeTank en `$4000-$7FFF` . La palette du TMS9918A est la palette usuelle en NTSC.

! CE QUI N'EST PAS ÉMULÉ

Un registre changé au milieu d'une image ne prend effet qu'à l'image suivante (les effets de trame ne sont pas reproduits) ; le mode DRAM de 4 Ko ; la synchronisation exacte des sprites et des accès à la mémoire vidéo ; l'entrée vidéo externe ; les artefacts de la vidéo composite. L'état de la puce vidéo n'est **pas** enregistré dans les états de la machine.

LÀ OÙ LA DOCUMENTATION SE TAIT

Le manuel de la carte (révision 0.4) ne dit pas comment la fenêtre est répétée, quelle est la synchronisation du bus, ce que contient la mémoire vidéo à la mise sous tension, si le CodeTank est protégé en écriture, ni comment exactement le cavalier IRQ/NMI est câblé. L'émulateur fait des choix raisonnables sur ces points (décrits plus haut) ; si vous possédez la carte et voyez une différence, dites-le-nous.

CHAPITRE 7

L'éditeur

L'éditeur intégré écrit des programmes Integer BASIC et des sources 6502, les vérifie à chaque frappe, les lance d'un clic dans l'Apple-1 émulé, et les exporte en cassette pour la vraie machine.

7.1 Les onglets et les fichiers

📝 **Éditeur** ouvre le panneau, ancré à gauche de l'écran. Sa barre d'outils est compacte, pour laisser le plus de lignes possible au texte. Elle commence par des icônes, expliquées par leurs infobulles :

Icône	Action
Nouveau (première icône)	nouvel onglet : « Programme BASIC » ou « Source assembleur 6502 »
📂 / 💾 / 💾…	Ouvrir (`.bas` , `.txt` , `.s` , `.asm` , `.a65` , `.inc`) / Enregistrer (Ctrl+S) / Enregistrer sous…
✂, copier, 📋	Couper, copier, coller
🔍	Rechercher et remplacer (Ctrl+F, Ctrl+H)
⟲ ⟳	Annuler (Ctrl+Z), rétablir (Ctrl+Maj+Z)
A▲▼	Taille du texte : un seul contrôle compact, la lettre A avec deux petits triangles empilés ; ▲ agrandit, ▼ réduit

Viennent ensuite, sur la même rangée (ou la suivante si le panneau est étroit), les actions de l'onglet : ▶ **Lancer**, 📼 **Exporter WAV** et **Mémoire** pour le BASIC, ou 📃 **Listing**, ● **Point d'arrêt** et ⏩ **Jusqu'au curseur** pour une source 6502.

Chaque fichier est un onglet. Un point • signale un onglet modifié ; × le ferme (un second clic est demandé s'il n'est pas enregistré). Un onglet marqué 📚 a été ouvert depuis la bibliothèque : ouvrir un listing ou une source de la bibliothèque (▶, ⬇ ou double-clic) ouvre son onglet quand l'éditeur est visible, ou l'affiche s'il est déjà ouvert. Les onglets, texte non enregistré compris, sont retrouvés au lancement suivant. Un clic droit sur un onglet propose **Enregistrer sous…** et **Fermer**. Dans le texte, un **clic du milieu** colle la sélection primaire (Linux) : le curseur se place là où vous cliquez, puis le dernier texte surligné dans n'importe quelle application est inséré.

La taille du texte se règle avec le contrôle **A▲▼**, Ctrl+molette, ou Ctrl++ / Ctrl+− / Ctrl+0 quand le texte a le clavier.

Détacher met l'éditeur dans sa propre fenêtre, par exemple sur un deuxième écran ; ⊟ **Ancrer** le remet en place. Les touches tapées dans cette fenêtre vont à l'éditeur, jamais à l'Apple-1.

7.2 Rechercher, remplacer et le menu du clic droit

🔍 (ou Ctrl+F) ouvre une barre de recherche au-dessus du texte : Entrée va à l'occurrence suivante, Maj+Entrée à la précédente, **Aa** respecte la casse, **Mot** cherche des mots entiers, et le compteur indique « 3 sur 7 ». Ctrl+H ajoute le champ **Remplacer par**, avec **Remplacer** (l'occurrence courante) et **Tout remplacer**. Échap ferme la barre.

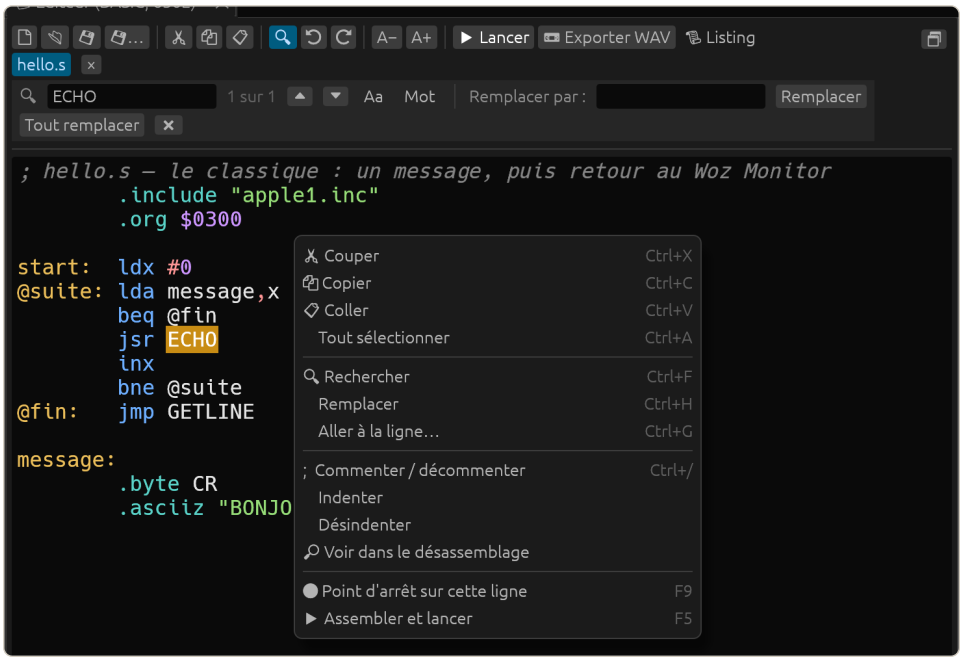


Figure 21 — La barre de recherche ouverte sur ECHO, et le menu du clic droit d'une source 6502.

Le **clic droit** dans le texte ouvre un menu avec toutes les commandes et leur raccourci. Le menu agit sur la **sélection**, qui est conservée quand vous faites un clic droit dessus : **Couper** et **Copier** travaillent sur le texte sélectionné. Les commandes qui reposent sur un mot (aller à la définition, aide) utilisent le **mot sous le clic**, pas celui sous le curseur :

Commande	Raccourci
Couper, Copier, Coller, Tout sélectionner	Ctrl+X, C, V, A
Rechercher, Remplacer	Ctrl+F, Ctrl+H
Aller à la ligne…	Ctrl+G
Commenter / décommenter (6502), REM / dé-REM (BASIC)	Ctrl+/
Indenter, Désindenter	
Aller à la définition d'une étiquette (6502) ou d'un numéro de ligne (BASIC)	F12
Aide sur *mot* (sur un mot-clé, une instruction ou une directive : ouvre sa fiche dans la référence)	
Voir dans le désassemblage (6502)	
Renuméroter… (BASIC : `GOTO`, `GOSUB` et `THEN` suivent)	
Point d'arrêt sur cette ligne	F9

Commande	Raccourci
Assembler et lancer (6502), Envoyer et lancer (BASIC)	F5

Couleurs. Les couleurs du texte suivent le thème de l'interface. ⚙ **Préférences → Éditeur** propose, pour le BASIC et pour l'assembleur 6502 séparément, les palettes *Automatique*, *Classique (sombre)*, *Clair*, *Ambre*, *Contraste élevé*, ou vos propres couleurs, une pastille par sorte de mot (chapitre 11).

7.3 Un programme BASIC

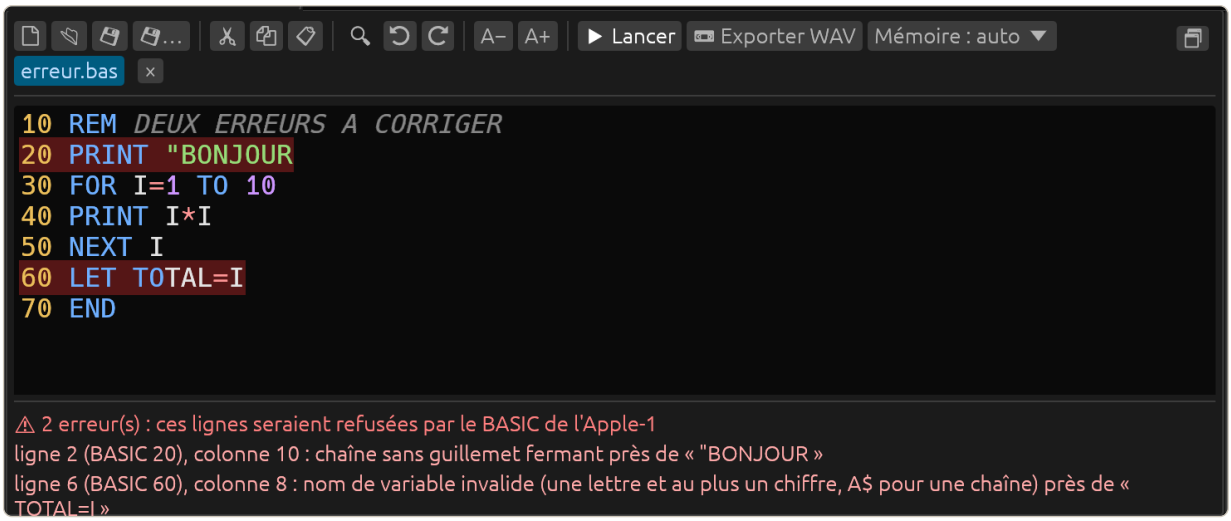


Figure 22 — L'éditeur BASIC : une erreur signalée sur fond rouge, avec son numéro de ligne et sa colonne en bas du panneau.

Le texte est coloré : numéros de ligne, mots-clés, nombres, chaînes, `REM`, opérateurs. Chaque ligne est **tokenisée** comme le ferait l'Integer BASIC, par un tokeniseur natif comparé octet pour octet au vrai. Une ligne que le BASIC refuserait passe sur fond rouge, et l'erreur indique la ligne du texte, le numéro de ligne BASIC et la colonne ; un clic y place le curseur. Erreurs détectées : syntaxe, nom de variable invalide, nombre supérieur à 32767, commande du mode direct (`LIST`, `RUN`…) dans le programme, chaîne non fermée, ligne de plus de 255 octets, mémoire pleine.

- **Mémoire** : automatique (8 Ko avec LOMEM `$0800`, ou `$0300` si le programme est grand, sinon 12, 16 ou 32 Ko), ou imposée.
- ▶ Lancer : charge l'Integer BASIC s'il manque (comme le bouton BASIC), met le programme tokenisé en mémoire, ajoute de la RAM si besoin, puis tape `E2B3R` et `RUN`. C'est instantané : rien n'est tapé ligne à ligne.
- 📼 Exporter WAV : une cassette (`.wav`, ou `.aiff` pour un smartphone) de deux enregistrements, avec sa fiche dans la bibliothèque. Le bas du panneau affiche ce qu'il faut taper sur le vrai Apple-1, par exemple `C100R` puis `004A.00FFR 0C3B.0FFFR`, puis `E2B3R` et `RUN` ; 📋 **Copier** le met dans le presse-papiers.

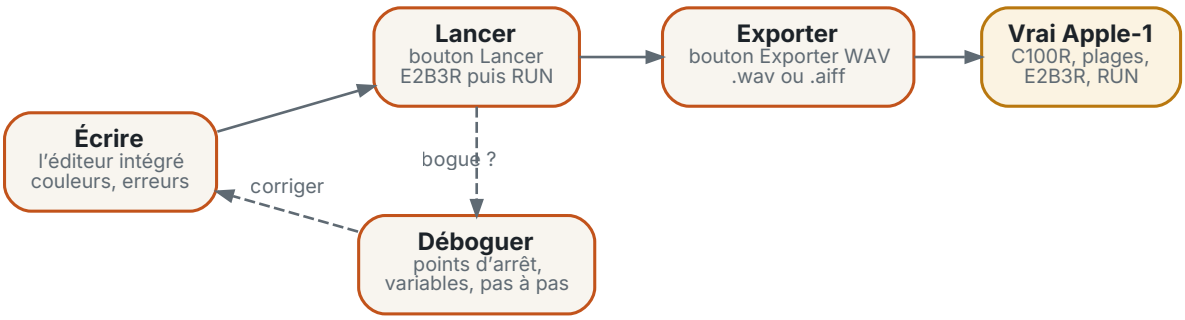


Figure 23 — De l'éditeur au vrai Apple-1 : écrire, lancer, corriger, exporter.

7.4 Une source 6502

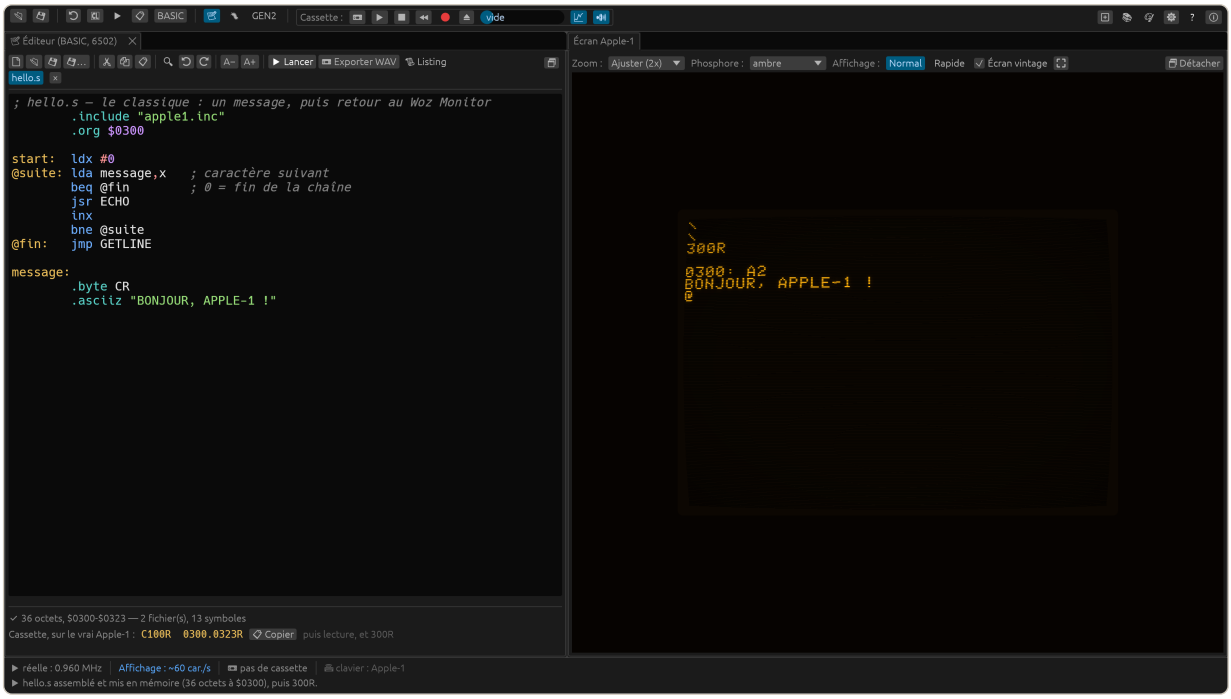


Figure 24 — Une source 6502 dans l'éditeur : les couleurs, le résultat de l'assemblage et la ligne à taper sur le vrai Apple-1.

Une source est assemblée **à chaque modification** par l'assembleur intégré à deux passes, qui accepte la syntaxe de ca65 (et celle de xa, traduite automatiquement). Le bas du panneau affiche la taille et les adresses (`36 octets, $0300-$0323 — 2 fichier(s), 13 symboles`), puis la ligne pour la vraie machine (`C100R` puis `0300.0323R` , puis `300R`).

- **Erreurs** : fichier, ligne et colonne ; un clic y mène, et ouvre le fichier inclus s'il le faut. Les avertissements (page zéro codée en absolu) sont en jaune.
- 📃 **Listing** : adresse, octets et source de chaque ligne, avec les fichiers inclus et les macros développées, puis la table des symboles ; un clic ramène à la ligne.
- ▶ **Lancer** : met les octets en mémoire à l'origine (`.org` , `$0300` par défaut), ajoute de la RAM si besoin, puis tape par exemple `300R` .
- 📼 **Exporter WAV** : la cassette, comme pour le BASIC.
- • **Point d'arrêt** et ⏩ **Jusqu'au curseur** : le lien avec le débogueur (chapitre 8).

- **Lien avec le débogueur** : quand vous sélectionnez une instruction dans la source (ou placez le curseur sur sa ligne), les octets qu'elle a produits sont surlignés dans le **désassemblage** du débogueur et dans le **dump hexa / ASCII**, si ces panneaux sont visibles ; s'ils sont ailleurs en mémoire, les panneaux défilent pour les montrer. La table des lignes source et des adresses de l'assembleur le rend possible.

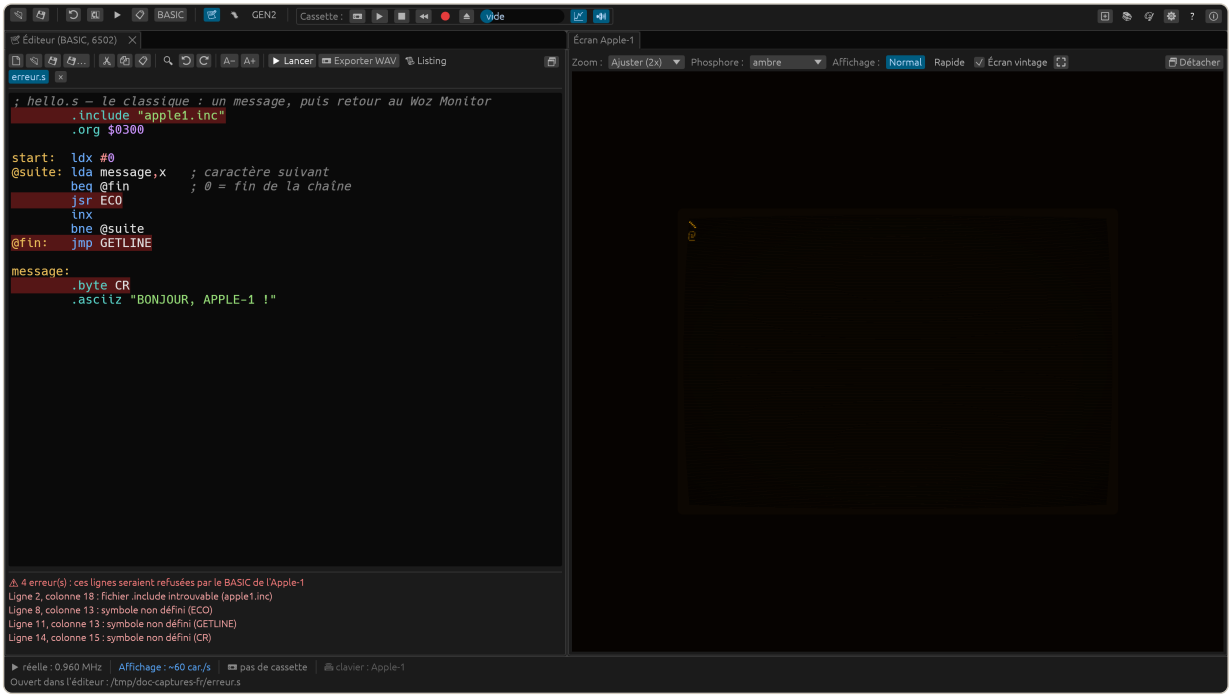


Figure 25 — Une erreur d'assemblage : ECO au lieu de ECHO, symbole inconnu, signalé avec sa ligne et sa colonne.

La partie III est entièrement consacrée à l'assembleur : syntaxe, directives, listing, exemples.

CHAPITRE 8

Le débogueur, le désassemblage et le dump

Le débogueur arrête l'Apple-1 et l'exécute pas à pas, instruction par instruction en 6502 ou ligne par ligne en BASIC, en montrant les registres, les variables et la mémoire.

8.1 Le panneau du débogueur

🐛 **Débogueur** ouvre le panneau, ancré à droite ou détachable comme l'éditeur. Ses commandes :

Commande	Effet
⏸ **Pause** / ▶ **Continuer**	arrête ou relance l'Apple-1
⬇ **Pas**	exécute une instruction
↪ **Par-dessus**	exécute un `JSR` d'un seul coup, et s'arrête au retour
⤴ **Sortir**	continue jusqu'au `RTS` du sous-programme en cours
⏭ **Ligne BASIC**	avance d'une ligne du programme BASIC
⏩ **Jusqu'au curseur**	(dans l'éditeur) continue jusqu'à la ligne du curseur

Le cadre **Processeur** montre A, X, Y, S, PC et les drapeaux N V B D I Z C ; tous sont modifiables, en cliquant sur la valeur. Le nombre de cycles et d'instructions exécutés est affiché dessous.

8.2 Les points d'arrêt

Un point d'arrêt se pose de quatre façons :

- ● **Point d'arrêt** dans l'éditeur (ou `F9`, ou le menu du clic droit), sur la ligne du curseur : dans une source `.s` (grâce à la table des adresses de l'assembleur) ou dans un programme BASIC (numéro de ligne) ;
- un clic dans la marge du 📃 **Listing** ;
- un clic sur une ligne du désassemblage ;
- la saisie d'une adresse ou d'une étiquette : `$0368`, `ASK`, `PUTS+3`, ou `L190` pour la ligne BASIC 190. Une étiquette qui n'existe pas encore n'est pas refusée : le point d'arrêt est gardé et posé dès que l'étiquette existe (quand vous assemblez la source de l'éditeur, ou la lancez), avec un message qui le dit.

Les points d'arrêt, les variables suivies et la mémoire surveillée sont retenus d'une session à l'autre, mais charger un autre programme efface les points d'arrêt et les étiquettes du précédent. Quand le BASIC intégré est chargé, la section BASIC du débogueur s'intitule **RDK G2 Basic** ; sans BASIC en `$E000`, elle le dit sans insister. Les couleurs de la ligne courante, des adresses du dump et de la colonne ASCII se changent dans ⚙ **Préférences → Interface** (« Couleurs de l'interface »). Les panneaux étroits suppriment la colonne des octets du désassemblage et font passer à la ligne la rangée de poke au lieu de la couper.

8.3 Déboguer un programme BASIC

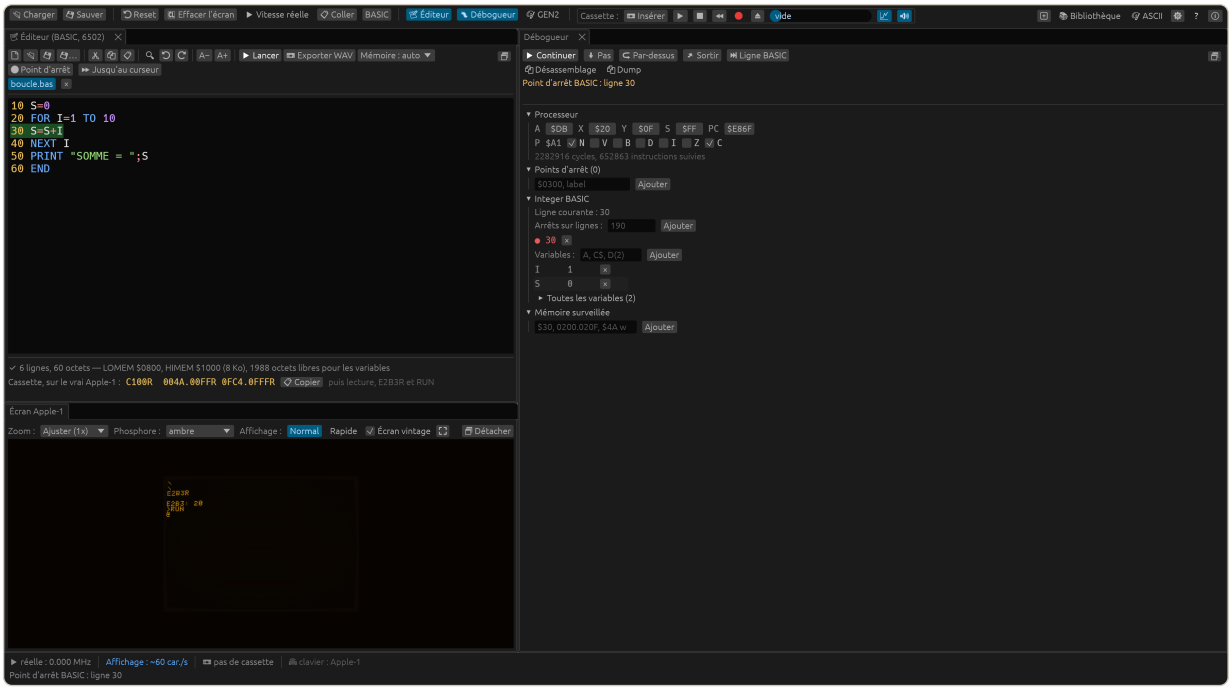


Figure 26 — boucle.bas arrêté sur un point d'arrêt en ligne 30 : la ligne courante en vert dans l'éditeur, les variables I et S suivies dans le débogueur.

La ligne courante est surlignée en vert dans l'éditeur, les points d'arrêt en violet. Le cadre BASIC (**RDK G2 Basic** ou **Integer BASIC**, selon celui qui est chargé) indique la ligne courante, les arrêts sur numéros de ligne et les **variables suivies** : entières (`A`, `B1`), éléments de tableau (`D(2)`), tableau entier (`D`), chaînes (`C$`), plus **Toutes les variables**.

Le débogage marche avec le **RDK G2 Basic**, le BASIC intégré à l'émulateur, comme avec l'original : le vrai Integer BASIC n'est plus nécessaire. Le débogueur remarque le début de chaque ligne (ligne courante, points d'arrêt sur ligne, ⏭ **Ligne BASIC**) et lit les variables dans les tables de l'interpréteur. Pour l'original il lit la ligne dans les pointeurs de page zéro `$DC` / `$DD` ; le G2 Basic la garde ailleurs, et le débogueur connaît les deux organisations.

> **! UNE TOUCHE ARRÊTE LE PROGRAMME**
>
> Comme sur la vraie machine, une touche frappée pendant que le programme BASIC tourne l'arrête (`STOPPED AT`). Pendant une séance de débogage, gardez le clavier dans le panneau du débogueur, pas sur l'écran de l'Apple-1. Un programme G2 Basic peut lire le clavier sans être arrêté avec `KEY(0)` (chapitre 15).

8.4 Déboguer un programme 6502

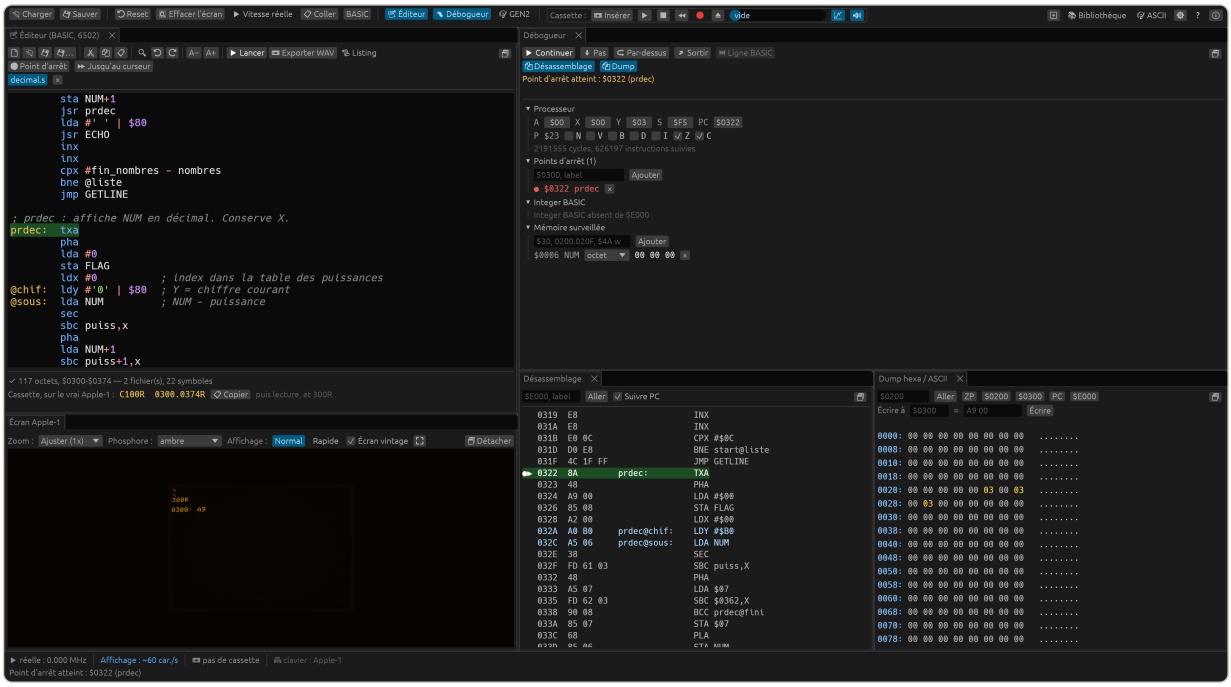


Figure 27 — decimal.s arrêté sur l'étiquette prdec : registres, désassemblage avec les étiquettes du programme, mémoire surveillée $06.$08.

Le **désassemblage** montre 24 lignes autour de PC, avec les étiquettes du programme assemblé dans leur propre colonne. Il ne se recentre que quand PC sort de la zone visible, pour que l'affichage ne saute pas à chaque pas. **Aller** fixe la liste sur une adresse ou une étiquette ; **Suivre PC** la fait de nouveau suivre l'exécution.

Quand vous sélectionnez une instruction dans la source 6502 de l'éditeur, ses octets sont aussi **surlignés** dans le désassemblage et dans le dump (chapitre 7), et les deux panneaux défilent jusqu'à eux s'ils sont hors de vue ; le désassemblage cesse alors de suivre PC jusqu'à ce que vous appuyiez sur **Suivre PC**.

La **mémoire surveillée** affiche un octet, un mot ou de l'ASCII (`$30` , `$30 w` , `0200.020F` , une étiquette), modifiable, en jaune quand la valeur change.

8.5 Le dump

Le **Dump** affiche la mémoire en hexadécimal et en ASCII, 16 octets par ligne (8 si le panneau est étroit), le bit 7 enlevé pour l'ASCII. **Aller** saute à une adresse, ou aux raccourcis ZP, `$0200` , `$0300` , PC, `$E000` . Les octets changés sont en jaune, l'octet de PC sur fond vert. **Écrire** pose des octets (`0300 = A9 00`) ; un clic sur un octet remplit la ligne d'écriture.

Le désassemblage et le dump sont des panneaux à part entière : détachés par défaut, ils peuvent être ancrés près du débogueur (chapitre 10).

8.6 En ligne de commande

Pour les démonstrations et les essais automatiques : `--debogueur` ouvre le panneau, `--dbg-arret ASK,L190` pose des points d'arrêt, `--dbg-surveiller 'A,C$,$30 w'` suit des variables et de la mémoire, et `--dbg-commande 5:pas` envoie une commande après 5 secondes (`pause`, `continuer`, `pas`, `dessus`, `sortir`, `basic`).

CHAPITRE 9

Photo → ASCII

L'outil Photo → ASCII transforme une photo en image de 40 × 24 caractères de l'Apple-1, puis en programme BASIC ou 6502, prêt à être lancé ou exporté sur cassette.

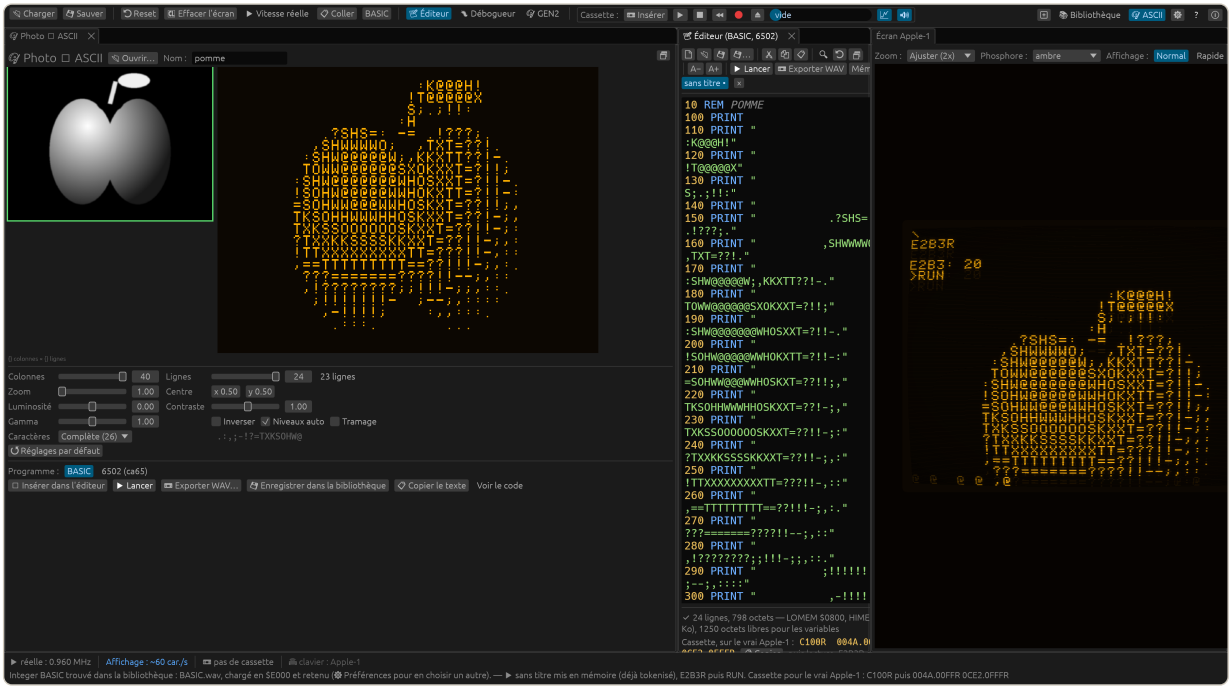


Figure 28 — L'outil Photo → ASCII : la photo et son cadre de recadrage, les réglages, et l'aperçu dessiné avec la police du 2513.

9.1 Ouvrir une image

🎨 **ASCII** ouvre l'outil, un panneau ancrable comme les autres. **Glissez-déposez** une photo `.jpg` ou `.png` sur la fenêtre, ou utilisez 📂 **Ouvrir…** (une photo, ou un art ASCII déjà fait en `.txt`).

Le cadre vert sur la photo montre la partie convertie : faites-le glisser pour le déplacer, la molette zoome.

9.2 Les réglages

Réglage	Rôle
Colonnes, **Lignes**	40 × 24 au plus. **23 lignes** laisse la dernière ligne à l'invite, pour que l'image ne défile pas.
Zoom, **Centre**	la partie de la photo gardée

Réglage	Rôle
Luminosité, **Contraste**, **Gamma**	les réglages habituels (gamma > 1 éclaircit)
Inverser	sombre = beaucoup d'encre
Niveaux auto	étire les niveaux de gris (2 % à 98 %) avant les autres réglages
Tramage	Floyd-Steinberg : plus de nuances, plus de bruit
Caractères	rampe **complète** (26 niveaux), **simple** (10), celle du **diaporama de Schmenk**, ou **personnalisée**

Seuls les caractères du 2513 sont possibles : majuscules, chiffres et symboles, sans minuscules, ni `"`, ni `_`. Ils sont classés par le nombre de points allumés dans la police de l'Apple-1 : c'est ce nombre, et non la forme, qui donne le niveau de gris.

> ✦ **POUR UN PORTRAIT**
>
> 40 × 24 caractères, c'est peu : recadrez sur le visage (zoom 1,3 à 1,6), montez un peu le contraste, et essayez un gamma inférieur à 1 pour une photo claire. Un fond uni et sombre donne les meilleurs résultats.

9.3 Le programme produit

- **BASIC** : une ligne `PRINT "…"` par ligne de l'art. Une ligne pleine de 40 caractères finit par `;`, car l'écran passe tout seul à la ligne. Le programme est lent à taper (environ 2 s par ligne).
- **6502** : une source ca65 en `$0300`, lancée par `300R`. Une boucle envoie à ECHO une table compressée (suites de caractères identiques codées « nombre + caractère »), après avoir vidé l'écran. L'affichage est instantané.

Avec 24 lignes, le programme attend une touche avant de rendre la main ; avec 23, l'invite arrive sur la dernière ligne et l'image reste entière.

Les boutons : ↰ **Insérer dans l'éditeur** (nouvel onglet BASIC ou 6502), ▶ **Lancer**, 📼 **Exporter WAV…**, 💾 **Enregistrer dans la bibliothèque** (un `.txt` dans le dossier « Art ASCII »), 📋 **Copier le texte**, **Voir le code**.

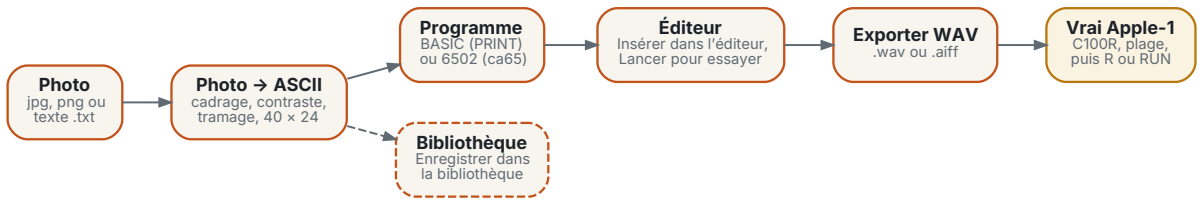


Figure 29 — De la photo au vrai Apple-1 : conversion, programme, cassette.

9.4 En ligne de commande

TERMINAL

```
apple1-emu ascii photo.jpg                      # l'art seul, à l'écran
apple1-emu ascii woz.jpg --zoom 1.3 --cx 0.45 --cy 0.32 \
          --contrast 1.3 --gamma 0.45 --rows 23 -o woz.txt
apple1-emu ascii woz.jpg -o woz.bas             # programme BASIC
apple1-emu ascii woz.jpg -o woz.s               # source 6502
apple1-emu ascii woz.jpg -o woz.wav             # cassette 6502, lancée par 300R
apple1-emu ascii woz.jpg -o woz.wav --format basic
```

Le chapitre 12 donne toutes les options.

▣ LE DIAPORAMA DE DAVE SCHMENK

En 2014, Dave Schmenk a écrit un diaporama de portraits pour l'Apple-1. La bibliothèque l'ajoute comme projet GitHub (`dschmenk/Apple1-Slideshow`) : il est téléchargé, pas fourni, car le dépôt n'a pas de licence. ▶ **Tester le dernier build** l'assemble (syntaxe xa, traduite automatiquement ; mêmes 3456 octets que xa 2.4.1) et le lance par `280R` .

CHAPITRE 10

La disposition des panneaux

Chaque panneau est un onglet que vous placez où vous voulez : côte à côte, en onglets, ou dans sa propre fenêtre sur un deuxième écran.

10.1 Déplacer un panneau

L'éditeur, l'écran de l'Apple-1, le moniteur couleur GEN2, le débogueur, le désassemblage, le dump, l'outil ASCII et la bibliothèque sont tous des **onglets ancrables**. Faites glisser un onglet par son titre : des repères apparaissent, et vous le déposez **à gauche**, **à droite**, **au-dessus**, **en dessous** d'un autre panneau, ou **dans le même groupe d'onglets**. Les séparations se tirent pour redimensionner. La disposition est retenue dans `gui.json`.

10.2 La disposition de départ

L'éditeur est à gauche ; au centre, l'écran de l'Apple-1, avec le moniteur couleur dessous quand la carte GEN2 est branchée ; à droite, le débogueur avec le désassemblage dessous, puis le dump, et la bibliothèque tout à droite. Au tout premier lancement le choix se fait d'après la largeur du **moniteur** (la fenêtre s'ouvre petite et est agrandie un instant plus tard) : en dessous de 1400 points (un portable 1366 × 768, un écran 1024 × 768) elle démarre en **éditeur et écran**, la paire la plus utile ; à partir de 1400 points, avec tous les panneaux. Choisissez une autre disposition dans le menu à tout moment. Elle suit la largeur de la fenêtre : chaque panneau garde sa part, d'un portable à un grand moniteur. Le menu **⊞ Disposition ▾** de la barre d'outils la remet à tout moment (**Par défaut**, tous panneaux ouverts) et propose quatre autres dispositions toutes prêtes : **Jeu** (écran Apple-1 et bibliothèque), **BASIC** (éditeur et écran), **Assembleur** (éditeur, écran et débogueur) et **Débogage** (éditeur, débogueur, désassemblage et dump). **Enregistrer la disposition actuelle…** demande un nom et garde la disposition en cours, avec ses fenêtres détachées et la taille de la fenêtre principale (la charger rouvre ces fenêtres là où elles étaient) ; vos dispositions sont listées sous un séparateur, par ordre alphabétique, et un clic droit permet de les **Renommer** ou de les **Supprimer**. Elles sont retenues dans `gui.json`.

10.3 L'assistant de démarrage

Au premier lancement, un assistant en trois écrans remplace l'ancienne fenêtre « Votre espace de travail ». Il est dessiné en ambre sur fond noir, avec l'effet de tube (balayage, halo, vignettage, courbure 0,07) sur le cadre ; le petit texte et les vignettes restent nets. Une petite croix **×** en haut à droite le ferme. **‹ Précédent** et **Suivant ›** passent d'un écran à l'autre ; le dernier écran a **Terminer**.

1. **Bienvenue** : ce que le kit permet de faire.

2. **ROM et BASIC** : le Woz Monitor, la ROM cassette et G2 Basic sont intégrés. Rien d'autre n'est obligatoire. Les puristes installent leur Integer BASIC original depuis une source externe ; le bouton ouvre le guide (chapitre « Installer une ROM »), qui ne contient que des liens, jamais les fichiers.
3. **Votre espace de travail** : les cinq dispositions (**Par défaut**, **Jeu**, **BASIC**, **Assembleur**, **Débogage**). Chaque vignette est cliquable (pas de bouton « Choisir »). Les rectangles gardent les numéros 1 à 6, et l'écran couleur (GEN2 / P-LAB) est le **7**, dans sa propre couleur, dans les vignettes qui l'affichent et dans la légende.

La case **Afficher cet assistant au démarrage** est sur les trois écrans, cochée par défaut. La même case est dans ⚙ **Préférences → Interface**. ⊞ **Disposition ▾ → Assistant de démarrage…** le rouvre. C'est une fenêtre ordinaire : elle ne reste pas au-dessus des autres. Elle ne s'ouvre pas quand la ligne de commande impose une taille (`--taille`), ni pour les captures.

10.4 La taille du texte de chaque fenêtre

L'éditeur, la liste produite par l'assembleur, le débogueur, le désassemblage, le dump et la bibliothèque ont chacun un contrôle compact **A 13 ▲▼** dans leur barre : les triangles changent la taille, qui est retenue pour cette fenêtre. Ctrl+molette et Ctrl+plus / Ctrl+moins agissent sur la fenêtre qui est sous la souris (sur l'écran de l'Apple-1, Ctrl+plus / Ctrl+moins changent le zoom). **Réinitialiser les préférences** remet toutes ces tailles à 13.

10.5 Ce qui est retenu d'un lancement à l'autre

La disposition est enregistrée toute seule quand vous quittez et retrouvée telle quelle : position et taille de chaque panneau, fenêtres détachées, zoom et disposition choisie. La fenêtre est rétablie à sa taille et à sa place, puis les panneaux reprennent les mêmes proportions, quel que soit l'écran : la restauration a été vérifiée à 2880 × 1620, 2560 × 1440, 1920 × 1080 et 1366 × 768, avec moins de 1 % de l'image différente après un relancement. Une disposition enregistrée sous un nom (**Enregistrer la disposition actuelle…**) garde en plus les fenêtres détachées. L'interface ne se zoome jamais toute seule : seul le réglage « Taille des icônes et du texte » change la taille des boutons.

10.6 Quand l'écran est plus grand que son volet

Avec un **zoom fixe** (2x, 3x…, ou Ctrl+`+`), l'écran de l'Apple-1 garde la taille demandée, même si le volet est plus petit : il **défile** alors (molette ou barres) au lieu d'être réduit, et la barre d'état le dit, avec le conseil d'agrandir le volet. Seul **Ajuster** adapte le zoom au volet. Dans une fenêtre étroite (1024 × 768, par exemple), la barre d'outils défile horizontalement au lieu d'être coupée : le bouton **»** amène le groupe Cassette, un second clic revient au début. Le bouton **Lancer** de l'éditeur reste toujours visible.

10.7 Détacher un panneau

Chaque panneau a un bouton **Détacher** en haut à droite (aussi par clic droit sur l'onglet) qui l'ouvre dans **sa propre fenêtre**, déplaçable sur un deuxième écran ; sa taille et sa place sont retenues. ⊟ **Ancrer** le remet à sa place de départ.

- L'écran de l'Apple-1 et le moniteur couleur se détachent séparément. Le clavier va à l'Apple-1 depuis l'une ou l'autre de leurs fenêtres, et depuis la fenêtre principale. Fermer la fenêtre d'un écran le ré-ancre.
- Fermer la fenêtre d'un outil (éditeur, débogueur…) le ferme ; son bouton de la barre d'outils le rouvre au même endroit.
- Les boîtes de dialogue (Ouvrir, Enregistrer sous, Exporter) s'ouvrent toujours dans la fenêtre principale. Dans les fenêtres de fichiers, Échap ferme la fenêtre (même depuis le champ du nom), Ctrl+L modifie le chemin (son texte est sélectionné : taper le remplace) ; Enregistrer sous et Exporter d'un onglet démarrent dans le dossier de son fichier.
- Les dispositions enregistrées gardent les fenêtres détachées ; la fenêtre principale garde la même taille d'un lancement à l'autre, quel que soit le zoom.
- L'oscilloscope de la cassette reste une petite fenêtre flottante dans la fenêtre principale.

Pour jouer

Fermez l'éditeur et le débogueur, gardez la bibliothèque. **Ajuster** agrandit l'écran ; F11 passe en plein écran, avec l'écran vintage.

Pour programmer

Éditeur à gauche, écran au centre, débogueur et dump à droite. Détachez l'éditeur sur un deuxième moniteur pour lui donner toute la hauteur.

Pour la GEN2

Branchez la carte : le moniteur couleur se place sous l'écran de l'Apple-1. Détachez-le pour l'avoir en grand, comme un second moniteur.

Pour présenter

Détachez l'écran de l'Apple-1 sur le vidéoprojecteur, en plein écran, la fenêtre principale sur votre portable.

CHAPITRE 11

Les préférences

⚙ *Préférences regroupe tous les réglages durables. Tout est retenu dans ~/.config/apple1-emu/gui.json (ou dans le dossier du kit, en mode portable).*

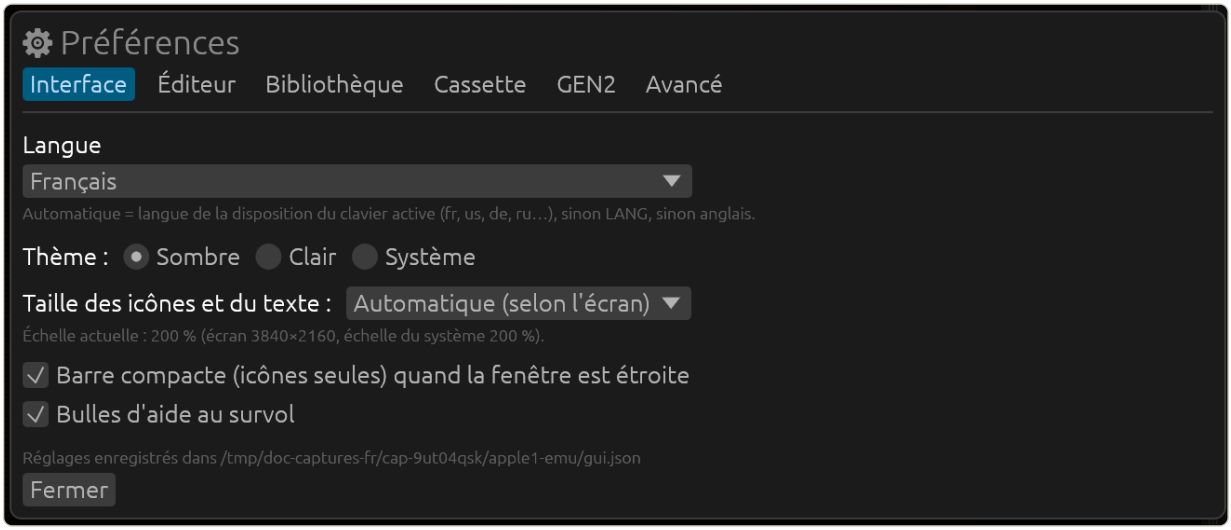


Figure 30 — L'onglet Interface des préférences : langue, thème, taille des icônes et du texte, barre compacte, bulles d'aide, fenêtre de démarrage et couleurs de l'interface.

Les réglages sont répartis en six onglets :

Onglet	Réglages
Interface	langue : automatique (selon la disposition du clavier), ou imposée : français, anglais, allemand, russe, espagnol, italien, portugais ; la langue détectée et sa source sont indiquées. Thème : sombre, clair ou système. Taille des icônes et du texte : petite, normale (par défaut), grande, très grande. Barre compacte (icônes seules quand la fenêtre est étroite). Bulles d'aide au survol. **Afficher cette fenêtre au démarrage** (la fenêtre « Votre espace de travail ») et le bouton qui l'ouvre tout de suite. **Couleurs de l'interface** : les couleurs des listes de l'interface, choisies **séparément pour le thème sombre et pour le thème clair** (boutons actifs, ligne courante du débogueur, ligne surlignée, cellule sélectionnée, ligne d'un point d'arrêt, adresses et colonne ASCII du dump, valeurs changées, étiquettes) ; chaque couleur a une pastille avec aperçu immédiat, les valeurs par défaut sont lisibles (contraste d'au moins 4,5:1 sur le fond du thème), et des boutons **Défaut** remettent un thème ou les deux.
Éditeur	couleurs de la syntaxe, pour le BASIC et pour l'assembleur 6502 séparément (chapitre 7)
Bibliothèque	le dossier principal (**Parcourir…**, **⟲ Par défaut**, **Ouvrir le dossier**), les autres dossiers, les projets GitHub et le jeton GitHub facultatif (seulement pour les dépôts privés et les artefacts Actions ; jamais affiché, et la variable `GITHUB_TOKEN` passe en priorité) ; position de la bibliothèque, à gauche ou à droite (à droite par défaut)
Cassette	son de la cassette (muet en vitesse rapide) et son volume, oscilloscope pendant la lecture ou l'écriture

Onglet	Réglages
GEN2	carte couleur d'Uncle Bernie, balayage 60 ou 50 Hz, palette LORES (chapitre 6)
Avancé	affichage et vitesse : afficher la bibliothèque, affichage limité à ~60 caractères/s comme le vrai terminal, vitesse rapide ; **Écran vintage : réglages fins** (communs aux deux écrans, chapitre 3) ; le BASIC chargé en `$E000` : le **RDK G2 Basic** intégré (par défaut) ou l'**Integer BASIC d'origine** (votre propre fichier, 📂 **Choisir…** / **Oublier**)

Chaque curseur des préférences (intensité de l'écran vintage, rémanence et courbure, volume de la cassette) affiche sa **valeur numérique** à côté de lui. Au premier lancement la courbure de l'écran vintage est réglée sur **0,07** (un léger bombé du tube) ; une installation existante garde sa valeur. L'aperçu de l'onglet **Éditeur** suit la langue de l'interface.

Le zoom, le phosphore et la case de l'écran vintage ne sont pas dans les préférences : ils sont dans la rangée d'affichage de chaque écran (chapitre 3).

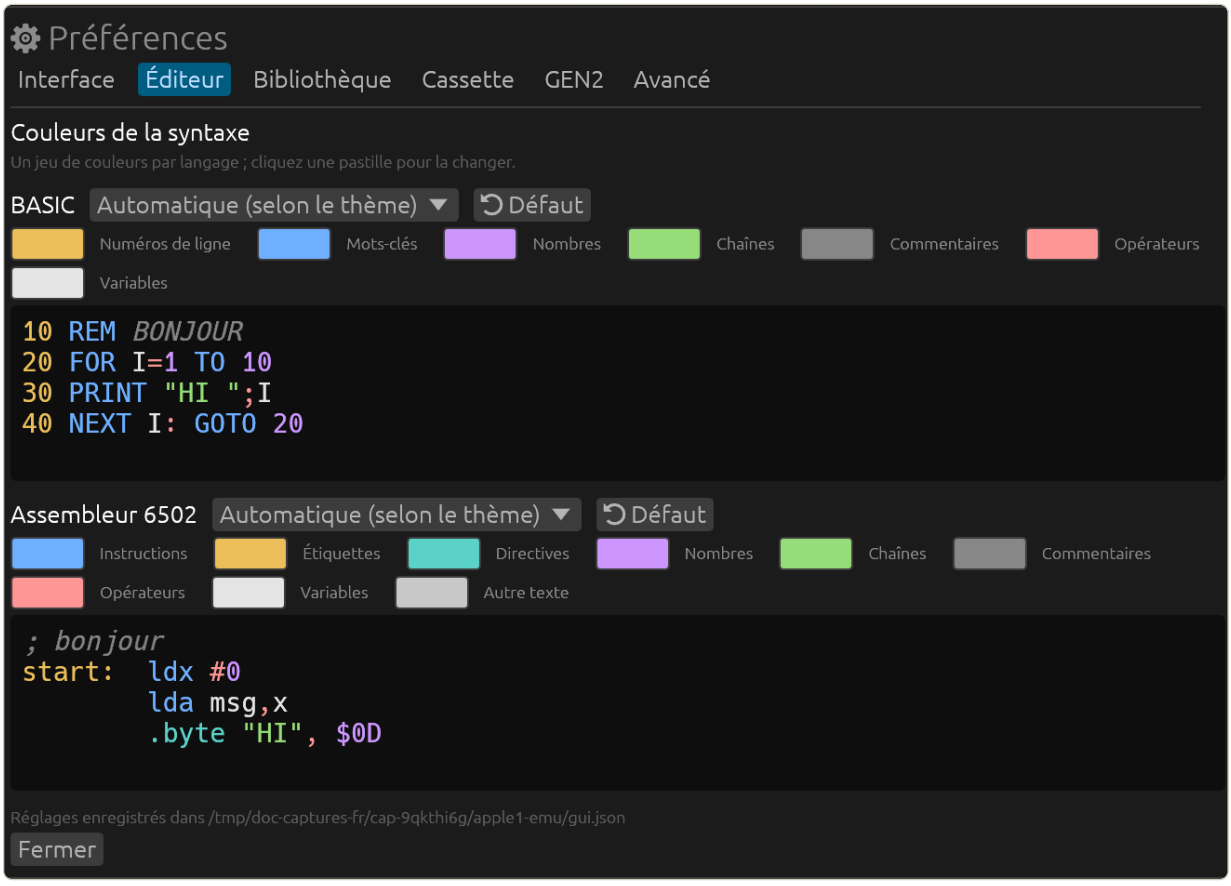


Figure 31 — L'onglet Éditeur : une palette de couleurs pour le BASIC et pour l'assembleur 6502, ou vos propres couleurs.

La langue automatique. L'émulateur suit la disposition du clavier active : `fr` donne le français, `us` ou `gb` l'anglais, `de`, `ch` ou `at` l'allemand, `ru` le russe, etc. Il l'interroge auprès de GNOME, de `setxkbmap`, de Cinnamon (cas de Linux Mint), de `localectl`, puis de `/etc/default/keyboard`, et enfin des variables `LANG`. En ligne de commande : `--langue fr|en|de|ru|es|it|pt|auto`.

✦ REPARTIR DE ZÉRO

Réinitialiser les préférences, en bas de la fenêtre, remet les réglages d'origine après confirmation (langue, thème, disposition, écran vintage…). Vos fichiers de bibliothèque restent en place ; les onglets de l'éditeur qui ne sont pas enregistrés sont fermés. Au premier lancement d'une nouvelle version du kit, la même question est posée, parce que `gui.json` survit quand on efface le dossier du programme pour en installer un autre.

Pour un kit portable, placez un fichier vide nommé `portable` à la racine du kit (à côté de `run.sh`), ou lancez-le avec `APPLE1_PORTABLE=1` : les préférences (`config/gui.json`), la bibliothèque par défaut (`programmes`) et le cache sont alors dans le dossier du kit.

Fermer l'émulateur et renommer `~/.config/apple1-emu/gui.json` revient au même que le bouton, au lancement suivant. Les fiches des programmes restent dans les `programs.json` de chaque dossier.

CHAPITRE 12

La ligne de commande

Deux programmes : apple1-emu-gui, l'interface graphique, et apple1-emu, l'émulateur en terminal et la boîte à outils de conversion. Tout ce que font les boutons d'export se fait aussi en ligne de commande, pour vos scripts.

12.1 L'émulateur en terminal

`apple1-emu` sans commande lance l'Apple-1 dans le terminal (au moins 42 colonnes × 28 lignes) : 8 Ko de RAM plus `$E000`, ACI, vitesse réelle.

TERMINAL

```
apple1-emu                                  # 8 Ko + E000, ACI, vitesse réelle
apple1-emu --fast                           # démarrer en mode rapide
apple1-emu --load samples/hello.txt --run 300
apple1-emu --tape samples/hello.wav         # cassette dans le lecteur virtuel
apple1-emu --basic ~/apple1/integer_basic.bin
apple1-emu --ram 32 --no-e000               # 32 Ko, pas de RAM en E000
```

Option	Effet
`--ram KO`	RAM à partir de `$0000`, de 4 à 48 Ko (8 par défaut)
`--no-e000`	pas de RAM en `$E000-$EFFF` (le cavalier « W → E » de la carte)
`--no-aci`	pas de carte cassette
`--basic FICHIER`	votre Integer BASIC (binaire de 4 Ko ou vidage hex), chargé en `$E000`
`--load FICHIER[@ADR]`	un programme en mémoire (`.txt` , `.bin@ADR` , `.wav` , `.aif`) ; répétable
`--tape FICHIER`	une cassette dans le lecteur
`--type FICHIER`	tape un fichier texte au démarrage
`--run ADR`	tape `ADRR` après le chargement
`--fast` , `--no-throttle`	mode rapide ; affichage sans la limite de 60 caractères/s
`--no-refresh`	sans le modèle de rafraîchissement DRAM (1,0227 MHz au lieu de 0,960)
`--carte-couleur` , `--couleur-50hz`	la carte GEN2 (registres et mémoire ; l'image couleur n'existe que dans l'interface graphique)
`--cursor-hz HZ` , `--cursor-duty F`	clignotement du curseur (2,18 Hz, 2/3 visible par défaut)
`--monitor-rom` , `--monitor-page`	une autre image du Woz Monitor (256 ou 512 octets, page 0 ou 1)

Option	Effet
`--aci-rom`, `--aci-page`	une autre image de la PROM de l'ACI

Touches du terminal. Ctrl+R RESET, Ctrl+L CLEAR SCREEN, F1 aide, F2 ligne de commande, F3 vitesse, F4 limite d'affichage, F5/F6 cassette lecture/stop, F10 ou Ctrl+Q quitter. Un texte collé dans le terminal est tapé au clavier de l'Apple-1. Le retour arrière tape `_`, l'effacement du Woz Monitor.

Commandes de F2.

Commande	Effet
charger `FICHIER[@ADR]`	charge en mémoire un `.txt`, un `.bin` (`@ADR` obligatoire), un `.wav` ou un `.aiff`
sauver `FICHIER DEBUT.FIN …`	sauve la mémoire ; l'extension choisit le format. Les plages peuvent être recopiées depuis la ligne ACI : `sauver st.aiff 004A.00FFR 0300.0FFFR`
cassette `FICHIER`	insère une cassette, qui démarre à la première lecture de l'ACI
`lecture`, `stop`, `rembobiner`, `ejecter`	le lecteur
enregistrer `FICHIER`	arme l'enregistreur ; tapez ensuite `C100R` puis `0300.03FFW`
`basic FICHIER`	charge votre Integer BASIC en `$E000`
`taper FICHIER`	tape un fichier texte au clavier
`reset`, `effacer`, `rapide`, `reel`, `debit`, `aide`, `quitter`	—

12.2 Les conversions

Le format de sortie est choisi par l'**extension** : `.aif` / `.aiff` donne un AIFF (profil smartphone), `.wav` un WAV, `.txt` / `.hex` un vidage Woz, `.bin` un binaire.

export (alias de `export-wav`) produit une cassette. Le signal est créé par la vraie ROM de l'ACI exécutée dans le 6502 émulé : les temps de bits sont ceux d'une Apple-1 qui ferait `W`.

TERMINAL

```
apple1-emu export samples/hello.txt -o hello.aiff
apple1-emu export prog.bin@300 -o prog.wav --rate 48000 --bits 16
apple1-emu export startrek.txt --range "004A.00FFR 0300.0FFFR" -o startrek.aiff
```

Option	WAV (défaut)	AIFF (défaut)
`--rate`	44 100 Hz	48 000 Hz
`--bits 8|16`	16	16
`--channels 1|2`	1	2, identiques
`--amplitude` (0,05 à 1)	0,8	0,89 (−1 dBFS)
`--silence S`	0,5 s	1,0 s

Option	WAV (défaut)	AIFF (défaut)
`--range`	une plage, ou toute une ligne ACI ; répétable	
`--no-refresh`, `--no-embed`	bits 6 % plus courts ; pas d'adresses dans le fichier	

import (alias de `import-wav`) décode une cassette : `-o prog.txt` ou `-o prog.bin`, `--addr ADR` pour une cassette sans adresses, `--range` pour placer plusieurs enregistrements.

info décrit un fichier (enregistrements, adresses, durée) ; avec `--aci-input stock|mimeo|gen2`, `--volts` et `--series`, il simule l'entrée analogique de l'ACI (section 5.8).

convert `ENTRÉE SORTIE` convertit entre `.txt`, `.bin`, `.wav` et `.aiff`.

12.3 BASIC et assembleur

TERMINAL

```
apple1-emu basic jeu.bas -o jeu.wav            # listing tokenisé -> cassette
apple1-emu basic jeu.bas -o jeu.txt --ram 16   # vidage Woz, pour 16 Ko
apple1-emu basic jeu.bas                       # vérifier seulement
apple1-emu asm jeu.s -o jeu.wav                # source 6502 -> cassette
apple1-emu asm jeu.s -o jeu.bin -l jeu.lst     # binaire + listing
apple1-emu asm jeu.s -I ../lib                 # vérifier, includes en plus
```

`basic` tokenise un listing comme l'interpréteur et écrit le bloc `$004A-$00FF` puis le programme ; `--ram 8|12|16|32` choisit la machine cible (par défaut, la plus petite qui suffit). `asm` assemble une source ca65 (ou xa) ; sans `-o`, il vérifie seulement. Les deux affichent la ligne à taper sur le vrai Apple-1.

> Sortie de apple1-emu asm hello.s -o hello.txt
>
> ```
> 36 octets, $0300-$0323 ; 2 fichier(s), 13 symboles
> Cassette (après C100R) : 0300.0323R   puis 300R
> écrit : hello.txt
> ```

12.4 Art ASCII

ascii `IMAGE` : `--cols`, `--rows` (23 pour garder l'invite), `--zoom`, `--cx`, `--cy`, `--brightness`, `--contrast`, `--gamma`, `--invert`, `--dither`, `--no-auto-level`, `--ramp simple|full|slideshow|LISTE`, `--format asm|basic`, `--title`, `-o` (`.txt`, `.bas`, `.s`, `.wav`, `.aif`).

12.5 PROM

prom prépare des fichiers pour programmer de vraies PROM 256 × 4 (82S129, 3601, 6301), dans le format des vidages de quartets :

```
apple1-emu prom split woz -o proms/     # PROM-A2.bin (D7-D4) et PROM-A1.bin (D3-D0)
apple1-emu prom split aci -o proms/     # PROM-A3.bin (D7-D4) et PROM-A4.bin (D3-D0)
apple1-emu prom join A2.bin A1.bin      # réassembler une lecture et la comparer
```
TERMINAL

`--swap-a1-a7` ne sert que pour une carte câblée d'après le schéma Apple de 1976, qui croise les lignes d'adresse 1 et 7 de la PROM A2.

Pour graver une EPROM 2716 à 27C512 ou une EEPROM 28C depuis l'interface graphique (programmateur TL866II+, T48 ou T56 avec `minipro`), voir le programmateur d'EPROM, section 2.6.

12.6 L'interface graphique

`apple1-emu-gui [OPTIONS] [FICHIER…]` charge les fichiers donnés au démarrage. Options principales :

Option	Effet
`--zoom N|ajuster`, `--taille LxH`	zoom de l'écran, taille de la fenêtre
`--phosphore vert|blanc|ambre`	phosphore
`--screening`, `--sans-screening`, `--screening-options LISTE`	écran vintage de l'Apple-1 et ses réglages
`--gen2-screening`	écran vintage sur le moniteur couleur GEN2
`--theme clair|sombre|systeme`	thème clair, sombre ou système
`--taille-icones auto|petite|normale|grande|tres-grande`	taille des icônes et du texte
`--barre-complete`	ne jamais réduire la barre d'outils aux icônes
`--bibliotheque DOSSIER`, `--basic FICHIER`	dossier de la bibliothèque, fichier du BASIC
`--carte-couleur`, `--sans-carte-couleur`, `--couleur-50hz`	carte GEN2
`--langue CODE|auto`	langue de l'interface
`--mode-rapide`, `--mode-cassette`	chargement de la bibliothèque : instantané (par défaut), ou par la cassette en vitesse réelle
`--cassette FICHIER`	insère une cassette
`--ecran-separe`, `--debogueur`	écran dans sa fenêtre, débogueur ouvert
`--editeur FICHIER`, `--ascii FICHIER`	ouvre un fichier dans l'éditeur ou l'outil ASCII

Pour les démonstrations et les captures, des options rejouent des actions après quelques secondes : `--lancer-biblio S:NOM`, `--charger-biblio S:NOM`, `--parametres S:NOM`, `--preferences S` (et `--onglet-prefs interface|editeur|bibliotheque|cassette|gen2|avance`), `--editeur-lancer S`, `--editeur-recherche TEXTE` (barre de recherche ouverte), `--editeur-menu` et `--ecran-menu` (menus du clic droit ouverts), `--menu-etat` (le menu État ouvert), `--eprom` (le programmateur d'EPROM ouvert), `--fenetre-fichiers` (la fenêtre Charger ouverte), `--editeur-exporter S:FICHIER`, `--ascii-reglages R`, `--ascii-lancer S`, `--taper TEXTE`, `--taper-a S:TEXTE`, `--dbg-arret`, `--dbg-surveiller`, `--dbg-commande S:CMD`, `--capture IMAGE.png` (après `--delai S`) et `--quitter`.

CHAPITRE 13

Dépannage et questions fréquentes

13.1 La fenêtre ne s'ouvre pas

Le pilote OpenGL manque, ou vous êtes dans une machine virtuelle. `run.sh` relance automatiquement avec le rendu logiciel de Mesa ; à la main : `LIBGL_ALWAYS_SOFTWARE=1 ./apple1-emu-gui`. L'écran vintage reste fluide en rendu logiciel.

13.2 Les touches n'arrivent pas à l'Apple-1

Cliquez sur l'écran de l'Apple-1. Quand un champ de texte ou une boîte de dialogue est actif, le clavier va à ce champ ; la barre d'état indique la destination du clavier.

13.3 « Integer BASIC introuvable »

Le BASIC n'est pas fourni. Mettez votre `BASIC.wav` (ou un `.bin` de 4096 octets) dans la bibliothèque, ou désignez-le par le bouton **BASIC** ou ⚙ **Préférences → Integer BASIC**. L'émulateur le cherche ensuite tout seul.

13.4 Un programme BASIC s'arrête avec STOPPED AT

Une touche a été frappée pendant l'exécution : sur l'Apple-1, n'importe quelle touche interrompt un programme BASIC. Tapez `RUN` pour recommencer, ou `GOTO` suivi du numéro de ligne pour reprendre sans effacer les variables.

13.5 Le programme ne se lance pas après le chargement

- Programme BASIC : il faut `E2B3R` (entrée à chaud) et non `E000R`, qui efface le programme. Le BASIC doit être en mémoire **avant** le programme.
- Langage machine : vérifiez l'adresse de lancement dans la fiche ⚙ ; ce n'est pas toujours `300R` (`280R`, `1000R`, `2000R`…).
- Programme de plus de 8 Ko : la RAM est ajoutée par ▶, mais pas par un chargement à la main. Lancez par ▶, ou utilisez `--ram 16`.

13.6 La cassette ne se lit pas sur le vrai Apple-1

1. Volume au maximum, tous les effets audio coupés, aucune notification.
2. RETURN pendant l'en-tête, pas avant le début de la lecture ni après la fin de l'en-tête.

3. Les plages exactes, sur quatre chiffres : `004A.00FFR`, jamais `4A.FFR`.
4. Essayez un autre lecteur (VLC), puis `--rate 44100`, `--channels 1`, ou `--no-refresh`.
5. Vérifiez le niveau avec `apple1-emu info prog.aiff --aci-input stock` (section 5.8).
6. Si la lecture ne finit jamais : RESET, puis recommencez avec plus de volume.

13.7 L'écran est plein de @ en inverse

C'est le moniteur couleur de la GEN2 : sa mémoire démarre à zéro, et le code `$00` est un `@` en inverse. Le programme n'a pas encore rempli la page texte. Sur la vraie carte, ce serait un contenu aléatoire.

13.8 Les caractères sortent lentement

C'est normal : le terminal de l'Apple-1 affiche environ 60 caractères par seconde. Cliquez sur **Affichage ~60 car./s** dans la barre d'état (ou **Rapide** dans la rangée d'affichage de l'écran) pour un affichage instantané, et passez en vitesse rapide (`F3`) pour tout accélérer.

13.9 Le son de la cassette est muet

Il est coupé en vitesse maximale, et quand ALSA ou la carte son manquent : l'oscilloscope l'indique (« muet en vitesse maximale », « pas de sortie son »). Le volume se règle dans l'oscilloscope ; 🔊 dans la barre d'outils coupe ou remet le son.

13.10 Le projet GitHub ne se construit pas

« Tester le dernier build » compile avec ca65/ld65 (paquet `cc65`) s'il ne trouve ni Release ni artefact. Installez `cc65`, ou ajoutez un `apple1.toml` qui désigne le bon fichier. Certains sources demandent un ca65 récent (2.19 ou plus) ; l'erreur de ca65 s'affiche alors telle quelle.

13.11 Ce que l'émulateur ne fait pas

- Les opcodes non documentés du 6502 sont traités comme des NOP de la bonne taille, et les opcodes JAM bloquent le processeur ; leurs effets exacts ne sont pas reproduits (le logiciel de l'Apple-1 ne les utilise pas).
- Le comparateur de l'ACI est idéal pendant l'émulation ; seule la commande `info` le simule en analogique.
- L'émulation a été comparée aux ROM, aux listings et aux outils indépendants (c2t, ca65, xa), mais les fichiers audio n'ont pas encore été joués sur un vrai Apple-1 par l'auteur de ce manuel.

CHAPITRE 14

Installer une ROM ou l'Integer BASIC original

Rien n'est obligatoire. Le Woz Monitor, la ROM de la cassette ACI et le RDK G2 Basic sont déjà dans le programme. Cette page est pour qui veut l'Integer BASIC d'Apple, ou une autre ROM compatible, depuis une source extérieure.

Le kit **ne redistribue aucun fichier ROM propriétaire**. Apple-1, l'Integer BASIC de 1976 et les cassettes du commerce restent à leurs ayants droit. Les liens ci-dessous mènent vers des sites qui existent déjà : vous y prenez votre propre copie, le kit ne la contient pas.

14.1 Ce qui est déjà là

- **Woz Monitor**, en `$FF00`.
- **ROM cassette ACI**, en `$C100`.
- **RDK G2 Basic**, un BASIC libre compatible avec l'Integer BASIC, chargé par le bouton **BASIC** en `$E000`.

14.2 Integer BASIC original

1. Procurez-vous votre copie, par exemple la cassette `BASIC.wav` de la collection « Apple1Cassettes » sur archive.org : https://archive.org/details/Apple1Cassettes.
2. Rangez le fichier dans votre bibliothèque, ou gardez-le où vous voulez.
3. Ouvrez ⚙ **Préférences** → **Avancé** → **BASIC**, choisissez **Integer BASIC d'origine**, puis 📂 **Choisir…** et désignez le fichier (`.wav` , `.txt` ou `.bin` de 4 096 octets).
4. Le bouton **BASIC** charge alors ce fichier. **Oublier** revient au G2 Basic intégré.

La communauté Apple-1 se retrouve sur https://www.applefritter.com/. Le kit ne télécharge rien depuis ces sites : le lien est seulement une indication.

PARTIE II

L'Integer BASIC

Le BASIC de Steve Wozniak et son clone libre RDK G2 Basic : toutes les instructions, la mémoire, les messages d'erreur, les bugs connus et leurs contournements, et le portage depuis l'Applesoft

CHAPITRE 15

Premiers pas en BASIC

L'Integer BASIC de l'Apple-1 tient en 4 Ko. Il ne connaît que les entiers, des noms de variables d'une lettre, et pas de DATA ni de fonctions de chaînes : il est petit, rapide, et a ses pièges.

> ▣ **D'OÙ VIENT CE BASIC**
>
> Steve Wozniak l'a écrit en 1976, à la main, en assembleur, sans ordinateur pour l'assembler. Il était vendu sur cassette (75 $) et se charge en `$E000-$EFFF`. L'émulateur ne fournit pas l'original d'Apple : utilisez votre propre `BASIC.wav` ou image de 4 Ko (chapitre 4) si vous le voulez. Tous les essais de cette partie ont été faits dans l'émulateur, avec cet original.

15.1 Deux BASIC : le RDK G2 Basic et l'original

Depuis la version 7.9, l'émulateur a son propre BASIC, le **RDK G2 Basic**, intégré au programme. C'est un clone « salle blanche » de l'Integer BASIC, écrit de zéro en assembleur 6502 (il ne contient aucun code de l'interpréteur d'Apple) et distribué sous licence MIT. Il est dédié à Uncle Bernie.

- **Compatible** : même syntaxe, mêmes jetons, même format de programme en mémoire, même format de cassette. Les programmes, listings et cassettes écrits pour l'original tournent sans changement. Il a été comparé à l'original avec 22 séances d'essai dont les sorties sont identiques.
- **Corrigé** : les bugs connus de l'original, décrits au chapitre 18, sont corrigés.
- **Plus gros** : 7 933 octets au lieu de 4 Ko. Il a besoin de RAM de `$E000` à `$FEFF` ; l'émulateur la fournit, mais il ne tiendrait pas dans la fenêtre d'EPROM de 4 Ko de l'original.
- **Conscient de la GEN2** : au démarrage il cherche la carte GEN2 et, s'il la trouve, lance le pilote de la carte (puis place LOMEM en `$2000`, au-dessus des pages graphiques). Les instructions graphiques elles-mêmes sont prévues pour la version 2 du G2 Basic.
- **Bandeau** : `RDK G2 BASIC 1.0`, puis `DEDICATED TO UNCLE BERNIE`.
- **Étendu** (version 7.9.2) : `LOMEM=` et `HIMEM=` fonctionnent comme dans l'original, et une nouvelle fonction `KEY(n)` lit le clavier sans attendre, pour qu'un jeu puisse regarder les touches sans que `INPUT` le bloque et sans qu'une touche arrête le programme (chapitre 15). `LIST a,b` et `DEL a,b` utilisent les plages de l'original, et une ligne listée plus longue que l'écran passe à la ligne comme dans l'original.

Le bouton **BASIC** charge le G2 Basic. Pour utiliser l'original d'Apple à la place, ouvrez ⚙ **Préférences → Avancé → BASIC**, choisissez **Integer BASIC d'origine (votre propre fichier, chargé en $E000)** et désignez le fichier ; le choix est retenu, et le bouton charge alors celui-là. Les deux se comportent de la même façon pour tout ce qui est décrit dans cette partie ; les différences sont signalées là où elles comptent (HIMEM au démarrage, les bugs du chapitre 18).

> **SOURCE ET ESSAIS**
> Le G2 Basic a son propre dépôt, `Old-Wirehead/rdk-gen2basic-apple1`, avec son manuel, son source et ses tests de non-régression.

15.2 Démarrer le BASIC

Cliquez sur **BASIC** dans la barre d'outils : l'émulateur charge le BASIC que vous avez choisi (le G2 Basic par défaut) et vous le démarrez avec `E000R`. L'invite du BASIC est `>`, celle du Woz Monitor `\`.

```
WOZ MONITOR
\
E000R
E000: 4C
>
```

- `E000R` : **démarrage à froid**, efface le programme en mémoire.
- `E2B3R` : **démarrage à chaud**, garde le programme. C'est l'adresse à utiliser après le chargement d'un programme par cassette, et pour revenir au BASIC après un passage par le moniteur (RESET).

E000RE2B3R

15.3 Mode direct et mode programme

Une ligne qui commence par un **numéro** (0 à 32767) est rangée dans le programme ; une ligne sans numéro est exécutée tout de suite.

```
WOZ MONITOR
>PRINT 2*3+4*5
26
>10 PRINT "BONJOUR"
>20 GOTO 10
>RUN
BONJOUR
BONJOUR
…
```

Appuyez sur une touche pour arrêter le programme : le BASIC affiche `STOPPED AT 10`. Retaper une ligne avec le même numéro la remplace ; un numéro seul l'efface.

> **! LE MODE DIRECT EST LIMITÉ**
> En mode direct, le BASIC refuse les instructions séparées par `:` (`SYNTAX ERR`), ainsi que `FOR` et `GOSUB`. Pour essayer une boucle, écrivez-la dans une ligne numérotée.

15.4 Corriger une ligne

Le clavier de l'Apple-1 n'a pas de touche d'effacement : le **retour arrière** de l'émulateur tape `_`, qui annule le caractère précédent (la ligne affichée n'est pas corrigée, mais le BASIC reçoit la bonne). Échap annule toute la ligne : le BASIC affiche `\` et attend la ligne suivante. Pour une correction plus confortable, utilisez l'éditeur (chapitre 7) : il colore la syntaxe, signale les erreurs avant l'envoi, et tape le programme pour vous.

15.5 Un premier programme

boucle.bas — INTEGER BASIC

```
10 S=0
20 FOR I=1 TO 10
30 S=S+I
40 NEXT I
50 PRINT "SOMME = ";S
60 END
```

WOZ MONITOR

```
>RUN
SOMME = 55
>
```

Dans l'éditeur, ▶ **Lancer** tape le programme et `RUN` ; dans le débogueur, vous pouvez le suivre ligne par ligne (figure du chapitre 8).

15.6 Les commandes

Commande	Effet
LIST `[n[,m]]`	liste le programme, la ligne `n`, ou les lignes `n` à `m`
RUN `[n]`	efface les variables et lance le programme, depuis la ligne `n` le cas échéant
DEL `n[,m]`	efface la ligne `n`, ou les lignes `n` à `m`
SCR	efface le programme et les variables
CLR	efface les variables, garde le programme
AUTO `n[,pas]`	numérotation automatique à partir de `n` (pas de 10 par défaut) ; Ctrl+D puis RETURN en sort, Échap n'en sort pas
MAN	revient à la numérotation manuelle
HIMEM `=n`, LOMEM `=n`	les bornes de la mémoire du BASIC (chapitre 16)

`GOTO n` en mode direct reprend un programme arrêté **sans** effacer les variables, à la différence de `RUN n`.

✦ LIST ET L'AFFICHAGE LENT

Le terminal de l'Apple-1 affiche environ 60 caractères par seconde : lister un long programme prend du temps. Désactivez la limite d'affichage (**Rapide** dans la rangée d'affichage, ou dans la barre d'état) pour lire plus vite, ou listez par tranches : `LIST 100,200`.

15.7 Trois programmes à taper

Devine un nombre

devine.bas — INTEGER BASIC

```
10 REM DEVINE UN NOMBRE DE 1 A 100
20 N=RND(100)+1:C=0
30 PRINT "J'AI CHOISI UN NOMBRE DE 1 A 100."
40 INPUT "TON ESSAI",E
50 C=C+1
60 IF E<N THEN PRINT "PLUS GRAND !"
70 IF E>N THEN PRINT "PLUS PETIT !"
80 IF E#N THEN 40
90 PRINT "BRAVO, EN ";C;" COUPS."
100 END
```

WOZ MONITOR

```
>RUN
J'AI CHOISI UN NOMBRE DE 1 A 100.
TON ESSAI?50
PLUS GRAND !
TON ESSAI?75
PLUS PETIT !
TON ESSAI?62
PLUS PETIT !
TON ESSAI?56
PLUS GRAND !
TON ESSAI?57
BRAVO, EN 5 COUPS.
```

`RND(100)` donne un entier de 0 à 99 ; `#` veut dire « différent de ».

Une table de multiplication

table.bas — INTEGER BASIC

```
10 DIM T(10)
20 INPUT "TABLE DE",N
30 FOR I=1 TO 10:T(I)=N*I:NEXT I
40 FOR I=1 TO 10
50 PRINT I;" X ";N;" = ";T(I)
60 NEXT I
70 S=0:FOR I=1 TO 10:S=S+T(I):NEXT I
80 PRINT "TOTAL : ";S
90 END
```

Jouer avec les chaînes

chaines.bas — INTEGER BASIC

```
10 DIM N$(20),R$(20)
20 INPUT "TON NOM",N$
30 L=LEN(N$)
40 PRINT "INITIALE : ";N$(1,1)
50 PRINT "FIN : ";N$(L-1)
60 R$="":FOR I=L TO 1 STEP -1
70 R$(LEN(R$)+1)=N$(I,I)
80 NEXT I
90 PRINT "A L'ENVERS : ";R$
100 IF R$=N$ THEN PRINT "C'EST UN PALINDROME !"
110 END
```

WOZ MONITOR

```
>RUN
TON NOM?KAYAK
INITIALE : K
FIN : AK
A L'ENVERS : KAYAK
C'EST UN PALINDROME !
```

Les chaînes se découpent avec `N$(début,fin)` et s'allongent avec `R$(LEN(R$)+1)=…` : il n'y a ni `MID$`, ni `+` pour les chaînes (chapitre 16).

Les quatre programmes sont dans `docs/exemples/basic/` : ouvrez-les dans l'éditeur, ou faites-en une cassette avec `apple1-emu basic devine.bas -o devine.aiff`.

CHAPITRE 16

Référence des instructions et des fonctions

Toutes les instructions reconnues par l'Integer BASIC de l'Apple-1, d'après la table de mots-clés de l'interpréteur, avec ce qu'elles font vraiment. Chaque exemple a été essayé.

16.1 Les instructions

`PRINT liste [;]`

PRINT affiche des nombres et des chaînes. `;` colle les éléments sans espace (`PRINT A;B` affiche `12` pour A=1, B=2) ; `,` passe à la colonne suivante, de 8 en 8 (`PRINT 1,2,3` affiche `1 2 3`). Un `;` final supprime le retour à la ligne. `PRINT` seul passe à la ligne.

`INPUT ["invite",] variable [,variable…]`

INPUT lit un nombre ou une chaîne. Le BASIC affiche toujours `?` après l'invite : `INPUT "NOM",N$` affiche `NOM?`. Une chaîne trop longue pour son `DIM` donne `STR OVFL ERR`.

`LET variable = expression`

LET est facultatif : `A=5` suffit.

`IF expression THEN instruction | numéro`

IF exécute l'instruction quand l'expression n'est pas nulle (`IF A THEN …` est permis). `THEN 100` saute à la ligne 100. **Attention** : seule la première instruction dépend de la condition (chapitre 18, bug 4).

`GOTO expression — GOSUB expression — RETURN`

GOTO et GOSUB acceptent une **expression** : `GOTO K*100`, `GOSUB 300+K` (le « GOTO calculé » remplace le `ON … GOTO` absent). Une ligne inexistante donne `BAD BRANCH ERR`. Huit GOSUB imbriqués au plus (`>8 GOSUBS ERR` au neuvième), `RETURN` sans `GOSUB` donne `BAD RETURN ERR`.

`FOR var = début TO fin [STEP pas] … NEXT var`

FOR/NEXT : le corps est exécuté **au moins une fois**, même si début dépasse fin (`FOR I=5 TO 1` passe une fois). `STEP` peut être négatif. Huit boucles imbriquées au plus (`>8 FORS ERR`). `NEXT` doit nommer la variable ; un `NEXT` sans `FOR` donne `BAD NEXT ERR`.

`DIM var(n) [,var$(n)…]`

DIM crée un tableau d'entiers de `n` éléments, numérotés **de 1 à n** (`A(0)` donne `RANGE ERR`), ou réserve `n` caractères (au plus 255) pour une chaîne. Toute chaîne doit être dimensionnée avant d'être affectée.

`REM texte`

REM : commentaire, jusqu'à la fin de la ligne (un `:` après REM fait partie du commentaire).

`END`

END termine le programme. Un programme qui se termine sans `END` affiche `*** END ERR` : ce n'est pas grave, mais terminez vos programmes par `END`.

`TAB n`

TAB est une **instruction** (et non une fonction de PRINT comme en Applesoft) : `TAB 10:PRINT "*"` affiche l'étoile en colonne 10.

`POKE adresse,valeur`

POKE écrit un octet (0 à 255, sinon `>255 ERR`). Les adresses au-delà de 32767 s'écrivent en négatif : adresse − 65536.

`CALL adresse`

CALL appelle un programme en langage machine, qui revient par `RTS`. Exemple : `POKE 768,96 : CALL 768` (96 = `RTS`). `CALL -256` revient au Woz Monitor proprement (`$FF00`).

16.2 Les fonctions

Fonction	Résultat	Exemple vérifié
PEEK `(adr)`	l'octet à l'adresse	`PEEK(76)+PEEK(77)*256` donne HIMEM (4096)
RND `(n)`	entier aléatoire de 0 à n−1 ; `RND(0)` : `>32767 ERR` ; `RND(-5)` donne un nombre négatif	`RND(6)+1` : un dé
SGN `(x)`	−1, 0 ou 1 (aussi pour un nombre de 256 ou plus)	`SGN(-5);SGN(0)` affiche `-10`
ABS `(x)`	valeur absolue	`ABS(-9)` = 9
`KEY(n)` (G2 Basic seulement)	code de la touche frappée, sans attendre (voir plus bas)	`K=KEY(0)`
LEN `(a$)`	longueur de la chaîne	`LEN("ABC")` = 3
USR `(x)`	appelle `$0000` : **plante** sans programme préparé	—
ASC `(` , SCRN `(`	présents dans la table, mais `SYNTAX ERR`	—

16.3 Les opérateurs

Priorité	Opérateurs
1	`-` et `+` unaires, `NOT`
2	`*` `/` `MOD`
3	`+` `-`
4	`=` `#` (différent, aussi `<>`) `<` `>` `<=` `>=`

Priorité	Opérateurs
5	`AND`
6	`OR`

MOD, AND, OR et NOT :

- La division **tronque vers zéro** : `7/2` = 3, `-7/2` = −3. `MOD` prend le signe du dividende : `-7 MOD 3` = −1.
- Les comparaisons donnent 1 (vrai) ou 0 (faux) : `PRINT 5>3;5=3` affiche `10`. `NOT 0` = 1, `NOT 5` = 0.
- `AND` et `OR` sont logiques, pas bit à bit : `3 AND 0` = 0, `3 OR 0` = 1.
- `3*-2` est accepté et vaut −6.
- `^` existe dans la table, mais **bloque la machine** (chapitre 18, bug 2).
- Les chaînes ne se comparent qu'avec `=` et `#` ; `"ABC"<"ABD"` est refusé (`SYNTAX ERR`).

16.4 Les extensions du RDK G2 Basic

Elles ne marchent qu'avec le **RDK G2 Basic** (l'Integer BASIC d'origine répond `SYNTAX ERR`).

`KEY(n)`

`KEY(0)` rend le **code de la touche** frappée (le code ASCII sur 7 bits : 65 pour `A` , 13 pour Retour, 32 pour la barre d'espace) et la consomme, ou `0` si aucune touche n'a été frappée. Elle n'attend jamais : un programme peut donc tourner tout en surveillant le clavier. `KEY(1)` (tout autre argument non nul) **attend** une touche.

Attendre une touche sans INPUT — INTEGER BASIC

```
10 K=KEY(0)
20 IF K=0 THEN 10
30 PRINT K
```

Dans l'original, une touche frappée pendant qu'un programme tourne **l'arrête** (`STOPPED AT n`), et `INPUT` bloque jusqu'à Retour. Avec le G2 Basic, dès qu'un programme a appelé `KEY` , une touche ne l'arrête plus : seule `KEY` voit le clavier, et Ctrl+C (code 3) lu par `KEY` arrête le programme au prochain appel. Appelez donc `KEY` dans la boucle principale du programme.

`LOMEM= n — HIMEM= n`

Des commandes immédiates qui prennent une expression (`LOMEM=2048+512`) ; dans un programme elles donnent `SYNTAX ERR` , comme dans l'original. Elles sont décrites avec la mémoire au chapitre 16.

`LIST [a[,b]] — DEL a[,b]`

`LIST` liste tout, `LIST n` une ligne, `LIST a,b` les lignes de `a` à `b` . `DEL n` supprime une ligne, `DEL a,b` les lignes de `a` à `b` (si `b` est inférieur à `a` , seule la ligne `a`). Une ligne listée plus longue que l'écran passe à la ligne comme dans l'original (les lignes de suite sont décalées de six espaces).

Les tranches de chaînes suivent l'original : `A$(I)` avec `I` de 256 ou plus, ou négatif, donne `>255 ERR` , et `A$(I,J)` avec `J` inférieur à `I-1` donne `STRING ERR` .

16.5 Les mots-clés sans effet

La table contient `COLOR=`, `PLOT`, `HLIN`, `VLIN` et `VTAB`, hérités du projet de BASIC pour l'Apple II. Sur l'Apple-1, `COLOR=` et `PLOT` sont acceptés sans rien faire, `VTAB` donne `SYNTAX ERR`. N'en mettez pas dans vos programmes.

CHAPITRE 17

Variables, chaînes, tableaux et mémoire

17.1 Les nombres

L'Integer BASIC ne connaît que des **entiers de 16 bits signés**, de −32767 à 32767. Un calcul qui sort de cet intervalle s'arrête sur `*** >32767 ERR` (`200*200`, `32767+1`, et aussi `10/0` et `RND(0)`). La valeur −32768 n'est pas utilisable (chapitre 18, bug 5). Il n'y a ni virgule, ni `INT` : `7/2` vaut 3.

17.2 Les noms de variables

C'est la limite qui surprend le plus : un nom de variable est **une lettre, éventuellement suivie d'un chiffre**.

Nom	Résultat (essayé)
`A`, `Z`, `B1`, `Z9`	accepté
`AB`, `CPT`, `SCORE`, `COMPTEUR`	`SYNTAX ERR`
`A12`	`SYNTAX ERR`
`A$`, `N$`	chaîne : **une lettre** seulement
`B9$`, `Z9$`	`SYNTAX ERR`

Il y a donc 26 × 11 = 286 variables entières possibles, et 26 chaînes. Prenez l'habitude de noter dans un `REM` le rôle de chaque variable.

17.3 Les chaînes

Une chaîne doit être **dimensionnée** avant usage, avec sa longueur maximale (1 à 255) : `DIM N$(20)`. `DIM B$(256)` donne `>255 ERR`.

Opération	Écriture	Exemple (N$ = « KAYAK »)
longueur	`LEN(N$)`	5
sous-chaîne	`N$(i,j)` : caractères i à j	`N$(1,3)` = « KAY »
fin	`N$(i)` : du caractère i à la fin	`N$(4)` = « AK »
concaténer	`A$(LEN(A$)+1)=B$`	« APPLE » + « -1 » → « APPLE-1 »
comparer	`=` et `#` seulement	`IF N$="WOZ" THEN …`

Affecter une chaîne plus longue que son `DIM` donne `*** STR OVFL ERR`, de même qu'une réponse trop longue à `INPUT`. Il n'y a ni `MID$`, ni `LEFT$`, ni `CHR$`, ni `STR$`, ni `VAL`, ni `ASC` (ce dernier est dans la table mais ne marche pas).

> ✦ **REMPLACER CHR$ ET VAL**
>
> Pour afficher un caractère par son code, gardez une chaîne de référence et découpez-la : `DIM C$(26):C$="ABCDEFGHIJKLMNOPQRSTUVWXYZ"` puis `PRINT C$(K,K)`. Pour lire un nombre, utilisez `INPUT N` plutôt que `INPUT N$`.

17.4 Les tableaux

`DIM T(10)` crée un tableau de 10 entiers, **numérotés de 1 à 10**. `T(0)` et `T(11)` donnent `*** RANGE ERR`. Il n'y a qu'une dimension : pour une grille de 8 × 8, utilisez `G((L-1)*8+C)`. Dans l'émulateur, les éléments valent 0 après le `DIM` ; sur une vraie machine, la mémoire n'est pas forcément effacée : initialisez vos tableaux. Un tableau trop grand pour la mémoire donne `*** MEM FULL ERR` (`DIM A(3000)` en 8 Ko).

17.5 La mémoire du BASIC

Adresses	Contenu	Remarques
FF00–FFFF	**Woz Monitor** (PROM 256 octets)	vecteurs NMI, RESET, IRQ
E000–EFFF	**Integer BASIC** (4 Ko de RAM)	chargé par cassette
D010–D013	**PIA 6821**: clavier, écran	KBD, KBDCR, DSP, DSPCR
C250–C257	**Carte GEN2**: commutateurs	option, en lecture
C100–C1FF	**PROM de l'ACI**(C100R)	C000–C0FF : bascule
1000–BFFF	RAM d'extension (jusqu'à 48 Ko)	HIRES 2000–5FFF (GEN2)
0800–0FFF	programmes BASIC (LOMEM–HIMEM)	texte/LORES 2 : 0800
0300–07FF	vos programmes en langage machine	texte/LORES 1 : 0400
0200–027F	tampon de saisie du moniteur	ne rien y mettre
0100–01FF	pile du 6502	
0000–00FF	page zéro	moniteur 24–2B, BASIC 4A–FF

Figure 32 — La mémoire de l'Apple-1 : BASIC en $E000, programme sous HIMEM, variables au-dessus de LOMEM.

Le BASIC range le **programme en haut** de la mémoire, en descendant depuis HIMEM, et les **variables en bas**, en montant depuis LOMEM. L'espace libre est entre les deux.

Pointeur	Adresse	Valeur en 8 Ko	Rôle
LOMEM	`$4A-$4B` (74-75)	`$0800` (2048)	début des variables

Pointeur	Adresse	Valeur en 8 Ko	Rôle
HIMEM	`$4C-$4D` (76-77)	`$1000` (4096)	fin de la mémoire du BASIC
PP	`$CA-$CB` (202-203)	baisse avec le programme	début du programme
PV	`$CC-$CD` (204-205)	monte avec les variables	fin des variables

`PRINT PEEK(202)+PEEK(203)*256` donne le début du programme et `PRINT PEEK(204)+PEEK(205)*256` la fin des variables : la place libre est la différence des deux. Une variable entière comme `A` occupe 6 octets.

▣ **POURQUOI UNE CASSETTE BASIC COMMENCE EN $004A**

Ces pointeurs sont en page zéro, de `$4A` à `$FF`. Une cassette BASIC enregistre donc **deux plages** : `004A.00FF` (les pointeurs), puis le programme de PP à HIMEM, par exemple `0F26.0FFF`. La ligne de commande `apple1-emu basic` les calcule pour vous :

```
10 lignes, 218 octets ; LOMEM $0800 HIMEM $1000 PP $0F26 ; 1830 octets libres pour
les variables ; RAM 8 Ko
Plages ACI (après C100R) : 004A.00FFR 0F26.0FFFR   puis E2B3R et RUN
```

Changer LOMEM et HIMEM

La syntaxe est `LOMEM=n` et `HIMEM=n` (avec `=`, **pas** `:` comme sur l'Apple II, qui donne `SYNTAX ERR`).

- `LOMEM=2304` déplace le début des variables et **efface les variables** (le programme reste). Utile pour réserver `$0800-$08FF` à un programme en langage machine.
- `HIMEM=` **plus haut** (avec plus de RAM, par exemple `HIMEM=8192` en 16 Ko) fonctionne et garde le programme.
- `HIMEM=` **plus bas** est **ignoré sans message** par l'original, même après `SCR` (chapitre 18, bug 11). **Le RDK G2 Basic l'accepte** : le programme en mémoire descend avec la limite, ou `MEM FULL ERR` s'il ne tenait plus au-dessus des variables (`RANGE ERR` pour une valeur négative ; `LOMEM=` donne `RANGE ERR` s'il n'est pas sous le début du programme, et `>32767 ERR` au-dessus de 32767). Avec l'original, il existe un contournement vérifié : écrire le pointeur directement, puis `SCR` :

Abaisser HIMEM à 3072 ($0C00) — INTEGER BASIC

```
POKE 76,0 : POKE 77,12 : SCR
```

Au démarrage du G2 Basic, HIMEM est le haut de la RAM trouvée et LOMEM vaut `$0800` (`$2000` avec un pilote GEN2). (`76` / `77` = `$4C` / `$4D`, 12 × 256 = 3072.) Tapez ces trois instructions sur une ligne numérotée ou séparément en mode direct, **avant** d'entrer le programme : `SCR` l'efface.

17.6 Mélanger BASIC et langage machine

Zone (8 Ko)	Utilisable pour du 6502 ?
`$0000-$00FF`	page zéro : le Woz Monitor utilise `$24-$2B`, le BASIC au moins `$4A-$FF` ; à éviter, ou à sauver et restaurer
`$0100-$01FF`	pile du 6502 : non
`$0200-$027F`	tampon de ligne du Woz Monitor ; écrasé à chaque ligne tapée
`$0280-$07FF`	libre tant que le BASIC n'y touche pas : le meilleur endroit
`$0800-…`	variables du BASIC (sauf si LOMEM est relevé)
`… -$0FFF`	programme BASIC

Le programme machine se charge par cassette ou par POKE, et s'appelle par `CALL 768` (pour `$0300`). Il doit finir par `RTS` et préserver la page zéro du BASIC. Exemple minimal :

WOZ MONITOR

```
>POKE 768,96
>CALL 768
>
```

(96 = `$60` = `RTS` : l'appel revient tout de suite.)

CHAPITRE 18

Messages d'erreur et limites

18.1 Les messages

Une erreur affiche `*** MESSAGE ERR`, suivi en cours de programme de `STOPPED AT n`. Une ligne refusée à la saisie n'est pas rangée : retapez-la.

Message	Cause	Exemple essayé
SYNTAX ERR	instruction inconnue, nom de variable invalide, `:` en mode direct	`CPT=7`, `GET A$`, `X=5:PRINT X` en direct
>32767 ERR	dépassement, division par zéro, numéro de ligne > 32767	`200*200`, `10/0`, `RND(0)`, `32768 PRINT`
TOO LONG ERR	ligne trop complexe (expression trop imbriquée) ou trop longue	13 parenthèses imbriquées
MEM FULL ERR	plus de place entre variables et programme	`DIM A(3000)` en 8 Ko
TOO MANY PARENS ERR	parenthèses (prévu par l'interpréteur ; masqué par le bug 3)	—
STRING ERR	chaîne mal employée	—
END ERR	le programme se termine sans `END`	`10 PRINT "X"` puis `RUN`
BAD BRANCH ERR	`GOTO` / `GOSUB` vers une ligne inexistante	`GOTO 500`
>8 GOSUBS ERR	plus de 8 `GOSUB` imbriqués	récursion : 9 appels
BAD RETURN ERR	`RETURN` sans `GOSUB`	`RUN 30` sur un `RETURN`
>8 FORS ERR	plus de 8 `FOR` imbriqués, ou bug 1	9 boucles sur une ligne
BAD NEXT ERR	`NEXT` sans `FOR` correspondant	`20 NEXT I` seul
STOPPED AT	une touche a été frappée pendant l'exécution	—
>255 ERR	valeur > 255 : `POKE`, `DIM` d'une chaîne	`POKE 768,300`, `DIM B$(256)`
RANGE ERR	indice de tableau hors limites	`A(0)`, `A(3)` après `DIM A(2)`
DIM ERR	tableau ou chaîne redimensionné(e)	—
STR OVFL ERR	chaîne plus longue que son `DIM`	`A$="ABCDE"` avec `DIM A$(3)`
RETYPE LINE	réponse à `INPUT` invalide (lettres pour un nombre) : retapez	—

Les lignes « — » viennent de la table des messages de l'interpréteur ; elles n'ont pas été provoquées dans nos essais.

18.2 Les limites

Limite	Valeur
entiers	−32767 à 32767
numéros de ligne	0 à 32767
noms de variables	lettre + chiffre facultatif
noms de chaînes	une lettre + `$`
longueur d'une chaîne	255
dimensions d'un tableau	1, indices de 1 à n
`FOR` imbriqués	8
`GOSUB` imbriqués	8
parenthèses imbriquées	12
longueur d'une ligne tapée	120 caractères passent, 141 sont refusés

18.3 Gagner de la mémoire

En 8 Ko, le BASIC dispose d'environ 2 Ko pour le programme et les variables. Quelques moyens d'en gagner :

1. **Abaissez LOMEM** si vous n'avez pas de langage machine : `LOMEM=768` (`$0300`) donne 1280 octets de plus aux variables. Essayé : `DIM T(1200)` échoue en `MEM FULL` avec LOMEM par défaut, et passe après `LOMEM=768`. Les portages des *101 BASIC Computer Games* faits pendant le développement procèdent ainsi.
2. **Écrivez court** : le BASIC garde les espaces et les `REM`. `LIST` réindente tout seul, et une ligne longue regroupant plusieurs instructions coûte moins que plusieurs lignes.
3. **Réutilisez les tableaux et les chaînes** plutôt que d'en dimensionner de nouveaux.
4. **Ajoutez de la RAM** : la fiche ⚙ d'un programme, `--ram 16`, ou une vraie extension. Un programme de plus de 8 Ko exige une machine de 12, 16 ou 32 Ko : `apple1-emu basic --ram 16` le place en haut de cette mémoire.

CHAPITRE 19

Les bugs connus et leurs contournements

L'Integer BASIC de l'Apple-1 a été écrit sans assembleur, en 1976, et n'a jamais été corrigé. Voici ses défauts connus, chacun reproduit dans l'émulateur avec le vrai BASIC, et comment les éviter.

VÉRIFIÉ : reproduit dans l'émulateur avec l'Integer BASIC réel, transcription à l'appui. NON VÉRIFIÉ : rapporté par une source, pas reproduit. Le bug n correspond à la section 18.n. Comme l'émulateur exécute le code d'origine instruction par instruction, un bug reproduit ici existe aussi sur un vrai Apple-1.

19.1 Sortir d'une boucle FOR par GOTO VÉRIFIÉ

Une boucle quittée par `GOTO` reste « ouverte ». Si le programme repasse par le `FOR`, une nouvelle entrée s'empile ; au neuvième passage : `>8 FORS ERR`.

INTEGER BASIC

```
10 N=0
20 FOR I=1 TO 10
30 IF I=2 THEN GOTO 50
40 NEXT I
50 N=N+1:IF N<20 THEN 20
60 PRINT "FINI ";N
70 END
```

WOZ MONITOR

```
>RUN
*** >8 FORS ERR
STOPPED AT 20
```

✦ CONTOURNEMENT

Ne sortez jamais d'un `FOR` par `GOTO` : donnez à la variable sa valeur finale et passez par `NEXT`. Ici : `30 IF I=2 THEN I=10:GOTO 40`. Le programme affiche alors `FINI 20`. Les portages des 101 BASIC *Computer Games* suivent cette règle.

19.2 L'opérateur ^ bloque la machine VÉRIFIÉ

`^` (puissance) est dans la table de l'interpréteur, mais `PRINT 2^3` ne rend jamais la main : seul RESET en sort.

✦ CONTOURNEMENT

Multipliez : `2*2*2` , ou une boucle `P=1:FOR I=1 TO N:P=P*X:NEXT I` .

19.3 Le message « PPED AT » VÉRIFIÉ

Quand la pile des opérandes de l'interpréteur déborde (une expression trop imbriquée en mode direct), le BASIC devrait afficher un message d'erreur, mais il pointe **3 octets trop loin** dans la table des messages (le désassemblage d'Eric Smith le note : `ldy #erri_stopped_at+3 ; The "+3" is a bug!`). On lit alors la fin de `STOPPED AT` :

WOZ MONITOR

```
>PRINT 1+(1+(1+(1+(1+(1+(1+(1+(1+(1+(1+(1+(1))))))))))))
14
*** PPED AT  ERR
STOPPED AT 65535
```

✦ CONTOURNEMENT

Découpez les expressions longues avec des variables intermédiaires. Au-delà de 12 niveaux de parenthèses, une ligne de programme est de toute façon refusée (`TOO LONG ERR`).

19.4 IF … THEN ne conditionne que la première instruction VÉRIFIÉ

Dans `IF A=2 THEN PRINT "X":PRINT "Y"` , seul `PRINT "X"` dépend de la condition : `PRINT "Y"` est **toujours** exécuté (avec A=1, le programme affiche `Y`). C'est le piège le plus fréquent quand on porte un programme écrit pour un autre BASIC.

✦ CONTOURNEMENT

Inversez la condition et sautez par-dessus le bloc :

INTEGER BASIC

```
80 IF A#2 THEN 90
85 PRINT "X":PRINT "Y"
90 REM SUITE
```

19.5 −32768 VÉRIFIÉ

`X=-32767-1` affiche un `-` puis `*** >32767 ERR` . La plus petite valeur utilisable est −32767.

✦ CONTOURNEMENT

Restez dans −32767 … 32767. Pour lire un mot de 16 bits par `PEEK` , découpez-le : `PEEK(A)+PEEK(A+1)*256` déborde au-delà de 32767 ; testez d'abord l'octet haut (`IF PEEK(A+1)>127 THEN …`).

19.6 ASC(ne fonctionne pas VÉRIFIÉ

`ASC("A")` est dans la table des mots-clés, mais toute ligne qui l'emploie est refusée : `SYNTAX ERR`. Même chose pour `SCRN(`.

✦ CONTOURNEMENT (ESSAYÉ)

Cherchez le caractère dans une chaîne de référence. La ligne 60 referme la boucle (`K=26:NEXT K`) pour éviter le bug 1 :

Le rang d'une lettre, sans ASC — INTEGER BASIC

```
10 DIM C$(26),L$(1)
20 C$="ABCDEFGHIJKLMNOPQRSTUVWXYZ"
30 INPUT "LETTRE",L$
40 FOR K=1 TO 26:IF C$(K,K)=L$ THEN 60:NEXT K
50 PRINT "PAS UNE LETTRE":END
60 PRINT "RANG ";K:K=26:NEXT K
70 GOTO 30
```

WOZ MONITOR

```
LETTRE?W
RANG 23
LETTRE?E
RANG 5
LETTRE?5
PAS UNE LETTRE
```

19.7 Les mots-clés graphiques sans effet VÉRIFIÉ

`COLOR=`, `PLOT` et `HLIN` sont acceptés et ne font rien ; `VLIN` et `VTAB` donnent `SYNTAX ERR`. Ce sont des restes du BASIC destiné à l'Apple II.

✦ CONTOURNEMENT

Pour positionner le texte, utilisez `TAB n` et des `PRINT` vides. Pour dessiner en couleur, la carte GEN2 se pilote par `POKE` (chapitre 6).

19.8 Le bug des 251 caractères de l'Apple II ABSENT

Une version plus tardive de l'Integer BASIC (Apple II) plante sur certaines lignes de 251 caractères et plus. Sur l'Apple-1, la ligne tapée est limitée bien avant (environ 128 caractères) : le bug ne peut pas se produire.

19.9 STOPPED AT 65535 VÉRIFIÉ

Une erreur provoquée par une ligne **refusée pendant sa saisie** (trop imbriquée) ou par le bug 3 en mode direct est suivie de `STOPPED AT 65535` : le BASIC croit être dans un programme et affiche un numéro de ligne absurde. C'est sans conséquence ; le programme en mémoire n'est pas touché.

19.10 USR plante la machine VÉRIFIÉ

`X=USR(1)` saute en `$0000`, où il n'y a normalement pas de code : la machine se bloque. RESET puis `E2B3R` reviennent au BASIC sans perdre le programme.

> ✦ CONTOURNEMENT
>
> Appelez vos programmes machine avec `CALL adresse` ; passez les valeurs par `POKE` et `PEEK`.

19.11 HIMEM= ne peut pas baisser VÉRIFIÉ

`HIMEM=3072` (et toute valeur inférieure à l'actuelle) est **ignoré sans message**, même après `SCR`, parce que l'interpréteur compare la nouvelle valeur au pointeur de programme avant de l'accepter.

> ✦ CONTOURNEMENT (VÉRIFIÉ)
>
> `POKE 76,0 : POKE 77,12 : SCR` met HIMEM à 3072 ; le programme suivant est rangé en dessous. Voir le chapitre 16.

19.12 Le deux-points refusé en mode direct VÉRIFIÉ

`X=5:Y=X*2:PRINT Y` tapé sans numéro donne `SYNTAX ERR` ; la même ligne numérotée fonctionne. `FOR` et `GOSUB` sont aussi refusés en mode direct.

> ✦ CONTOURNEMENT
>
> Tapez une instruction par ligne, ou écrivez une ligne `0` temporaire et `RUN 0` … sans oublier de l'effacer (`DEL 0`).

19.13 INPUT ajoute toujours « ? » VÉRIFIÉ

`INPUT "AGE ? ",N` affiche `AGE ? ?` : le BASIC ajoute son propre point d'interrogation. Écrivez l'invite sans `?` : `INPUT "AGE",N` affiche `AGE?`.

19.14 La boucle FOR passe au moins une fois VÉRIFIÉ

`FOR I=5 TO 1 … NEXT I` exécute le corps une fois (avec I=5), là où d'autres BASIC le sautent.

> ✦ CONTOURNEMENT
>
> Testez avant : `IF N<1 THEN 100` avant `FOR I=1 TO N`.

19.15 Une touche arrête le programme VÉRIFIÉ

N'importe quelle touche frappée pendant l'exécution l'arrête (`STOPPED AT`), sauf pendant un `INPUT`. Ce n'est pas un bug mais un choix : il n'y a pas de touche BREAK. Conséquence : un jeu en temps réel ne peut pas attendre une touche sans s'arrêter. Lire le clavier par `PEEK(-12271)` (`$D011`) et `PEEK(-12272)` (`$D010`) semble possible, mais l'interpréteur teste lui aussi le clavier entre deux instructions et risque de s'arrêter avant : nous ne l'avons pas essayé (NON VÉRIFIÉ). Pour un jeu en temps réel, passez au langage machine (partie III).

19.16 Résumé

#	Bug	Statut	Contournement
1	sortie de FOR par GOTO → `>8 FORS`	vérifié	finir la boucle par NEXT
2	`^` bloque	vérifié	multiplier
3	« PPED AT » (+3)	vérifié	variables intermédiaires
4	`IF…THEN a:b` exécute b	vérifié	condition inverse + saut
5	−32768	vérifié	rester ≥ −32767
6	`ASC(`, `SCRN(`	vérifié	chaîne de référence
7	COLOR=, PLOT, HLIN sans effet ; VLIN, VTAB refusés	vérifié	TAB, POKE (GEN2)
8	251 caractères (Apple II)	absent	—
9	STOPPED AT 65535	vérifié	sans conséquence
10	USR plante	vérifié	CALL
11	HIMEM= ne baisse pas	vérifié	POKE 76/77 + SCR
12	`:` refusé en direct	vérifié	une instruction par ligne
13	INPUT ajoute « ? »	vérifié	invite sans « ? »
14	FOR passe une fois	vérifié	tester avant
15	lecture du clavier par PEEK pendant RUN	non vérifié	langage machine

CHAPITRE 20

Porter un programme Applesoft ou Microsoft BASIC

La plupart des listings des années 1970-1980 sont écrits en Microsoft BASIC (Applesoft, BASIC des « 101 BASIC Computer Games »). Ils se portent sur l'Apple-1, à condition de connaître les équivalences. Chaque ligne « refusé » ci-dessous a été tapée dans le vrai BASIC.

20.1 Les équivalences

Applesoft / Microsoft	Integer BASIC de l'Apple-1	Note
`ON X GOTO 100,200,300`	`GOTO 100*X` (si les lignes s'y prêtent) ou une suite de `IF X=1 THEN 100`	refusé
`GET A$`	`INPUT A$`	refusé
`DATA` / `READ` / `RESTORE`	tableau rempli par des affectations, ou chaîne découpée	refusés
`MID$(A$,I,N)`	`A$(I,I+N-1)`	refusé
`LEFT$(A$,N)` / `RIGHT$(A$,N)`	`A$(1,N)` / `A$(LEN(A$)-N+1)`	—
`A$+B$`	`A$(LEN(A$)+1)=B$`	—
`CHR$(N)`, `ASC(A$)`	chaîne de référence (chapitre 18, bug 6)	refusés
`STR$(N)`, `VAL(A$)`	`PRINT N` direct ; `INPUT N`	refusés
`INT(X)`	inutile : tout est entier	refusé
`RND(1)` (réel de 0 à 1)	`RND(N)` (entier de 0 à N−1)	—
`X^2`	`X*X`	bloque (bug 2)
`HOME`	24 × `PRINT`, ou CLEAR SCREEN (Ctrl+L) à la main	refusé
`PRINT TAB(10);"X"`	`TAB 10:PRINT "X"`	—
`STOP`	`END`	refusé
`DEF FN`	`GOSUB`	—
`A<>B`	`A#B` (`<>` est aussi accepté)	—
`IF A THEN X:Y`	condition inverse + saut (bug 4)	piège
`DIM A(10,10)`	`DIM A(121)` et `A(I*11+J+1)`	—
`A(0)`	indices à partir de 1	`RANGE ERR`

Applesoft / Microsoft	Integer BASIC de l'Apple-1	Note
`SCORE`, `NAME$`	`S`, `N$` : une lettre (+ chiffre)	`SYNTAX ERR`
nombres réels	entiers ×10 ou ×100 (virgule fixe)	—

20.2 La méthode

1. **Listez les variables** du programme d'origine et attribuez à chacune une lettre (+ chiffre). Notez la table dans des `REM` en tête.
2. **Remplacez les DATA** par des affectations dans un sous-programme d'initialisation, ou par une chaîne qu'on découpe.
3. **Cherchez chaque `IF … THEN … : …`** et réécrivez-le avec un saut (bug 4).
4. **Cherchez chaque sortie de `FOR` par `GOTO`** et remplacez-la par « valeur finale + `NEXT` » (bug 1).
5. **Décalez les indices de tableau** de 1 si le programme utilise l'indice 0.
6. **Vérifiez dans l'éditeur** : il signale à la frappe les mots inconnus de l'Integer BASIC. ▶ **Lancer** tape le programme : toute ligne refusée apparaît à l'écran avec `SYNTAX ERR`.
7. **Mesurez la mémoire** : `apple1-emu basic prog.bas` indique la taille et la RAM nécessaire. Trop gros pour 8 Ko ? `LOMEM=768` (chapitre 17), ou une machine de 16 Ko.
8. **Exportez** la cassette (📼 **Exporter WAV**) et notez les plages pour la vraie machine.

20.3 Un exemple de portage

Un fragment typique en Microsoft BASIC :

> Original (Microsoft BASIC)

```
100 READ N$,S
110 IF S>HI THEN HI=S:BEST$=N$
120 IF N$<>"FIN" THEN 100
130 PRINT "MEILLEUR : ";BEST$;" (";HI;")"
140 DATA "ANNE",12,"BOB",17,"CLAIRE",9,"FIN",0
```

Sur l'Apple-1 : pas de `READ` / `DATA`, pas de noms longs, et le `IF` de la ligne 110 affecterait `BEST$` à chaque passage (bug 4).

Port en Integer BASIC (essayé)

INTEGER BASIC

```
10 DIM N$(10),B$(10),S(3)
20 S(1)=12:S(2)=17:S(3)=9
30 H=0
40 FOR I=1 TO 3
50 GOSUB 200
60 IF S(I)<=H THEN 80
70 H=S(I):B$=N$
80 NEXT I
90 PRINT "MEILLEUR : ";B$;" (";H;")"
100 END
200 IF I=1 THEN N$="ANNE"
210 IF I=2 THEN N$="BOB"
220 IF I=3 THEN N$="CLAIRE"
230 RETURN
```

WOZ MONITOR

```
>RUN
MEILLEUR : BOB (17)
```

20.4 L'expérience des portages

Huit jeux des 101 BASIC *Computer Games* (DEC, 1975) ont été portés de cette façon pendant le développement de l'émulateur : BAGLES, MUGWMP, HEX, BULCOW, STARS, TRAP, CHOMP et WAR. Chacun a été joué dans l'émulateur, exporté en cassette et rechargé par la ROM de l'ACI. Ces portages ne sont pas distribués (les listings d'origine ne sont pas libres), mais ils ont fixé les règles de ce chapitre : variables d'une lettre, tableaux à partir de 1, pas de `READ` / `DATA`, pas de `GOTO` hors d'un `FOR`, une seule instruction conditionnée par `IF`, et `LOMEM=768` pour les plus gros.

PARTIE III

L'assembleur 6502

Le processeur, l'assembleur intégré et ses outils, les entrées-sorties et les routines du Woz Monitor, des exemples commentés et des astuces

CHAPITRE 21

Le 6502 en bref

Le 6502 de l'Apple-1 est un processeur 8 bits à 1,023 MHz, avec trois registres, 56 instructions et 64 Ko d'adresses. Ce chapitre en donne l'essentiel pour lire et écrire les exemples de cette partie.

21.1 Les registres

Registre	Taille	Rôle
A	8 bits	l'accumulateur : calculs, comparaisons, lectures et écritures
X, **Y**	8 bits	les index : compteurs de boucle, décalage dans un tableau
S	8 bits	le pointeur de pile ; la pile occupe la page `$0100-$01FF`
PC	16 bits	le compteur ordinal : l'adresse de l'instruction suivante
P	8 bits	les drapeaux N V – B D I Z C

Les drapeaux sont mis à jour par presque toutes les instructions :

`N (négatif)`
: copie du bit 7 du résultat ; testé par `BMI` / `BPL`.

`V (débordement)`
: débordement signé d'une addition ou d'une soustraction ; `BVS` / `BVC`.

`Z (zéro)`
: le résultat est nul ; `BEQ` / `BNE`.

`C (retenue)`
: retenue d'une addition, absence d'emprunt d'une soustraction, bit sorti d'un décalage ; `BCS` / `BCC`.

`D (décimal)`
: `ADC` et `SBC` calculent en BCD. Le Woz Monitor fait `CLD` au démarrage.

`I (interruptions)`
: masque l'IRQ. L'Apple-1 n'en a pas en standard.

21.2 Les modes d'adressage

Mode	Écriture	Octets	Exemple	Effet
implicite	`INX`	1	`inx`	X ← X + 1

Mode	Écriture	Octets	Exemple	Effet
accumulateur	`ASL A`	1	`asl a`	décale A
immédiat	`#val`	2	`lda #$8D`	A ← $8D
page zéro	`zp`	2	`lda $06`	A ← mémoire $0006
page zéro, X	`zp,X`	2	`lda $06,x`	
absolu	`adr`	3	`lda $D010`	
absolu, X / Y	`adr,X`	3	`lda message,x`	tableau indexé
indirect indexé	`(zp),Y`	2	`lda ($06),y`	pointeur 16 bits en page zéro, plus Y
indexé indirect	`(zp,X)`	2	`lda ($06,x)`	table de pointeurs
relatif	`étiquette`	2	`bne boucle`	branchement de −128 à +127 octets
indirect	`(adr)`	3	`jmp ($FFFC)`	`JMP` seulement

La **page zéro** (`$0000-$00FF`) est la plus précieuse : ses instructions sont plus courtes et plus rapides, et seul ce mode permet les pointeurs `(zp),Y` . Le Woz Monitor n'utilise que `$24-$2B` ; l'Integer BASIC en utilise beaucoup plus (chapitre 16).

21.3 Les instructions par famille

Famille	Instructions
chargement, rangement	`LDA LDX LDY STA STX STY`
transferts	`TAX TAY TXA TYA TSX TXS`
pile	`PHA PLA PHP PLP`
arithmétique	`ADC SBC` (avec la retenue : `CLC` avant `ADC` , `SEC` avant `SBC`)
incréments	`INC INX INY DEC DEX DEY`
logique	`AND ORA EOR BIT`
décalages	`ASL LSR ROL ROR`
comparaisons	`CMP CPX CPY` (C = 1 si registre ≥ valeur, Z = 1 si égal)
branchements	`BEQ BNE BCC BCS BMI BPL BVC BVS`
sauts	`JMP JSR RTS RTI BRK`
drapeaux	`CLC SEC CLD SED CLI SEI CLV`
rien	`NOP`

> ✦ **COMPARER SANS SIGNE**
>
> Après `CMP #n` , `BCC` saute si A < n et `BCS` si A ≥ n. Pour « A > n », comparez avec n + 1 : c'est ce que font les exemples avec `cmp #'9' | $80 + 1` .

21.4 Un 6502 NMOS, pas un 65C02

L'Apple-1 a un 6502 d'origine (NMOS), et l'émulateur aussi. Les opcodes non documentés s'exécutent comme des `NOP` de la bonne longueur, et les opcodes `JAM` arrêtent le processeur. L'assembleur accepte les instructions du 65C02 après `.setcpu "65C02"` (`BRA`, `STZ`, `PHX`, `PLX`, `PHY`, `PLY`, `INC A`…), pour une carte équipée d'un 65C02, mais elles **ne fonctionnent pas** dans l'émulateur :

c02.s (essai) 6502

```
        .setcpu "65C02"
        .org $0300
        lda #$C1        ; "A"
        bra suite       ; sur un 6502 NMOS : NOP de 2 octets
        lda #$D8        ; "X"
suite:  jsr $FFEF
        jmp $FF1F
```

Ce programme affiche `A` sur un 65C02, et `X` dans l'émulateur : `BRA` ($80) y est un `NOP` qui saute son opérande, et le `LDA #$D8` est exécuté.

21.5 La carte mémoire en une figure

Adresses	Contenu
`$0000-$00FF`	page zéro (Woz Monitor : `$24-$2B` ; BASIC : beaucoup plus)
`$0100-$01FF`	pile
`$0200-$027F`	tampon de ligne du Woz Monitor
`$0280-$1FFF`	libre : vos programmes, souvent en `$0280` ou `$0300` (RAM de 8 Ko par défaut dans l'émulateur ; 4 Ko, jusqu'à `$0FFF`, sur un Apple-1 de base)
`$C000-$C0FF`	carte ACI (cassette) ; `$C100-$C1FF` : son programme
`$C250-$C257`	commutateurs de la carte GEN2
`$D010-$D013`	PIA : clavier et écran
`$E000-$EFFF`	Integer BASIC (si chargé)
`$FF00-$FFFF`	Woz Monitor

L'annexe A donne la carte complète.

CHAPITRE 22

L'assembleur du devkit

L'émulateur contient un assembleur 6502 complet, à deux passes, compatible avec ca65. Il sert dans l'éditeur, qui assemble à chaque frappe, dans la ligne de commande, et pour les projets GitHub. Vous n'avez rien à installer.

22.1 Le cycle de travail

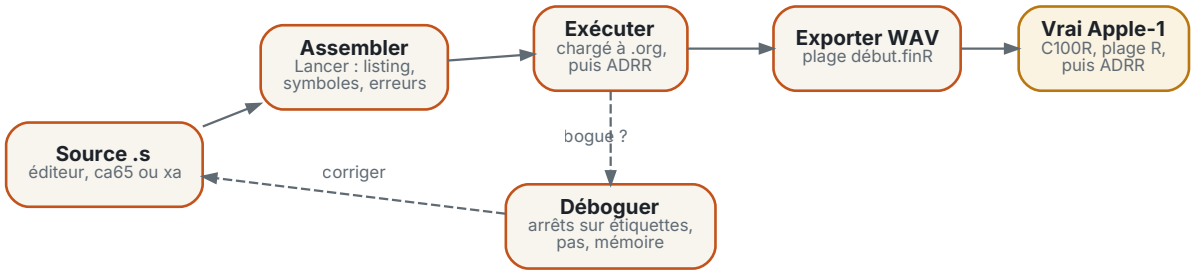


Figure 33 — Le cycle de travail en assembleur : écrire, assembler, lancer, déboguer, puis exporter vers le vrai Apple-1.

1. **Écrire** : 📝 **Éditeur**, puis **Nouveau** ou **Ouvrir** un fichier `.s`, `.asm` ou `.a65`. La coloration distingue instructions, directives, étiquettes, nombres et commentaires.
2. **Assembler** : rien à faire, l'assemblage suit chaque modification. Le bas du panneau donne la taille, les adresses et la ligne à taper sur la vraie machine ; les erreurs s'affichent avec leur ligne et leur colonne.
3. **Lancer** : ▶ **Lancer** met les octets en mémoire à l'adresse du `.org` et tape la commande du Woz Monitor (`300R`).
4. **Déboguer** : ● **Point d'arrêt** sur la ligne du curseur, puis pas à pas dans le débogueur (chapitre 8), avec vos étiquettes dans le désassemblage.
5. **Exporter** : 📼 **Exporter WAV** écrit la cassette ; sur le vrai Apple-1, tapez `C100R` , la plage affichée (`0300.0323R`), puis `300R` .

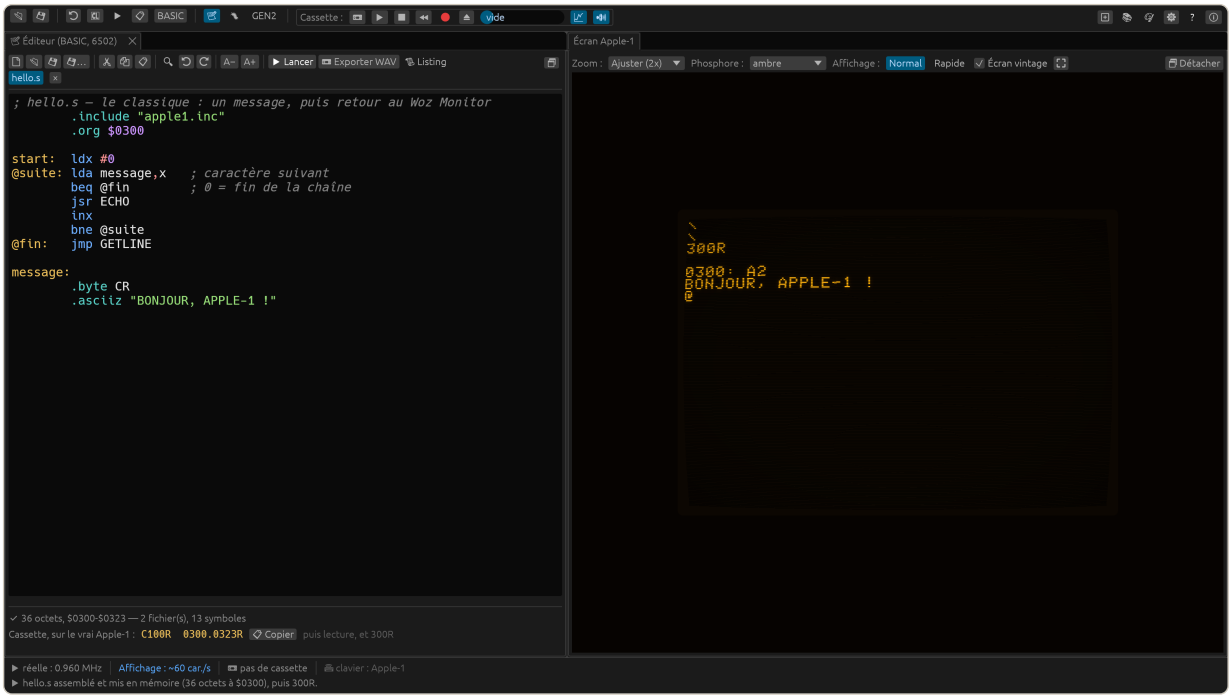


Figure 34 — Une source dans l'éditeur : à gauche le texte coloré, en bas le résultat de l'assemblage ; à droite l'Apple-1 a exécuté le programme.

22.2 Le listing et les symboles

📃 **Listing** affiche, pour chaque ligne, l'adresse, les octets produits et la source ; les lignes produites par une macro sont marquées `+`, les fichiers inclus sont développés, et la table des symboles suit. Un clic sur une ligne ramène à la source ; un clic dans la marge pose un point d'arrêt. Voici le listing de `hello.s` (chapitre 23), produit par la ligne de commande :

hello.lst

```
0300  A2 00       start:  ldx #0
0302  BD 10 03    @suite: lda message,x   ; caractère suivant
0305  F0 06               beq @fin        ; 0 = fin de la chaîne
0307  20 EF FF            jsr ECHO
030A  E8                  inx
030B  D0 F5               bne @suite
030D  4C 1F FF    @fin:   jmp GETLINE

                  message:
0310  8D                  .byte CR
0311  42 4F 4E 4A         .asciiz "BONJOUR, APPLE-1 !"
0315  4F 55 52 2C
…
; ---- Symboles (13)
; ECHO         $FFEF
; GETLINE      $FF1F
; message      $0310
; start        $0300
; start@fin    $030D
; start@suite  $0302
```

Les étiquettes locales `@suite` et `@fin` appartiennent à `start` : la table les nomme `start@suite` et `start@fin`, et le débogueur accepte ces noms comme adresses de points d'arrêt.

22.3 La ligne de commande

`apple1-emu asm` fait la même chose sans fenêtre :

TERMINAL

```
apple1-emu asm hello.s                  # vérifier seulement
apple1-emu asm hello.s -l hello.lst     # avec le listing
apple1-emu asm hello.s -o hello.wav     # cassette (.wav ou .aif)
apple1-emu asm hello.s -o hello.bin     # binaire brut
apple1-emu asm hello.s -o hello.hex     # vidage Woz, à coller dans le moniteur
apple1-emu asm jeu.s -I ../commun       # autre dossier pour les .include
```

Chaque commande résume l'assemblage et donne la ligne pour le vrai Apple-1 :

```
36 octets, $0300-$0323 ; 2 fichier(s), 13 symboles
Cassette (après C100R) : 0300.0323R   puis 300R
écrit : hello.wav
```

Sans `.org`, le programme est placé en `$0300`. Le vidage `.hex` a la forme que le Woz Monitor accepte au clavier, terminée par la commande de lancement :

t1.hex (extrait)

```
0280: 34 12 34 12 10 05 FF 34
0288: 12 FF FF A8 02 EA EA EA
…
0280R
```

22.4 La syntaxe

Une ligne contient, dans l'ordre et toutes facultatives : une étiquette, une instruction ou une directive, et un commentaire après `;`. Les mnémoniques et les directives s'écrivent en minuscules ou en majuscules ; les symboles distinguent les deux.

```6502
PTR     = $06           ; une constante (pas d'octet produit)
start:  lda #0          ; une étiquette globale, suivie de deux-points
@boucle: inx            ; une étiquette locale : visible jusqu'à la globale suivante
        bne @boucle
:       dex             ; une étiquette anonyme
        bne :-          ; :- la précédente, :+ la suivante, :-- l'avant-dernière
```

Les nombres : `255` en décimal, `$FF` en hexadécimal, `%1010` en binaire, `'A'` pour un caractère ($41). Le clavier rend ses codes avec le **bit 7 à 1** : pour comparer une touche, écrivez `'A' | $80` ; `apple1.inc` définit de même `CR = $8D`. `*` désigne l'adresse courante.

Les opérateurs, du moins prioritaire au plus prioritaire :

Priorité	Opérateurs	Sens
1	`||`	ou logique (résultat 0 ou 1)
2	`&&`	et logique
3	`=` `<>` `<` `>` `<=` `>=`	comparaisons (0 ou 1)
4	`+` `-` `|`	addition, soustraction, ou bit à bit
5	`*` `/` `&` `^` `<<` `>>`	multiplication, division, et, ou exclusif, décalages
6 (unaires)	`<` `>` `-` `~` `!`	octet bas, octet haut, opposé, complément, non logique

Ainsi `lda #<message` charge l'octet bas de l'adresse et `lda #>message` l'octet haut ; `#>S+1` vaut `(>S)+1`. Il n'y a pas d'opérateur modulo (`%` ou `.mod` donnent une erreur de syntaxe).

Page zéro ou absolu. Une adresse inférieure à `$100` connue au moment de l'assemblage est codée en page zéro (`lda $0006` donne `A5 06`, comme ca65). Pour forcer la forme absolue, écrivez `a:` (`lda a:$06` donne `AD 06 00`) ; pour forcer la page zéro, `z:`.

! DÉFINISSEZ LES CONSTANTES DE PAGE ZÉRO AVANT DE LES UTILISER

Si `PTR = $06` est écrit **après** `lda PTR`, l'assembleur ne connaît pas encore la valeur et code l'adresse sur 2 octets. Il le signale en jaune : `avertissement : adresse en page zéro codée sur 2 octets (symbole défini plus bas ; définissez-le avan`
Le programme marche, mais il est plus long, et les modes `(zp),Y` deviennent impossibles.

22.5 Les directives

Directive	Effet	Exemple
`.org adr`	adresse d'assemblage (et de chargement)	`.org $0300`
`.byte` (`.byt`)	octets, chaînes comprises	`.byte CR, "OK", 0`
`.word` (`.addr`)	mots de 16 bits, octet bas d'abord	`.word 10000, fin`
`.asciiz`	chaîne terminée par un 0	`.asciiz "BONJOUR"`
`.res n[, val]`	réserve n octets (0, ou val)	`.res 3, $EA`
`.include "f"`	insère un fichier	`.include "apple1.inc"`
`.macro nom args` … `.endmacro`	définit une macro (`.mac` , `.endmac`)	voir `macros.s`
`.local a, b`	étiquettes propres à chaque appel de macro	
`.repeat n[, var]` … `.endrep`	répète un bloc, var = 0 … n−1	`.repeat 16, I`
`.if` `.elseif` `.else` `.endif`	assemblage conditionnel	`.if LARGEUR > 30`
`.ifdef` `.ifndef`	selon qu'un symbole existe	`.ifdef APPLE1`
`.ifblank` `.ifnblank`	argument de macro vide ou non	
`.defined(sym)`	1 si le symbole existe (dans une expression)	`.if .defined(d)`
`.define nom valeur`	substitution textuelle (`.undefine`)	`.define CARACTERE '*'`
`.setcpu "65C02"`	accepte les instructions du 65C02 (`.pc02` , `.p02`)	
`.segment` `.code` `.data` `.rodata` `.bss`	acceptés pour la compatibilité	
`.feature string_escapes`	`\n` , `\"` … dans les chaînes	

! LES SEGMENTS NE DÉPLACENT RIEN

Avec ca65 et ld65, `.code` , `.data` et `.bss` envoient le code dans des zones définies par la configuration de l'éditeur de liens. L'assembleur intégré produit **un seul bloc**, dans l'ordre de la source : les directives de segment sont acceptées mais n'ont pas d'effet, et `.bss` produit des zéros. Pour un projet qui a besoin d'un vrai fichier `.cfg` , la bibliothèque utilise ca65 et ld65 (chapitre 4).

22.6 Les macros, .repeat et l'assemblage conditionnel

Une macro est un modèle de lignes, recopié à chaque appel avec ses arguments. `.local` donne à chaque appel ses propres étiquettes, sans conflit. `.repeat` déroule un bloc à l'assemblage, ce qui sert aussi à **calculer des tables** :

```6502
carres:
.repeat 16, I
        .byte I * I     ; 0, 1, 4, 9 … 225
.endrep
```

Le chapitre 23 présente `macros.s`, qui utilise tout cela, et `gen2-lores.s`, qui calcule les adresses des 24 rangées de l'écran avec `.repeat`.

22.7 Les messages d'erreur

Chaque erreur donne le fichier, la ligne, la colonne et le texte en cause ; aucun octet n'est écrit tant qu'il reste une erreur.

Message	Cause habituelle
`erreur de syntaxe`	opérateur inconnu, parenthèse manquante
`instruction, directive ou macro inconnue`	faute de frappe ; instruction du 65C02 sans `.setcpu` ; étiquette sans deux-points
`mode d'adressage impossible pour cette instruction`	`sta #5`, `ldx ($06),y` …
`symbole non défini`	`jsr ECO` au lieu de `ECHO` ; `.include` oublié
`symbole déjà défini`	deux étiquettes du même nom
`valeur hors limites`	`lda #300`, adresse au-delà de `$FFFF`
`branchement trop loin (-128 à +127 octets)`	remplacez `bne loin` par `beq :+` / `jmp loin` / `:`
`fichier .include introuvable`	nom, ou dossier : ajoutez `-I`
`bloc mal fermé (.macro/.endmacro, .if/.endif)`	`.endif` ou `.endmacro` oublié
`division par zéro`	dans une expression
`chaîne ou caractère sans guillemet fermant`	`"BONJOUR`
`valeur inconnue à cet endroit (.org, .res, .if, .repeat)`	ces directives veulent une valeur déjà définie

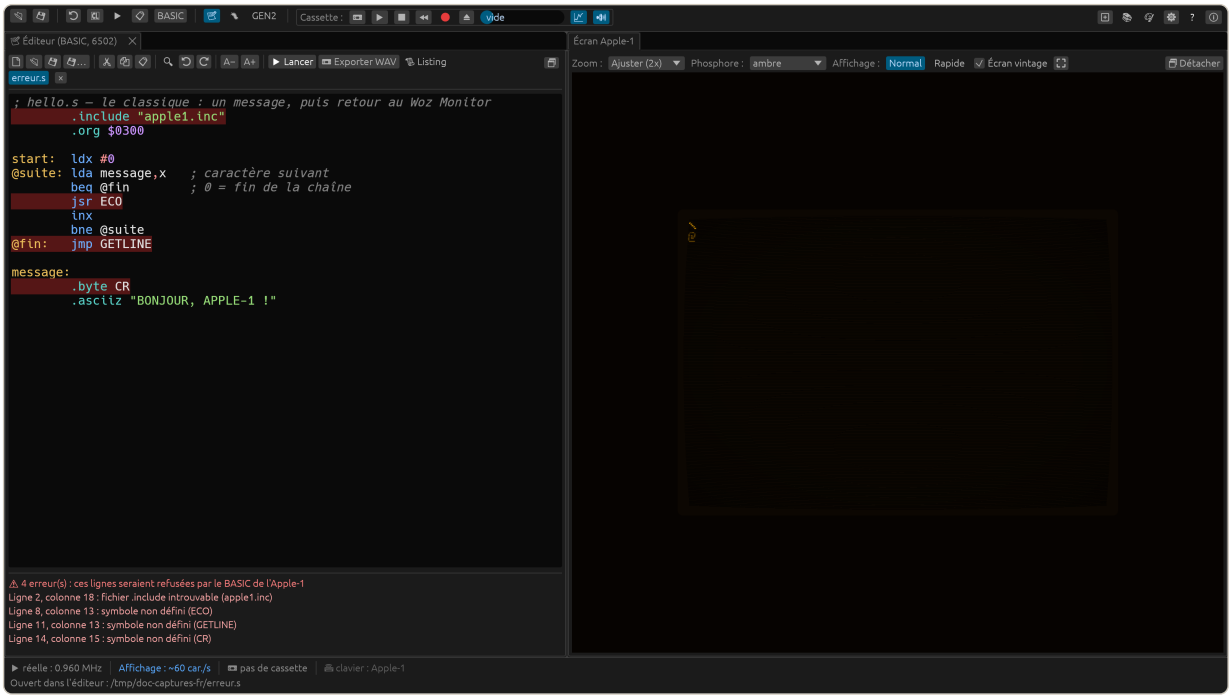


Figure 35 — Une erreur dans l'éditeur : ECO au lieu de ECHO. Un clic sur le message mène à la ligne.

22.8 Compatibilité avec ca65 et xa

L'assembleur intégré suit la syntaxe de **ca65** (paquet cc65) pour tout ce qui sert à un programme d'un seul bloc. Les huit exemples du chapitre 23 ont été assemblés avec les deux outils et comparés octet par octet :

Source	Octets	Intégré = ca65
`hello.s`	36	**identique**
`clavier.s`	32	**identique**
`hexa.s`	64	**identique**
`decimal.s`	117	**identique**
`jeu.s`	254	**identique**
`macros.s`	137	**identique**
`gen2-lores.s`	110	**identique**
`gen2-hires.s`	94	**identique**

Les sources écrites pour **xa** (xa65) sont reconnues à leurs lignes `#include` ou `#define`, et traduites ligne pour ligne avant l'assemblage (les numéros de ligne des erreurs restent justes) : `#define`, `#include`, `*=`, `.BYT`, `.ASC`, commentaires `/* */` et étiquettes en colonne 0 sans deux-points. Le diaporama de Dave Schmenk (chapitre 9) est assemblé ainsi, aux mêmes 3456 octets que xa 2.4.1.

Pour les **projets GitHub**, la bibliothèque préfère ca65 et ld65 s'ils sont installés (ou si le projet fournit une configuration ld65), avec `-D APPLE1` par défaut ; sinon, elle utilise l'assembleur intégré. Le fichier `apple1.toml` du dépôt précise les adresses et les symboles (chapitre 4).

CHAPITRE 23

Le clavier, l'écran et le Woz Monitor

L'Apple-1 n'a que quatre registres d'entrée-sortie, dans un PIA 6820 (ou 6821), et 256 octets de moniteur. C'est peu, mais cela suffit : ce chapitre montre comment lire une touche, écrire un caractère et utiliser les routines de Steve Wozniak.

23.1 Les quatre registres du PIA

Adresse	Nom	Lecture	Écriture
`$D010`	`KBD`	code de la touche, bit 7 à 1 ; la lecture **acquitte** la touche	—
`$D011`	`KBDCR`	bit 7 = 1 : une touche attend	configuration (faite par le moniteur)
`$D012`	`DSP`	bit 7 = 1 : l'écran est occupé	caractère à afficher (7 bits)
`$D013`	`DSPCR`	—	configuration (faite par le moniteur)

Ces noms sont définis dans `apple1.inc`, inclus par tous les exemples :

apple1.inc — 6502

```
; apple1.inc — adresses d'E/S et routines du Woz Monitor
; (vérifiées dans la ROM : FFDC PRBYTE, FFE5 PRHEX, FFEF ECHO, FF1F GETLINE)
KBD     = $D010         ; donnée clavier (bit 7 = 1)
KBDCR   = $D011         ; bit 7 = 1 : une touche attend
DSP     = $D012         ; écriture : caractère vers l'écran (bit 7 lu = occupé)
DSPCR   = $D013
PRBYTE  = $FFDC         ; affiche A en 2 chiffres hexa (détruit A)
PRHEX   = $FFE5         ; affiche le quartet bas de A (détruit A)
ECHO    = $FFEF         ; affiche le caractère de A (A, X, Y conservés)
GETLINE = $FF1F         ; retour au moniteur : retour à la ligne, puis invite
CR      = $8D           ; retour chariot (bit 7 mis, comme le moniteur)
```

23.2 Lire une touche

On attend que le bit 7 de `KBDCR` passe à 1, puis on lit `KBD` :

```6502
lire:   lda KBDCR       ; une touche ?
        bpl lire        ; bit 7 = 0 : pas encore
        lda KBD         ; code ASCII + $80
```

Le clavier ne produit que des majuscules (l'émulateur convertit les minuscules), avec le bit 7 à 1 : `A` donne `$C1`, RETURN `$8D`, ESC `$9B`. Pour savoir si une touche est appuyée **sans attendre**, testez seulement `KBDCR` :

```6502
        lda KBDCR
        bpl rien        ; pas de touche : on continue le travail
        lda KBD         ; une touche : on la lit (et on l'acquitte)
```

`jeu.s` (chapitre 23) s'en sert pour tirer un nombre au hasard : il compte en boucle jusqu'à ce que le joueur appuie sur une touche.

23.3 Écrire un caractère

Le terminal de l'Apple-1 prend un caractère à la fois, et seulement quand il est prêt : le bit 7 de `DSP` reste à 1 tant que le caractère précédent n'est pas affiché. La routine `ECHO` du moniteur fait l'attente pour vous. Voici ses quatre instructions, lues dans le moniteur de l'émulateur :

```WOZ MONITOR
\
FFEF.FFFFR

FFEF: 2C
FFF0: 12 D0 30 FB 8D 12 D0 60
FFF8: 00 00 00 0F 00 FF 00 00
```

Soit `BIT DSP` / `BMI ECHO` / `STA DSP` / `RTS`. `ECHO` conserve A, X et Y. Comme le terminal d'origine n'affiche qu'**un caractère par image**, environ 60 par seconde, un programme qui écrit beaucoup est limité par l'écran, pas par le processeur ; l'émulateur respecte cette limite par défaut (F4 la lève, chapitre 2).

Ce que fait l'écran de chaque code :

- **RETURN** (`$8D`, ou `$0D`) passe à la ligne ; en bas de l'écran, tout remonte d'une ligne.
- Les **autres caractères de contrôle** (`$00-$1F`, avec ou sans bit 7) sont ignorés : pas d'effacement, pas de retour arrière, pas de sonnerie.
- Les **minuscules** `$60-$7F` s'affichent comme les majuscules `$40-$5F` : le terminal ne garde que 6 bits.
- Après la 40e colonne, le curseur passe seul à la ligne suivante.
- Il n'y a pas de code pour effacer l'écran : c'est un bouton de la machine (**Effacer écran**, Ctrl+L).

23.4 Les routines du Woz Monitor

Adresse	Nom	Entrée	Effet	Registres
`$FFEF`	`ECHO`	A = caractère	l'affiche	A, X, Y conservés
`$FFDC`	`PRBYTE`	A = octet	l'affiche en 2 chiffres hexadécimaux	A détruit ; X, Y conservés
`$FFE5`	`PRHEX`	A (quartet bas)	affiche un chiffre hexadécimal	A détruit ; X, Y conservés
`$FF1F`	`GETLINE`	—	retour au moniteur : à la ligne, puis attend une commande	—
`$FF1A`	`ESCAPE`	—	comme `GETLINE`, en affichant d'abord `\`	—
`$FF00`	`RESET`	—	initialise le PIA et le moniteur	—

Le moniteur utilise la page zéro `$24-$2B` et le tampon de ligne `$0200-$027F` ; tout le reste est à vous (sauf si l'Integer BASIC doit survivre, chapitre 16). Ses commandes (`300R`, `0300.031F`, `300: A9 8D`…) sont résumées à l'annexe B.

23.5 Lancer et terminer un programme

Un programme est lancé par la commande `R` du moniteur (`300R`), qui fait un `JMP` à l'adresse, pas un `JSR`. Il n'y a donc **pas d'adresse de retour** sur la pile : terminez par `jmp GETLINE` (`$FF1F`), pas par `RTS`.

> **! RTS ET BRK NE RENDENT PAS LA MAIN**
>
> Un `RTS` à la fin d'un programme lancé par `R` dépile n'importe quoi. Un `BRK` saute au vecteur d'interruption de la ROM, qui vaut `$0000` (les trois derniers mots du moniteur, ci-dessus : NMI `$0F00`, RESET `$FF00`, IRQ/BRK `$0000`) : le processeur exécute alors la page zéro. Dans les deux cas, faites RESET (Ctrl+R).

Un programme appelé depuis le BASIC par `CALL 768` est au contraire appelé par un `JSR` : il doit finir par `RTS` pour revenir au BASIC. Le chapitre 16 montre comment loger du code machine sous le programme BASIC.

WOZ MONITOR

```
>POKE 768,96
>CALL 768
>
```

(`96` = `$60`, l'opcode de `RTS` : le plus petit sous-programme possible.)

23.6 Le PIA, la cassette et la carte couleur

La carte ACI occupe `$C000-$C0FF` (chaque accès fait basculer la sortie) et son programme `$C100-$C1FF` ; le chapitre 5 explique son fonctionnement et l'annexe C ses temps. La carte GEN2 se commande par simple **lecture** de `$C250-$C257` (`bit SS_GR`) ; ses adresses sont dans `gen2.inc` :

gen2.inc 6502

```
; gen2.inc — commutateurs de la carte couleur GEN2 d'Uncle Bernie
; (les $C050-$C057 de l'Apple II, déplacés en $C25x ; une LECTURE commute)
SS_GR    = $C250        ; graphique (quitte le mode texte)
SS_TEXT  = $C251        ; texte
SS_FULL  = $C252        ; plein écran
SS_MIX   = $C253        ; 4 lignes de texte en bas
SS_PAGE1 = $C254
SS_PAGE2 = $C255
SS_LORES = $C256
SS_HIRES = $C257
```

CHAPITRE 24

Exemples commentés

Huit programmes, du plus simple au plus complet. Ils sont dans le dossier `docs/exemples/asm/` de la distribution ; chacun a été assemblé par l'assembleur intégré et par ca65 (mêmes octets), puis exécuté dans l'émulateur. Les sorties montrées ici sont celles de l'émulateur.

Pour les essayer : ouvrez le fichier dans l'éditeur et cliquez sur ▶ **Lancer**, ou bien, en ligne de commande, `apple1-emu asm hello.s -o hello.wav` puis lisez la cassette (chapitre 5). Gardez `apple1.inc` (et `gen2.inc`) dans le même dossier que les sources.

Fichier	Octets	Adresse	Ce qu'il montre
`hello.s`	36	`$0300`	une chaîne, `ECHO`, étiquettes locales, retour au moniteur
`clavier.s`	32	`$0300`	lire le clavier, comparer des caractères
`hexa.s`	64	`$0300`	pointeur en page zéro, `(zp),Y`, addition 16 bits, `PRBYTE`
`decimal.s`	117	`$0300`	nombres de 16 bits, soustractions, tables de mots
`jeu.s`	254	`$0300`	un jeu complet : hasard, sous-programmes, messages
`macros.s`	137	`$0300`	macros, `.local`, `.repeat`, `.if`, tables calculées
`gen2-lores.s`	110	`$0C00`	la carte GEN2 en LORES
`gen2-hires.s`	94	`$0C00`	la carte GEN2 en HIRES

24.1 `hello.s` : afficher un message

hello.s 6502

```
; hello.s — le classique : un message, puis retour au Woz Monitor
        .include "apple1.inc"
        .org $0300

start:  ldx #0
@suite: lda message,x   ; caractère suivant
        beq @fin        ; 0 = fin de la chaîne
        jsr ECHO
        inx
        bne @suite
@fin:   jmp GETLINE

message:
        .byte CR
        .asciiz "BONJOUR, APPLE-1 !"
```

X parcourt la chaîne jusqu'à l'octet 0 posé par `.asciiz`. Le `bne @suite` après `inx` n'est jamais faux pour une chaîne de moins de 256 caractères : c'est un saut court et sûr. Le programme finit par `jmp GETLINE`, jamais par `RTS` (chapitre 22).

WOZ MONITOR

```
\
300R
0300: A2
BONJOUR, APPLE-1 !
```

Le moniteur affiche d'abord `0300: A2`, l'octet à l'adresse de lancement, puis le programme prend la main.

24.2 `clavier.s` : l'écho des touches

clavier.s 6502

```
; clavier.s — lit le clavier et renvoie chaque touche à l'écran.
; Les chiffres sont doublés ; ÉCHAP (ESC) rend la main au moniteur.
        .include "apple1.inc"
        .org $0300

lire:   lda KBDCR       ; une touche ?
        bpl lire        ; bit 7 = 0 : pas encore
        lda KBD         ; code ASCII + $80 ; la lecture acquitte la touche
        cmp #$9B        ; ESC
        beq fin
        jsr ECHO
        cmp #'0' | $80
        bcc lire        ; < '0'
        cmp #'9' | $80 + 1
        bcs lire        ; > '9'
        jsr ECHO        ; un chiffre : on l'affiche une 2e fois
        jmp lire
fin:    jmp GETLINE
```

`cmp #'0' | $80` compare au code du chiffre 0, avec le bit 7 comme le clavier. `bcc` (« plus petit ») et `bcs` (« plus grand ou égal ») encadrent les chiffres. Avec les touches `A`, `B`, `1`, `2` puis ESC, l'écran montre :

```
0300: ADAB1122
\
```

Le `AD` vient du moniteur (`lda KBDCR` commence par l'opcode `$AD`) ; `AB` puis `11` et `22` viennent du programme ; ESC rend la main au moniteur, qui affiche `\`.

24.3 `hexa.s` : un vidage façon moniteur

hexa.s 6502

```
; hexa.s — vidage mémoire façon moniteur avec PRBYTE :
; 2 lignes de 8 octets à partir de $FF00
        .include "apple1.inc"
        .org $0300
PTR     = $06           ; pointeur en page zéro (libre pour le moniteur)

start:  lda #<$FF00
        sta PTR
        lda #>$FF00
        sta PTR+1
        ldx #2          ; nombre de lignes
ligne:  lda #CR
```

```
        jsr ECHO
        lda PTR+1       ; adresse : octet fort puis faible
        jsr PRBYTE
        lda PTR
        jsr PRBYTE
        lda #':' | $80
        jsr ECHO
        ldy #0
octet:  lda #' ' | $80
        jsr ECHO
        lda (PTR),y
        jsr PRBYTE      ; PRBYTE détruit A mais garde X et Y
        iny
        cpy #8
        bne octet
        clc             ; PTR += 8
        lda PTR
        adc #8
        sta PTR
        bcc :+
        inc PTR+1
:       dex
        bne ligne
        jmp GETLINE
```

`PTR` et `PTR+1` forment un **pointeur 16 bits** en page zéro, octet bas d'abord ; `lda (PTR),y` lit l'octet à l'adresse PTR + Y. L'addition `PTR += 8` montre la retenue : `clc`, `adc #8`, et si la retenue sort (`bcc` non pris), `inc PTR+1`. Le résultat est le début du Woz Monitor :

```
FF00: D8 58 A0 7F 8C 12 D0 A9
FF08: A7 8D 11 D0 8D 13 D0 C9
```

24.4 `decimal.s` : afficher un nombre en décimal

decimal.s 6502

```
; decimal.s — affiche un entier 16 bits non signé en décimal
; (soustractions successives des puissances de 10, sans zéros de tête)
        .include "apple1.inc"
        .org $0300
NUM     = $06           ; nombre à afficher (2 octets)
FLAG    = $08           ; 1 dès qu'un chiffre non nul est sorti

start:  lda #CR
        jsr ECHO
        ldx #0
@liste: lda nombres,x   ; affiche chaque nombre de la liste
        sta NUM
        lda nombres+1,x
```

```
        sta NUM+1
        jsr prdec
        lda #' ' | $80
        jsr ECHO
        inx
        inx
        cpx #fin_nombres - nombres
        bne @liste
        jmp GETLINE

; prdec : affiche NUM en décimal. Conserve X.
prdec:  txa
        pha
        lda #0
        sta FLAG
        ldx #0          ; index dans la table des puissances
@chif:  ldy #'0' | $80  ; Y = chiffre courant
@sous:  lda NUM         ; NUM - puissance
        sec
        sbc puiss,x
        pha
        lda NUM+1
        sbc puiss+1,x
        bcc @fini       ; négatif : chiffre trouvé
        sta NUM+1
        pla
        sta NUM
        iny
        bne @sous
@fini:  pla
        tya
        cmp #'0' | $80
        bne @affi
        ldy FLAG        ; zéro de tête ? on le saute
        beq @suiv
@affi:  jsr ECHO
        inc FLAG
@suiv:  inx
        inx
        cpx #8          ; 4 puissances : 10000 à 10
        bne @chif
        lda NUM         ; reste : les unités, toujours affichées
        ora #'0' | $80
        jsr ECHO
        pla
        tax
        rts

puiss:  .word 10000, 1000, 100, 10
nombres:
```

```
        .word 0, 7, 42, 1976, 6502, 65535
fin_nombres:
```

Le 6502 ne sait pas diviser : `prdec` soustrait 10000 autant de fois que possible (le compte donne le chiffre des dizaines de milliers), puis 1000, 100 et 10. La soustraction 16 bits se fait en deux temps, `sbc` sur l'octet bas puis sur l'octet haut, la retenue faisant le lien. `FLAG` supprime les zéros de tête. Sortie :

```
0 7 42 1976 6502 65535
```

✦ **CONSERVER X DANS UN SOUS-PROGRAMME**

`prdec` utilise X pour lui-même, alors que la boucle principale en a besoin : elle le sauve avec `txa` / `pha` au début et le restaure avec `pla` / `tax` à la fin. Le 6502 NMOS n'a pas de `PHX`.

24.5 `jeu.s` : devinez le chiffre

jeu.s 6502

```
; jeu.s — devinez le chiffre (0 à 9) : boucle de jeu simple.
; Le hasard vient du temps que met le joueur à appuyer sur une touche.
        .include "apple1.inc"
        .org $0300
SECRET  = $06
COMPTE  = $07

start:  ldx #<titre
        ldy #>titre
        jsr prmsg
        ldx #0          ; X tourne pendant l'attente : graine
@att:   inx
        lda KBDCR
        bpl @att
        lda KBD
        txa
@mod:   cmp #10         ; X modulo 10
        bcc @ok
        sbc #10         ; C = 1 ici
        bcs @mod
@ok:    sta SECRET
        lda #0
        sta COMPTE

tour:   ldx #<question
        ldy #>question
        jsr prmsg
        jsr touche      ; A = chiffre 0..9 (les autres touches sont ignorées)
        inc COMPTE
        cmp SECRET
```

```
        beq gagne
        bcc @plus
        ldx #<moins
        ldy #>moins
        jsr prmsg
        jmp tour
@plus:  ldx #<plus
        ldy #>plus
        jsr prmsg
        jmp tour

gagne:  ldx #<bravo
        ldy #>bravo
        jsr prmsg
        lda COMPTE
        ora #'0' | $80  ; < 10 coups : un seul chiffre
        cmp #'9' | $80 + 1
        bcc :+
        lda #'+' | $80
:       jsr ECHO
        ldx #<coups
        ldy #>coups
        jsr prmsg
        jmp GETLINE

; touche : attend un chiffre, l'affiche, le rend en binaire dans A
touche: lda KBDCR
        bpl touche
        lda KBD
        cmp #'0' | $80
        bcc touche
        cmp #'9' | $80 + 1
        bcs touche
        jsr ECHO
        and #$0F
        rts

; prmsg : affiche la chaîne terminée par 0 dont l'adresse est en Y:X
MSG     = $08
prmsg:  stx MSG
        sty MSG+1
        ldy #0
:       lda (MSG),y
        beq :+
        jsr ECHO
        iny
        bne :-
:       rts

titre:    .byte CR
```

```
          .asciiz "J'AI CHOISI UN CHIFFRE. APPUYEZ SUR UNE TOUCHE."
question: .byte CR
          .asciiz "VOTRE CHIFFRE ? "
plus:     .asciiz " C'EST PLUS."
moins:    .asciiz " C'EST MOINS."
bravo:    .byte " GAGNE EN ", 0
coups:    .asciiz " COUPS !"
```

Le hasard vient du joueur : X compte sans arrêt (`@att`) jusqu'à la première touche, puis le reste de la division par 10 donne le chiffre secret. `prmsg` reçoit l'adresse d'un message dans **Y:X** (octet haut dans Y), la range dans le pointeur `MSG` , et l'affiche avec `(MSG),y` . Une partie, en tapant `x` puis les chiffres dans l'ordre :

```
J'AI CHOISI UN CHIFFRE. APPUYEZ SUR UNE TOUCHE.
VOTRE CHIFFRE ? 0 C'EST PLUS.
VOTRE CHIFFRE ? 1 C'EST PLUS.
…
VOTRE CHIFFRE ? 7 C'EST PLUS.
VOTRE CHIFFRE ? 8 GAGNE EN 9 COUPS !
```

24.6 `macros.s` : laisser travailler l'assembleur

6502

macros.s

```
; macros.s — macros, .repeat et assemblage conditionnel
        .include "apple1.inc"
        .org $0300

LARGEUR = 20            ; essayez 10 ou 40
.define CARACTERE '*'

; afficher un caractère constant (la macro produit lda/jsr)
.macro putc c
        lda #(c) | $80
        jsr ECHO
.endmacro

; afficher une chaîne : un label local à chaque appel
.macro print texte
        .local boucle, fin, chaine
        ldx #0
boucle: lda chaine,x
        beq fin
        jsr ECHO
        inx
        bne boucle
        beq fin
chaine: .asciiz texte
fin:
```

```
.endmacro

start:  putc 13
        print "MACROS ET .REPEAT"
        putc 13
.repeat 3               ; trois lignes d'étoiles, déroulées à l'assemblage
        ldx #LARGEUR
:       putc CARACTERE
        dex
        bne :-
        putc 13
.endrep
.if LARGEUR > 30
        print "LIGNES LONGUES"
.else
        print "LIGNES COURTES"
.endif
        jmp GETLINE

; une table calculée par l'assembleur : les carrés de 0 à 15
carres:
.repeat 16, I
        .byte I * I
.endrep
```

`putc` et `print` produisent du code à chaque appel ; `.local` évite que les deux appels de `print` définissent deux fois `boucle`. `.repeat 3` déroule trois fois la ligne d'étoiles (chacune avec sa propre étiquette anonyme), `.if` choisit le message selon `LARGEUR`, et la table `carres` est calculée à l'assemblage. Sortie :

```
MACROS ET .REPEAT
**********
**********
**********
LIGNES COURTES
```

Changez `LARGEUR = 20` en `40` : l'éditeur réassemble aussitôt, les lignes s'allongent et le message devient `LIGNES LONGUES`.

24.7 `gen2-lores.s` : la couleur en basse résolution

Ces deux programmes demandent la carte GEN2 (⚙ **Paramètres ▸ Carte couleur**, chapitre 6) et se lancent en `$0C00`, au-dessus de la page LORES.

6502

gen2-lores.s

```
; gen2-lores.s — barres de couleurs en LORES (40 x 48, 16 couleurs)
; Il faut la carte GEN2 (Paramètres ▸ Carte couleur). Une touche : retour texte.
        .include "apple1.inc"
```

```
        .include "gen2.inc"
        .org $0C00      ; au-dessus de la page LORES 1 ($0400-$07FF)
PTR     = $06

start:  ldx #0          ; X = rangée de texte (0..23)
rangee: lda lignes_lo,x
        sta PTR
        lda lignes_hi,x
        sta PTR+1
        ldy #39
colonne:
        tya             ; couleur = colonne / 2 (0..15, puis on recommence)
        lsr a
        and #$0F
        sta $00         ; un octet = 2 blocs : quartet bas en haut, haut en bas
        asl a
        asl a
        asl a
        asl a
        ora $00
        sta (PTR),y
        dey
        bpl colonne
        inx
        cpx #24
        bne rangee

        bit SS_LORES    ; LORES, plein écran, page 1, puis graphique
        bit SS_FULL
        bit SS_PAGE1
        bit SS_GR
attente:
        lda KBDCR
        bpl attente
        lda KBD
        bit SS_TEXT
        jmp GETLINE

; adresses des 24 rangées : $400 + (r & 7) * $80 + (r >> 3) * $28
lignes_lo:
.repeat 24, R
        .byte <($0400 + (R & 7) * $80 + (R >> 3) * $28)
.endrep
lignes_hi:
.repeat 24, R
        .byte >($0400 + (R & 7) * $80 + (R >> 3) * $28)
.endrep
```

Les deux tables `lignes_lo` et `lignes_hi` sont calculées par `.repeat` avec la formule des adresses de rangée du chapitre 6. Chaque octet porte deux blocs, d'où le quartet copié en haut et en bas. Les `bit SS_…` sont des **lectures** : c'est ainsi que se commutent les modes de la carte.

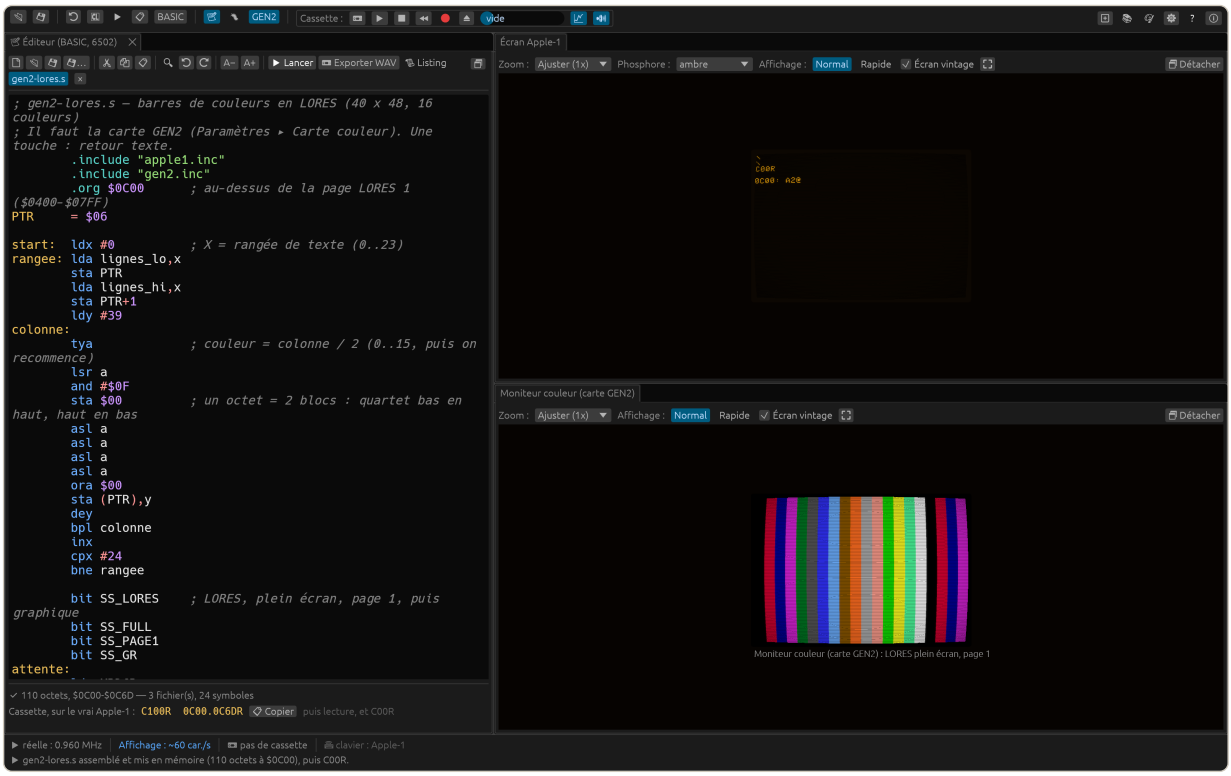


Figure 36 — gen2-lores.s : les seize couleurs LORES, deux colonnes chacune, sur le moniteur couleur.

24.8 `gen2-hires.s` : la haute résolution

gen2-hires.s 6502

```
; gen2-hires.s — trois bandes de couleur en HIRES (280 x 192)
; Dans chaque bloc de 128 octets de la page $2000, les octets 0-39 vont dans
; le tiers haut de l'écran, 40-79 au milieu, 80-119 en bas (120-127 : trous).
        .include "apple1.inc"
        .include "gen2.inc"
        .org $0C00
PTR     = $06

start:  lda #<$2000
        sta PTR
        lda #>$2000
        sta PTR+1
bloc:   ldy #0
        ldx #0          ; X = motif : 0 haut, 2 milieu, 4 bas
tiers:  lda #40
        sta $08
octet:  tya             ; colonne paire ou impaire ?
        and #1
        ora motif_idx,x
```

```
        sty $09
        tay
        lda motifs,y
        ldy $09
        sta (PTR),y
        iny
        dec $08
        bne octet
        inx
        cpx #3
        bne tiers
        clc             ; bloc suivant (+128)
        lda PTR
        adc #$80
        sta PTR
        bcc bloc
        inc PTR+1
        lda PTR+1
        cmp #$40        ; fin de la page 1 ($2000-$3FFF)
        bne bloc

        bit SS_HIRES
        bit SS_FULL
        bit SS_PAGE1
        bit SS_GR
attente:
        lda KBDCR
        bpl attente
        lda KBD
        bit SS_TEXT
        jmp GETLINE

motif_idx: .byte 0, 2, 4
; deux octets par motif (colonne paire, colonne impaire) :
; violet, vert, puis bleu et orange (bit 7 = 1 : l'autre palette)
motifs: .byte $55, $2A          ; violet
        .byte $2A, $55          ; vert
        .byte $D5, $AA          ; bleu
```

En HIRES, la page `$2000-$3FFF` est découpée en blocs de 128 octets dont les 40 premiers vont dans le tiers haut de l'écran, les 40 suivants au milieu et les 40 d'après en bas. Le programme remplit chaque tiers avec un motif différent, en alternant deux octets selon la parité de la colonne : on obtient trois bandes de couleur.

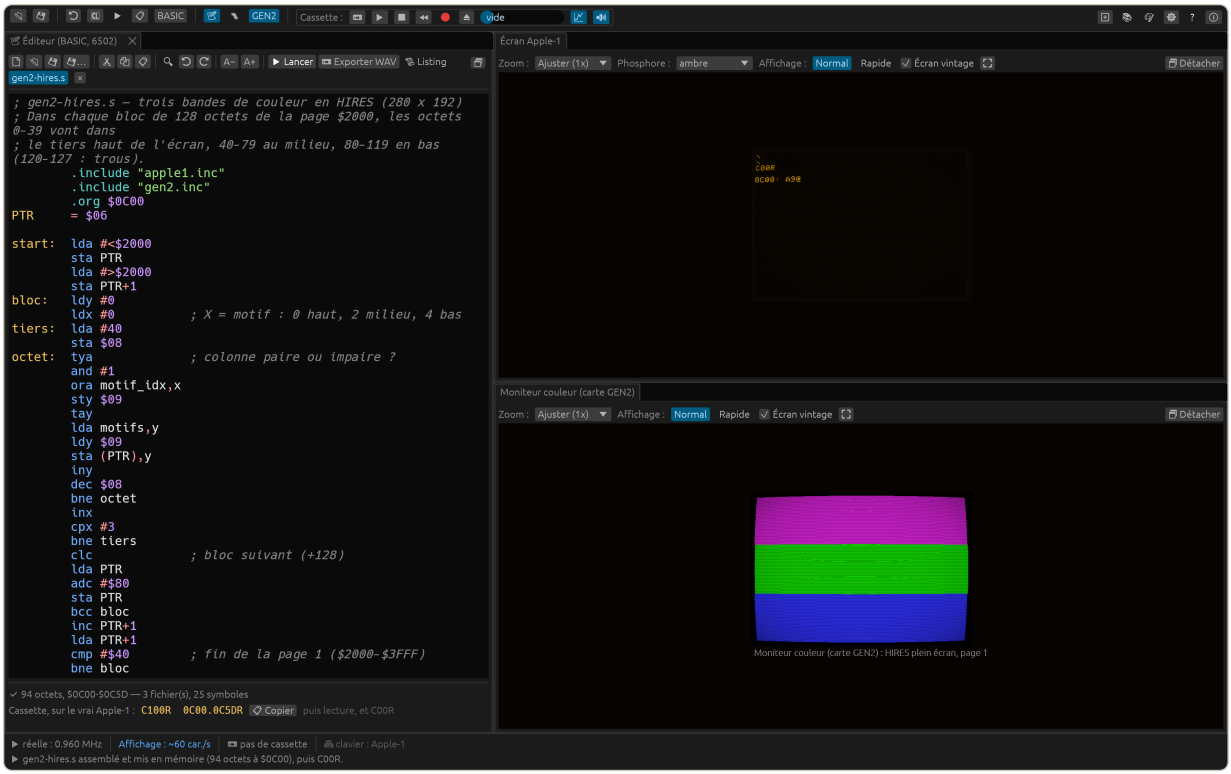


Figure 37 — gen2-hires.s : trois bandes de couleur en HIRES : violet, vert et bleu.

CHAPITRE 25

Astuces, pièges et limites

Ce qu'on apprend en écrivant des programmes pour l'Apple-1, dans l'émulateur puis sur la vraie machine.

25.1 Où placer un programme

Adresse	Pour
`$0280`	le plus bas possible : juste après le tampon du moniteur
`$0300`	l'usage courant (et l'adresse par défaut de l'assembleur)
`$0C00`	avec la carte GEN2 en LORES ou en texte (pages `$0400-$0BFF` libres pour l'écran)
sous HIMEM	avec l'Integer BASIC : réservez la place (chapitre 16)

Pour la page zéro, les exemples utilisent `$06-$09`, que le moniteur laisse libres. Si le programme doit cohabiter avec le BASIC, choisissez des adresses que le BASIC n'utilise pas, ou sauvez-les et restaurez-les.

25.2 Écrire du code qui tient dans 256 octets de saut

Un branchement ne va qu'à 127 octets en avant ou 128 en arrière. Quand l'assembleur répond `branchement trop loin`, inversez la condition et sautez par-dessus un `JMP` :

```6502
        beq :+          ; au lieu de : bne loin
        jmp loin
:
```

25.3 Les chaînes et le bit 7

Le clavier rend les codes avec le bit 7 à 1, et le moniteur compare avec ces codes. L'écran, lui, ignore le bit 7 : `.asciiz "BONJOUR"` s'affiche très bien. Pour **comparer** une touche, mettez le bit 7 dans la constante : `cmp #'Q' | $80`. Les minuscules d'une chaîne s'affichent en majuscules.

25.4 Attendre un peu

À 1,023 MHz (0,96 MHz effectifs), une boucle `dex` / `bne` coûte 5 cycles par tour :

```6502
attente:
        ldy #78         ; 78 x 256 tours de 5 cycles : environ 0,1 s
:       ldx #0
:       dex
        bne :-
        dey
        bne :--
        rts
```

L'émulateur en vitesse réelle respecte ces durées ; en vitesse rapide (F3), elles disparaissent. Les boucles d'attente de l'ACI sont comptées en cycles de la même façon (annexe C).

25.5 Déboguer efficacement

- Posez le point d'arrêt sur une **étiquette** (`start` , `prdec` , `start@fin`) plutôt que sur une adresse : il reste juste quand le code bouge.
- Passez les `jsr ECHO` avec ↪ **Par-dessus** : sinon vous suivez la boucle d'attente de l'écran, instruction par instruction.
- Si l'Apple-1 ne répond plus après votre programme, regardez PC : en page zéro, c'est presque toujours un `BRK` ou un `RTS` de trop (chapitre 22).
- Le **dump** montre la page zéro : vos pointeurs y sont visibles, octet bas d'abord.

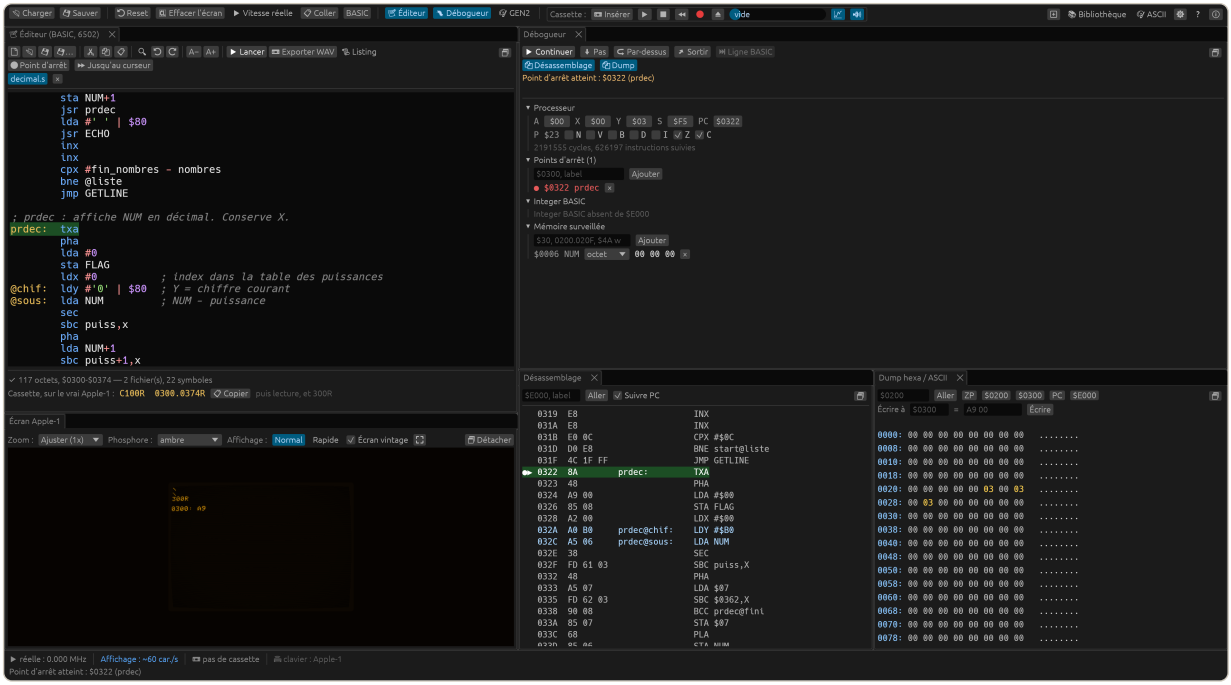


Figure 38 — Le débogueur arrêté dans une source 6502 : registres, drapeaux, désassemblage avec les étiquettes, et mémoire.

25.6 Vers le véritable Apple-1

1. Assemblez et essayez dans l'émulateur, **en vitesse réelle** et avec l'affichage limité à 60 caractères par seconde.
2. Exportez la cassette (📼 **Exporter WAV**, ou `-o prog.aif`) : le format AIFF est le plus sûr depuis un téléphone.
3. Sur l'Apple-1 : `C100R`, puis la plage affichée (`0300.0323R`), lancez la lecture, puis `300R`.
4. Sans ACI : le vidage `.hex` se tape (ou se colle dans un terminal série) directement dans le Woz Monitor ; sa dernière ligne lance le programme.

25.7 Les limites de l'assembleur intégré

- Un seul bloc de sortie, dans l'ordre de la source : les segments n'ont pas d'effet, et il n'y a pas d'éditeur de liens. Pour du code réparti sur plusieurs zones, utilisez ca65 et ld65.
- Pas d'opérateur modulo, pas de `.proc`, `.scope`, `.struct` ni `.enum` : ce sont des directives de ca65 que l'assembleur intégré ne connaît pas.
- Les instructions du 65C02 sont assemblées mais le processeur émulé est un 6502 NMOS : elles ne fonctionnent pas dans l'émulateur (chapitre 20).
- Dans le **listing**, une ligne produite par une macro perd parfois l'espace entre l'instruction et l'opérande (`ldachaine, x` pour `lda chaine,x`). C'est un défaut d'affichage du listing seulement : les octets sont justes.

ANNEXES

Annexes

Carte mémoire, commandes du Woz Monitor, temps de l'ACI, jeu de caractères et raccourcis clavier

ANNEXE A

Carte mémoire

Adresses	Contenu	Remarques
FF00–FFFF	**Woz Monitor** (PROM 256 octets)	vecteurs NMI, RESET, IRQ
E000–EFFF	**Integer BASIC** (4 Ko de RAM)	chargé par cassette
D010–D013	**PIA 6821**: clavier, écran	KBD, KBDCR, DSP, DSPCR
C250–C257	**Carte GEN2**: commutateurs	option, en lecture
C100–C1FF	**PROM de l'ACI**(C100R)	C000–C0FF : bascule
1000–BFFF	RAM d'extension (jusqu'à 48 Ko)	HIRES 2000–5FFF (GEN2)
0800–0FFF	programmes BASIC (LOMEM–HIMEM)	texte/LORES 2 : 0800
0300–07FF	vos programmes en langage machine	texte/LORES 1 : 0400
0200–027F	tampon de saisie du moniteur	ne rien y mettre
0100–01FF	pile du 6502	
0000–00FF	page zéro	moniteur 24–2B, BASIC 4A–FF

Figure 39 — La mémoire de l'Apple-1 dans l'émulateur, des vecteurs du moniteur à la page zéro.

Adresses	Contenu	Détail
`$0000-$00FF`	page zéro	Woz Monitor `$24-$2B` ; Integer BASIC `$4A-$FF` (pointeurs LOMEM `$4A`, HIMEM `$4C`, PP `$CA`, PV `$CC`)
`$0100-$01FF`	pile du 6502	
`$0200-$027F`	tampon de ligne du moniteur	128 caractères
`$0280-$0FFF`	RAM de base (4 Ko)	programmes en langage machine ; BASIC entre LOMEM (`$0800`) et HIMEM (`$1000`)
`$1000-$BFFF`	RAM d'extension	8 Ko par défaut dans l'émulateur (jusqu'à `$1FFF`), jusqu'à 48 Ko avec `--ram`
`$C000-$C0FF`	ACI : bascule de sortie, entrée bande	tout accès fait basculer la sortie
`$C100-$C1FF`	ACI : PROM (programme de l'ACI)	lancé par `C100R`
`$4000-$7FFF`	CodeTank de la P-LAB	EPROM de 16 Ko, en lecture seule, quand une ROM est branchée (par-dessus la RAM)

Adresses	Contenu	Détail
`$C250-$C257`	carte GEN2 : commutateurs	lecture ; pages texte/LORES `$0400`, `$0800`, HIRES `$2000`, `$4000`
`$CC00-$CC01`	carte P-LAB : TMS9918A	`$CC00` données, `$CC01` contrôle et état ; répété sur `$CC00-$CFFF`
`$D010-$D013`	PIA : `KBD`, `KBDCR`, `DSP`, `DSPCR`	chapitre 22
`$E000-$EFFF`	BASIC	RDK G2 Basic (7 933 octets, il a besoin de RAM jusqu'à `$FEFF`), ou Integer BASIC d'Apple (4 Ko de RAM, chargés par cassette ou image)
`$FF00-$FFFF`	Woz Monitor	vecteurs : NMI `$0F00`, RESET `$FF00`, IRQ/BRK `$0000`

ANNEXE B

Les commandes du Woz Monitor

Le moniteur attend une ligne terminée par RETURN. Les adresses et les octets s'écrivent en hexadécimal, sans `$` ; une ligne peut contenir plusieurs commandes séparées par des espaces.

Commande	Effet	Exemple
`adr`	affiche l'octet à cette adresse	`FFEF`
`adr.adr`	affiche une plage, 8 octets par ligne	`FF00.FF1F`
`.adr`	continue l'affichage jusqu'à cette adresse	`.FF3F`
`adr: oct oct …`	range des octets à partir de l'adresse	`300: A9 C1 20 EF FF`
`: oct …`	continue le rangement	`: 4C 1F FF`
`adrR`	lance le programme à cette adresse (`JMP`)	`300R`
Échap	annule la ligne (affiche `\`)	
`_` (retour arrière)	annule le dernier caractère	

Une ligne de plus de 127 caractères est annulée comme par Échap. Après `R`, le moniteur affiche d'abord l'octet de l'adresse (`0300: A2`), puis le programme prend la main.

Adresse	Commande	Rôle
`E000R`	démarrage à froid du BASIC	efface le programme
`E2B3R`	démarrage à chaud du BASIC	garde le programme
`C100R`	programme de l'ACI	puis `début.finR` (lire) ou `début.finW` (écrire)
`FF00R`	RESET du moniteur par logiciel	

ANNEXE C

Les temps de l'ACI

Les durées sont produites par le programme de l'ACI, en comptant les cycles entre deux basculements de la sortie. Valeurs mesurées dans l'émulateur, qui exécute ce programme sur son 6502 (horloge 1,022727 MHz, avec l'arrêt de 4 cycles sur 65 pour le rafraîchissement de la mémoire) :

Élément	Cycles	Durée	Fréquence
demi-période de l'en-tête	592	618 µs	≈ 809 Hz
1re moitié du bit de départ	180	188 µs	
demi-période d'un « 0 »	238	248 µs	≈ 2,0 kHz
demi-période d'un « 1 »	473	494 µs	≈ 1,0 kHz
en-tête complet	16 390 demi-périodes	≈ 10,1 s	

Format d'un enregistrement :

1. l'**en-tête** : environ 10 secondes de demi-périodes de 618 µs ;
2. le **bit de départ** : une demi-période courte (188 µs), puis une demi-période de « 0 » ;
3. les **données** : chaque octet, bit de poids fort en premier, une période complète par bit ;
4. rien d'autre : ni longueur, ni adresse, ni somme de contrôle. C'est la commande de lecture qui donne la plage.

À la lecture, le programme de l'ACI mesure les demi-périodes par une boucle de 12 cycles. Il prend pour bit de départ une demi-période de moins de 31 tours (≈ 372 cycles, 388 µs), et sépare « 0 » et « 1 » à environ 0,74 ms par période complète. Le décodeur de fichiers de l'émulateur accepte des en-têtes de 0,40 à 0,90 ms par demi-période, ce qui couvre l'outil c2t (1 kHz) et des écarts de vitesse de bande.

ANNEXE D

Le jeu de caractères du 2513



Figure 40 — Les 64 caractères du Signetics 2513, tels que l'émulateur les dessine : codes $20 à $5F (avec le bit 7, $A0 à $DF).

Le générateur de caractères ne connaît que 64 signes : l'espace, la ponctuation, les chiffres et les majuscules. Le terminal ne garde que 6 bits du code envoyé :

Code envoyé	Affichage
`$20-$5F` (ou `$A0-$DF`)	le caractère de la table
`$60-$7F` (minuscules, ou `$E0-$FF`)	la majuscule correspondante (`$40-$5F`)
`$0D` (ou `$8D`)	retour à la ligne
autres `$00-$1F`	rien : ignorés

Chaque caractère est une matrice de 5 × 7 points dans une case de 7 × 8 ; l'écran fait 40 × 24 caractères, soit 280 × 192 points.

ANNEXE E

Les raccourcis clavier

Touche	Effet
Ctrl + R	RESET de l'Apple-1
Ctrl + L	CLEAR SCREEN
Ctrl + V	coller : tape le presse-papiers au clavier de l'Apple-1
F1	mode aide (bulles détaillées)
F3	vitesse réelle ou rapide
F11	plein écran
Ctrl + + / Ctrl + − / Ctrl + 0	zoom de l'écran (ou du texte de l'éditeur, s'il a le clavier)
Ctrl + S	enregistrer le fichier de l'éditeur
Ctrl + F / Ctrl + H / Ctrl + G	éditeur : rechercher, remplacer, aller à la ligne
Ctrl + /	éditeur : commenter / décommenter (6502), REM / dé-REM (BASIC)
F5 / F9 / F12	éditeur : lancer, point d'arrêt sur la ligne, aller à la définition (6502)
Ctrl +lettre	caractère de contrôle (`Ctrl+D` = `$84`)
clic droit sur un écran	Copier, Coller (taper le texte), Effacer l'écran, Reset
Échap	ESC (`$9B`) : annule la ligne dans le moniteur et le BASIC
retour arrière	tape `_` , l'effacement du moniteur et du BASIC

Dans le **terminal** (`apple1-emu` sans fenêtre, chapitre 12) :

Touche	Effet
F2	ligne de commande de l'émulateur
F5 / F6	cassette : lecture, arrêt
F10 ou Ctrl + Q	quitter

ANNEXE F

Pièges et comportements à connaître

De petites choses qui surprennent la première fois. Aucune n'est un bogue ; chacune a sa raison.

- **BASIC et RUN. ▶ Lancer** de l'éditeur démarre le BASIC, charge le listing et tape `RUN` pour vous. Un programme qui demande une saisie attend vos touches sur l'écran de l'Apple-1 : cliquez d'abord sur l'écran.
- **RND.** Le premier `RND` d'une exécution serait toujours le même sur une machine qui vient d'être remise à zéro ; **▶ Lancer** attend un moment aléatoire avant `RUN`, pour qu'un jeu ne commence pas toujours de la même façon.
- **Commandes de cassette.** La ligne « Cassette, sur le vrai Apple-1 » sous l'éditeur est la commande du Woz Monitor à taper sur une vraie machine (`C100R`, puis les commandes `R` des plages) avant de jouer la bande. C'est une indication, pas quelque chose à taper dans l'émulateur.
- **Le clavier ne va qu'à un endroit à la fois.** La ligne en bas de la fenêtre dit où : l'Apple-1, l'éditeur, un champ. Cliquer sur l'écran de l'Apple-1 envoie les touches à la machine ; cliquer dans l'éditeur les reprend. Après un clic sur une erreur de la liste de l'éditeur, les touches reviennent à l'éditeur.
- **Débogueur en pause.** Les touches tapées pendant que la machine est en pause ne sont pas perdues, mais seule la **dernière** est gardée (le verrou du vrai clavier ne tient qu'un caractère) ; elle est livrée quand vous continuez.
- **Les points d'arrêt suivent le programme.** Charger un autre programme (fichier, bibliothèque, assembleur) efface les points d'arrêt et les étiquettes du précédent. Les points d'arrêt que vous posez ne sont retenus d'une session à l'autre que tant que vous ne chargez rien d'autre.
- **Coller avec le bouton du milieu.** Dans l'éditeur, un clic du milieu place le curseur là et colle la **sélection primaire** X11/Wayland (le texte que vous venez de surligner dans n'importe quelle application). L'éditeur n'*écrit* pas encore sa propre sélection dans la sélection primaire : pour copier depuis l'éditeur, utilisez Ctrl+C.
- **Petites fenêtres.** Sur un moniteur de moins de 1400 points de large, la disposition du premier lancement est éditeur et écran ; le menu **⊞ Disposition ▾** permet d'en choisir une autre. Sur un écran de 1024 × 768, la barre d'outils défile horizontalement : le bouton **»** rend le groupe Cassette accessible, au lieu de le couper à droite. Un zoom fixe (Ctrl++) plus grand que le volet de l'écran est gardé, et l'écran défile dans son volet ; la barre d'état le dit.
- **Enregistrer un onglet sans titre. Enregistrer sous** (icône, Ctrl+S ou menu du clic droit de l'onglet) ne laisse jamais deux onglets pour le même fichier : un onglet déjà ouvert sur ce fichier est fermé.
- **Premier lancement et réseau.** Le premier démarrage contacte GitHub pour récupérer le projet de démonstration (chapitre 4) ; retirez le projet de la liste « Projets GitHub » de **⚙ Préférences** pour arrêter cela une fois pour toutes.

- **Lire le clavier avec le G2 Basic.** Une touche frappée pendant qu'un programme tourne l'arrête, comme avec l'original, et `INPUT` attend. Un jeu qui doit surveiller les touches appelle `KEY(0)` dans sa boucle principale (chapitre 15) ; dès qu'il l'a fait, une touche n'arrête plus le programme.
- **Étiquettes et points d'arrêt.** Un point d'arrêt sur une étiquette qui n'existe pas encore (tapé avant d'assembler ou de lancer) est gardé et posé quand l'étiquette apparaît.
- **Dispositions et fenêtres.** Une disposition enregistrée retient les panneaux ancrés, les fenêtres détachées (place et taille) et la taille de la fenêtre principale. Elle ne retient pas quelle fenêtre a le clavier.

ANNEXE G

Licence et crédits

Retro Dev Kit Apple 1 est un logiciel libre, publié sous la **Licence publique générale GNU, version 3** (GPL-3.0), sans aucune garantie. Le texte complet est dans le fichier `LICENSE` de l'archive et du dépôt.

Certaines parties viennent d'autres projets et gardent leur propre licence, la licence **MIT**, compatible avec la GPL ; elles sont listées dans le fichier `NOTICE` :

- le **RDK G2 Basic**, le BASIC intégré, écrit pour ce projet (dépôt `Old-Wirehead/rdk-gen2basic-apple1`, MIT) ;
- les **tests GEN2 d'Uncle Bernie** (`gen2-test1` à `gen2-test4`, les barres) et les **glyphes de la GEN2**, fournis par Uncle Bernie.

Les jeux de la bibliothèque écrits par **Claude IA** (Le Pendu, Les Allumettes, La Bataille navale, Tunnel et Morpion) sont distribués avec le projet.

Ce que le kit ne contient **pas** : l'Integer BASIC d'Apple, les cassettes du commerce et les démos de la carte P-LAB. Elles restent à leurs propriétaires ; vous apportez vos propres copies.

La carte P-LAB est une création de P-LAB ; le TMS9918A est une puce de Texas Instruments. Ce projet n'est affilié ni à Apple, ni à Texas Instruments, ni à P-LAB.

Index des commandes

#

📼 Exporter WAV 42
▶ Lancer 42
>255 ERR 80
>32767 ERR 80
>8 FORS ERR 80
>8 GOSUBS ERR 80

A

ABS 73
adr.adr 124
adrR 124
AND 74
ASC 73
ascii 60
.asciiz 98
asm (ligne de commande) 60, 96
AUTO 69

B

BAD BRANCH ERR 80
BAD NEXT ERR 80
BAD RETURN ERR 80
BASIC 11
basic (ligne de commande) 60
.byte 98

C

C100R 30
$C250-$C257 34
CALL 73
cassette 59
Charger 11
charger 59
CLR 69
convert 60

D

DEL 69
DIM 72
DIM ERR 80
double-clic 23
DSP 101

E

E000R 68
E2B3R 68
ECHO 103
END 73
END ERR 80
enregistrer 59
étiquettes anonymes 97
étiquettes locales 97
export 59

F

FOR 72

G

GETLINE 103
GOSUB 72
GOTO 72

H

HIMEM 69, 78

I

IF 72
.if 98
import 60
.include 98
info 60
INPUT 72
instructions (6502) 92

K

KBD 101
KBDCR 101

L

LEN 73
LET 72
LIST 69
LOMEM 69, 78

M

.macro 98
MAN 69
MEM FULL ERR 80
MOD 74

N

NEXT 72
NOT 74

O

OR 74
.org 98

P

Par-dessus 45
Pas (débogueur) 45
PEEK 73
POKE 73
portage 88
PP (pointeur de programme) 78
PRBYTE 103
PRHEX 103
PRINT 72
prom 61

R

R (lire) 30
RANGE ERR 80
registres (6502) 91
REM 73
.repeat 98
.res 98
RETYPE LINE 80
RND 73
RUN 69

S

Sauver 11
sauver 59
SCR 69
SCRN 73
.setcpu 98
SGN 73
Sortir 45
STOPPED AT 80
STR OVFL ERR 80
STRING ERR 80
SYNTAX ERR 80

T

TAB 73
Tester le dernier build 25
TOO LONG ERR 80
TOO MANY PARENS ERR 80

U

USR 73

W

W (écrire) 30
.word 98